

Royaume du Maroc
Ministère de l'Éducation Nationale du préscolaire & des sports

Académie régionale de Fès - Meknès
Direction provinciale de TAZA
Centre de préparation Brevet de Technicien Supérieur

RAPPORT DE PFE

Conception et réalisation d'une plateforme de mise en relation entre freelances et clients

Réalisé par :

EL MERNISSI Ayoub
EL AZZOUZI Omar

Encadré par :

Hamid EL BOURAKKADI

Centre de formation :

Lycée Technique - Taza

2025 – 2026

Remerciement

Nous tenons à exprimer nos plus sincères remerciements à **M. Hamid EL BOURAKKADI**, notre encadrant pédagogique, pour son accompagnement tout au long de ce projet. Sa disponibilité, ses conseils et son soutien ont été précieux pour mener à bien ce travail.

Nous remercions également l'ensemble de l'équipe pédagogique de la filière **Développement Web Full Stack** du centre BTS de Taza pour les connaissances et compétences transmises durant ces deux années de formation.

Un remerciement spécial à nos **familles** et nos **proches** pour leur patience, leur soutien moral constant et leurs encouragements, sans lesquels ce projet n'aurait pas été possible.

Enfin, nous remercions toutes les personnes qui ont contribué de près ou de loin à la réussite de ce projet.

Résumé

Ce projet consiste en la conception et le développement d'une **plateforme web de mise en relation entre freelances et clients**. L'objectif principal est de faciliter la publication de services, la recherche de prestataires et la gestion des missions en ligne. La plateforme permet aux utilisateurs de créer un profil, de proposer ou rechercher des services, et de gérer les missions avec un suivi clair et efficace.

Sur le plan technique, l'application repose sur une architecture découplée : une **API REST Laravel 12** côté serveur, une **interface React 19** côté client, et une base de données **MySQL**. L'authentification s'appuie sur Laravel Sanctum, renforcée par la double authentification (2FA). Le module de paiement implémente un système de **séquestre (escrow)** sécurisé intégré à Stripe. La plateforme intègre également une **messagerie temps réel** (WebSockets) et des fonctions d'**intelligence artificielle**.

Ce travail inclut l'analyse des besoins, la conception de l'architecture du système, le développement complet de l'application et la validation des fonctionnalités. L'ensemble du projet vise à offrir une solution pratique, intuitive et sécurisée pour les freelances et leurs clients, améliorant ainsi l'efficacité des collaborations en ligne.

Mots-clés : marketplace, freelance, séquestre, Laravel, React, MySQL, Sanctum, Stripe, WebSocket, API REST, paiement sécurisé, intelligence artificielle.

Abstract

This project focuses on the design and development of a **web platform connecting freelancers with clients**. The main objective is to simplify service posting, freelancer search, and online project management.

The platform allows users to create profiles, offer or search for services, and manage tasks with clear and effective tracking. Technically, the application is built on a decoupled architecture: a **Laravel 12 REST API** on the server side, a **React 19** single-page application on the client side, and a **MySQL** database. Authentication uses Laravel Sanctum with two-factor authentication (2FA). Payments are secured through an **escrow system** integrated with Stripe, alongside **real-time messaging** (WebSockets) and **AI-powered** features.

The work includes requirements analysis, system architecture design, application development, and functional validation. Overall, the project aims to provide a practical, intuitive, and accessible solution for freelancers and clients, enhancing the efficiency of online collaborations.

Keywords: marketplace, freelance, escrow, Laravel, React, MySQL, Sanctum, Stripe, WebSocket, REST API, secure payment, artificial intelligence.

Sommaire

Introduction générale	14
Chapitre 1 : Contexte général et conduite du projet	16
I. Cadre du projet	16
II. Le marché du travail indépendant	16
III. Problématique	17
IV. Étude de l'existant	18
1 Plateformes de référence	18
2 Tableau comparatif	19
V. Solution proposée : la plateforme Panda	19
1 Objectif principal	19
2 Objectifs spécifiques	20
3 Périmètre du projet	20
4 Modèle économique	20
5 Priorisation des fonctionnalités (MoSCoW)	21
VI. Méthodologie de conduite de projet	21
1 Principes appliqués	21
2 Découpage en sprints	22
3 Gestion des risques	22
VII. Planification — diagramme de Gantt	22
VIII. Conclusion du chapitre	23
Chapitre 2 : Analyse et spécification des besoins	24
I. Identification des acteurs	24
1 Acteurs humains	24
2 Acteurs externes (systèmes tiers)	24
II. Besoins fonctionnels	25
1 Authentification et compte	25
2 Offres, propositions et profils	25
3 Catalogue de services	25
4 Contrats et collaboration	26

5	<i>Paielements et finance</i>	26
6	<i>Communication, IA et administration</i>	26
III.	Besoins non fonctionnels	27
IV.	Règles de gestion	27
V.	Langage de modélisation UML adopté	29
VI.	Diagramme de cas d'utilisation global	29
a	<i>Partie 1 — Authentification, Administrateur & Annonces</i>	29
b	<i>Partie 2 — Catalogue, Contrats, Paiements & Communication</i>	31
c	<i>Partie 3 — Notifications, Agences, IA & Marketplace</i>	33
1	<i>Module I — Authentification et gestion du compte</i>	35
2	<i>Module II — Espace Administrateur</i>	35
3	<i>Module III — Annonces et Freelances</i>	36
4	<i>Module IV — Freelance et Services (Catalogue)</i>	36
5	<i>Module V — Propositions et Offres</i>	37
6	<i>Module VI — Gestion des Paiements (Séquestre / Escrow)</i>	37
7	<i>Module VII — Dialogue Structuré (Messagerie et Contrats)</i>	38
8	<i>Module VIII — Agences et Talents</i>	38
9	<i>Module IX — Notifications</i>	38
VII.	Description textuelle des cas d'utilisation	39
1	<i>Cas « S'authentifier »</i>	39
2	<i>Cas « Publier une offre »</i>	39
3	<i>Cas « Postuler à une offre »</i>	39
4	<i>Cas « Financer le séquestre et valider un jalon »</i>	40
5	<i>Cas « Vendre un service et passer commande »</i>	40
6	<i>Cas « Demander un retrait »</i>	40
7	<i>Cas « Vérifier l'identité (KYC) »</i>	41
8	<i>Cas « Gérer une agence »</i>	41
9	<i>Matrice de traçabilité des exigences</i>	41
VIII.	Conclusion du chapitre	42
Chapitre 3 : Conception de la solution		43
I.	Architecture globale	43
II.	Architecture logicielle en couches	44

1	<i>Vue en paquetages</i>	45
III.	Sécurité et authentification	46
IV.	Diagramme de classes	47
a	<i>Sous-système 1 — Identité, Sécurité & Finance</i>	48
b	<i>Sous-système 2 — Marketplace, Contrats & Communication</i>	50
c	<i>Sous-système 3 — Catalogue, Agences & Talents</i>	52
1	<i>Groupe Identité et Authentification</i>	54
2	<i>Groupe Marketplace (Offres et Propositions)</i>	55
3	<i>Groupe Contrats et Jalons</i>	55
4	<i>Groupe Finance (Wallet et Transactions)</i>	55
5	<i>Groupe Communication et Évaluations</i>	56
V.	Diagrammes de séquence dynamiques	56
1	<i>Financement du séquestre et libération de jalon</i>	56
2	<i>Dépôt de fonds via Stripe</i>	58
3	<i>Génération de proposition par IA</i>	59
VI.	Choix technologiques	60
1	<i>Environnement et outils de développement</i>	61
2	<i>Pourquoi Laravel et React ?</i>	62
VII.	Patterns de conception appliqués	63
1	<i>Pattern Repository — abstraction de la persistance</i>	63
2	<i>Pattern Service Layer — isolation de la logique métier</i>	64
3	<i>Pattern Observer — découplage événementiel</i>	64
4	<i>Pattern Strategy — algorithmes interchangeables</i>	65
5	<i>Pattern Factory — construction d'objets complexes</i>	65
VIII.	Conclusion du chapitre	66
Chapitre 4 : Réalisation de l'application		67
I.	Environnement de développement	67
II.	Organisation du code source	67
1	<i>Cartographie des principaux points d'API</i>	68
2	<i>Conventions et bonnes pratiques</i>	69
3	<i>Modèles, relations et migrations</i>	69
III.	Le cœur financier : le service de grand livre (LedgerService)	70

1	Financement du séquestre	70
2	Libération d'un jalon et répartition	71
IV.	Authentification et sécurité	71
V.	Communication temps réel : la messagerie	73
VI.	Intégration des paiements Stripe	73
VII.	Intelligence artificielle	74
VIII.	Présentation des interfaces utilisateur — 57 pages	75
1	Module d'authentification — 6 pages	75
2	Pages publiques et marketing — 15 pages	78
3	Tableaux de bord — 3 pages	82
4	Place de marché des offres — 8 pages	85
5	Contrats et jalons — 3 pages	88
6	Paielements et finances — 6 pages	90
7	Talents et profils freelance — 3 pages	93
8	Gestion des talents — 3 pages	95
9	Paramètres et sécurité du compte — 3 pages	96
10	Autres fonctionnalités transversales — 6 pages	98
11	Design responsive et système de composants	102
IX.	Gestion du code source avec GitHub	102
1	Structure et statistiques du dépôt	103
2	Stratégie de branches — Git Flow simplifié	103
3	Conventions de commit (Conventional Commits)	104
4	Pipeline d'intégration continue — GitHub Actions	104
5	Releases et gestion des versions (SemVer)	105
X.	Gestion de projet avec Jira	106
1	Méthodologie Scrum appliquée au projet	106
2	Structure du backlog et Épics	107
3	Planification des sprints — 6 sprints de 2 semaines	108
4	Workflow des tickets et types de travaux	109
5	Tableau de bord Jira — métriques finales du projet	110
XI.	Conclusion du chapitre	111

Chapitre 5 : Tests, sécurité et déploiement 112

I. Stratégie de tests	112
II. Validation du moteur financier	113
III. Scénarios de tests fonctionnels	114
IV. Tests de sécurité API	116
V. Synthèse de la sécurité	117
VI. Déploiement	118
1 Conteneurs Docker et variables d'environnement	119
2 Optimisations de performance	120
3 Intégration et déploiement continu	121
VII. Conclusion du chapitre	121
Chapitre 6 : Bilan et perspectives	122
I. Bilan des compétences techniques acquises	122
II. Compétences transversales	122
III. Difficultés rencontrées	123
IV. Contraintes et limites	123
V. Perspectives d'évolution	124
VI. Synthèse du projet	124
VII. Conclusion du chapitre	125
Conclusion générale	126
Glossaire	128
Références bibliographiques et webographie	129
Annexes	130
A Configuration de l'environnement	130
B Exemple de réponse de l'API	130
C Principaux services métier	131
D Commandes utiles	131
F Liste complète des migrations de la base de données	131
E Extrait du script de création des tables financières	133
G Catalogue complet des points d'API REST	133
H Résumé des tests automatisés	135

Table des figures

Figure 1: Diagramme de Gantt — planification prévisionnelle du projet Panda	23
Figure 2: Diagramme de cas d'utilisation — Partie 1 : Authentification, Administrateur & Annonces	30
Figure 3: Diagramme de cas d'utilisation — Partie 2 : Catalogue, Contrats, Paiements & Communication	32
Figure 4: Diagramme de cas d'utilisation — Partie 3 : Notifications, IA, Agences & Marketplace	34
Figure 5: Architecture générale en trois couches et services externes	43
Figure 6: Organisation en couches du back-end (MVC enrichi de services métier)	45
Figure 7: Diagramme de séquence — authentification (Sanctum + 2FA)	47
Figure 8: Diagramme de classes — Sous-système 1 : Identité, Sécurité & Finance	49
Figure 9: Diagramme de classes — Sous-système 2 : Marketplace, Contrats & Communication _s	51
Figure 10: Diagramme de classes — Sous-système 3 : Catalogue, Agences & Talents	53
Figure 11: Diagramme de séquence — financement du séquestre et libération de jalon	57
Figure 12: Diagramme de séquence — dépôt de fonds via Stripe	59
Figure 13: Diagramme de séquence — génération de proposition assistée par IA	60
Figure 14: Écran de connexion — e-mail/mot de passe et bouton OAuth Google	76
Figure 15: Écran d'inscription — choix du rôle et saisie des informations	77
Figure 16: Page d'accueil de la plateforme Panda	80
Figure 17: Tableau de bord du client — solde, offres actives et contrats	83
Figure 18: Tableau de bord du freelance — gains, propositions et missions	84
Figure 19: Tableau de bord administrateur — indicateurs de la plateforme	85
Figure 20: Place de marché des offres — filtres, recherche et cartes d'offres	86
Figure 21: Détail d'un contrat — onglets jalons, fichiers et suivi du temps	89
Figure 22: Portefeuille utilisateur — soldes, transactions et retraits	91
Figure 23: Place de marché des freelances — cartes avec compétences et tarifs	94
Figure 24: Messagerie temps réel — liste de conversations et fenêtre de chat	99
Figure 25: Assistant d'intelligence artificielle — interface conversationnelle	100
Figure 26: Diagramme de déploiement — conteneurs Docker	119

Liste des tableaux

Tableau 1: Attentes croisées des clients et des freelances	17
Tableau 2: Positionnement comparatif de Panda face aux plateformes de référence	19
Tableau 3: Objectifs spécifiques du projet et bénéfices attendus	20
Tableau 4: Priorisation MoSCoW des fonctionnalités	21
Tableau 5: Découpage du projet en sprints	22
Tableau 6: Principaux risques du projet et mesures de réduction	22
Tableau 7: Acteurs humains de la plateforme Panda	24
Tableau 8: Acteurs externes intégrés à la plateforme	24
Tableau 9: Besoins non fonctionnels de la plateforme	27
Tableau 10: Règles de gestion de la plateforme Panda (RG-01 à RG-20)	29
Tableau 11: Matrice de traçabilité des exigences vers les composants	42
Tableau 12: Synthèse des couches de l'architecture	44
Tableau 13: Décomposition logique en paquetages	46
Tableau 14: Classes du sous-système Identité, Sécurité & Finance	50
Tableau 15: Classes du sous-système Marketplace, Contrats & Communication	52
Tableau 16: Classes du sous-système Catalogue, Agences & Talents	54
Tableau 17: Classes du groupe Identité	54
Tableau 18: Classes du groupe Marketplace	55
Tableau 19: Classes du groupe Contrats et Jalons	55
Tableau 20: Classes du groupe Finance	56
Tableau 21: Classes du groupe Communication et Évaluations	56
Tableau 22: Pile technologique back-end	61
Tableau 23: Pile technologique front-end	61
Tableau 24: Services tiers et IA intégrés	61
Tableau 25: Outils et environnement de développement du projet Panda	62
Tableau 26: Exemples d'application du pattern Repository via Eloquent	64
Tableau 27: Services métier du pattern Service Layer	64
Tableau 28: Événements et écouteurs — pattern Observer	65

Tableau 29: Applications du pattern Strategy dans Panda	65
Tableau 30: Applications du pattern Factory dans Panda	66
Tableau 31: Environnement et outils de développement	67
Tableau 32: Extrait de l'API — authentification, offres et catalogue	68
Tableau 33: Extrait de l'API — contrats et jalons	68
Tableau 34: Extrait de l'API — paiements, IA et administration	69
Tableau 35: Services d'intelligence artificielle de la plateforme	75
Tableau 36: Pages du module d'authentification	76
Tableau 37: Pages publiques et marketing de la plateforme Panda	79
Tableau 38: Tableaux de bord par rôle utilisateur	83
Tableau 39: Pages du module Offres et propositions	86
Tableau 40: Pages du module Contrats et jalons	89
Tableau 41: Pages du module Paiements et finances	91
Tableau 42: Pages du module Talents et profils freelance	93
Tableau 43: Pages du module Gestion des talents	95
Tableau 44: Pages du module Paramètres et sécurité du compte	97
Tableau 45: Pages transversales de l'application Panda	98
Tableau 46: Principes de design et mise en oeuvre technique	102
Tableau 47: Statistiques du dépôt GitHub du projet Panda	103
Tableau 48: Rôles et règles de protection des branches Git	104
Tableau 49: Types de commits Conventional Commits et impact sur le changelog	104
Tableau 50: Étapes du pipeline CI GitHub Actions	105
Tableau 51: Historique des releases GitHub du projet Panda	106
Tableau 52: Cérémonies Scrum appliquées au projet Panda	107
Tableau 53: Épics du backlog Jira avec estimation totale en Story Points	108
Tableau 54: Planification et vélocité des sprints du projet Panda	109
Tableau 55: Types de tickets Jira et exemples du projet Panda	110
Tableau 56: Métriques finales du tableau de bord Jira — projet Panda	110
Tableau 57: Couverture des tests par domaine fonctionnel	113
Tableau 58: Échantillon des cas de test du moteur financier	114

Tableau 59: Scénarios de tests fonctionnels manuels — résultats de recette	116
Tableau 60: Tests de sécurité API — résultats de la batterie de tests	117
Tableau 61: Synthèse des mesures de sécurité (défense en profondeur)	118
Tableau 62: Services Docker Compose de la plateforme Panda	120
Tableau 63: Optimisations de performance et impacts mesurés	120
Tableau 64: Axes d'évolution envisagés pour la plateforme Panda	124
Tableau 65: Récapitulatif des objectifs initiaux et de leur réalisation	125
Tableau 66: Glossaire des principaux termes	128
Tableau 67: Principaux services métier de la plateforme	131
Tableau 68: Liste des migrations et tables correspondantes	133
Tableau 69: Catalogue des principaux points d'API REST de Panda	135
Tableau 70: Résumé de la couverture et des résultats de la suite de tests automatisés	135

Introduction générale

La transformation numérique a profondément bouleversé le monde du travail au cours de la dernière décennie. L'essor du **travail indépendant** — le *freelancing* — en constitue l'une des manifestations les plus marquantes : de plus en plus de professionnels choisissent de proposer leurs compétences à distance, tandis que les entreprises, de la *start-up* à la grande organisation, externalisent une part croissante de leurs projets vers des talents qualifiés, où qu'ils se trouvent. Cette nouvelle économie, dite *économie des plateformes*, repose sur des places de marché numériques capables de mettre en relation l'offre et la demande de compétences, tout en sécurisant la transaction qui les unit.

Cependant, cette mise en relation soulève une difficulté centrale : la **confiance**. Comment un client peut-il être certain de ne payer que pour un travail effectivement livré et conforme ? Comment un freelance peut-il être assuré d'être rémunéré une fois la prestation réalisée ? Sur un marché où les deux parties ne se connaissent pas et ne se rencontrent jamais physiquement, l'absence de tiers de confiance constitue un frein majeur. Les plateformes internationales de référence (Upwork, Fiverr, Malt, Freelancer.com) ont répondu à ce problème par des mécanismes de **séquestre de fonds** (escrow), de jalons de paiement et de gestion des litiges, devenus aujourd'hui des standards incontournables du secteur.

C'est dans ce contexte que s'inscrit notre projet de fin d'études : la conception et le développement de **Panda**, une plateforme web complète de mise en relation entre freelances et clients. Loin de se limiter à un simple annuaire ou à un formulaire de contact, Panda ambitionne de couvrir l'intégralité du cycle de vie d'une collaboration : depuis la découverte d'un talent ou d'une offre, jusqu'au versement final des fonds, en passant par la contractualisation, le suivi des livrables et la communication. Le projet met un soin particulier à reproduire, dans un cadre académique, les briques techniques les plus exigeantes d'une telle plateforme — au premier rang desquelles un **moteur de paiement par séquestre fiable et auditable**.

Ce travail nous a permis de mobiliser et d'approfondir l'ensemble des compétences acquises durant notre formation : modélisation et conception logicielle (UML, Merise), développement back-end avec le framework **Laravel**, développement front-end avec la bibliothèque **React**, conception de bases de données relationnelles avec **MySQL**, sécurité

applicative, intégration de services tiers (Stripe, OAuth, intelligence artificielle) et communication temps réel. Il nous a également confrontés aux réalités d'un projet d'envergure : arbitrages de périmètre, gestion de la complexité et exigence de qualité.

Le présent rapport rend compte de la démarche suivie et des résultats obtenus. Il s'organise en **six chapitres**. Le **premier** présente le contexte général du projet, la problématique, l'étude de l'existant et la méthodologie de conduite adoptée. Le **deuxième** est consacré à l'analyse et à la spécification des besoins, formalisées à l'aide du langage UML. Le **troisième** détaille la conception de la solution : architecture, sécurité, modélisation objet, diagrammes dynamiques et conception de la base de données. Le **quatrième** décrit la réalisation effective, en s'appuyant sur des extraits de code significatifs et une présentation des interfaces. Le **cinquième** aborde les tests, la sécurité et le déploiement. Le **sixième**, enfin, dresse le bilan du projet, ses limites et ses perspectives d'évolution, avant une conclusion générale.

Chapitre 1 : Contexte général et conduite du projet

Ce premier chapitre pose les fondations du projet. Il en précise le cadre académique, présente le domaine métier — le marché du travail indépendant — et la problématique à laquelle Panda entend répondre. Il propose ensuite une étude des solutions existantes, définit les objectifs et le périmètre de notre plateforme, et expose enfin la méthodologie de conduite et la planification que nous avons adoptées pour mener le projet à bien.

I. Cadre du projet

Ce projet s'inscrit dans le cadre de la formation **BTS — Développement Digital, option Web Full Stack**, une filière professionnalisante de deux années post-baccalauréat dispensée au Centre de Préparation au BTS du Lycée Technique de Taza. Le projet de fin d'études (PFE) en constitue l'aboutissement : il a pour vocation de placer l'étudiant en situation réelle de conception et de développement d'une application complète, depuis l'analyse du besoin jusqu'à la livraison d'un produit fonctionnel.

Au-delà de la simple démonstration de compétences techniques, ce travail vise à développer des qualités professionnelles essentielles : l'analyse et la formalisation d'un besoin, la planification, la prise de décision en environnement contraint, l'autonomie et la rigueur. Nous avons délibérément choisi un sujet ambitieux — une place de marché transactionnelle — afin de nous confronter aux problématiques réelles d'une application professionnelle moderne : sécurité, intégrité des données financières, montée en charge et expérience utilisateur.

II. Le marché du travail indépendant

Le *freelancing* désigne l'exercice d'une activité professionnelle de manière indépendante, généralement à la prestation ou au projet, sans lien de subordination salariale. Porté par la généralisation du travail à distance, par la flexibilité recherchée tant par les professionnels que par les entreprises, et par la maturité des outils numériques, ce mode de travail connaît une croissance soutenue à l'échelle mondiale. Les domaines concernés sont nombreux : développement informatique, design graphique, rédaction et traduction, marketing digital, conseil, montage vidéo, etc.

Ce marché met en présence deux grandes catégories d'acteurs. D'un côté, les **clients** (particuliers, entrepreneurs, entreprises) qui cherchent à confier une mission précise à un professionnel compétent, dans un délai et un budget maîtrisés. De l'autre, les **freelances** qui souhaitent trouver des missions, valoriser leur expertise, gérer leur activité et être rémunérés de façon fiable. Entre les deux s'interpose le besoin d'une **plateforme de confiance**, capable d'organiser la rencontre, d'encadrer la collaboration et — point crucial — de sécuriser le paiement.

Deux grands modèles de collaboration coexistent sur ce marché. Le premier, dit « *appel d'offres* », part du **besoin du client** : celui-ci publie une mission, reçoit des candidatures (propositions) et sélectionne le freelance le plus adapté. Le second, dit « *service packagé* », part de l'**offre du freelance** : celui-ci propose des prestations standardisées à prix fixe que le client achète directement. Panda a fait le choix d'intégrer ces **deux modèles** au sein d'une même plateforme, afin de couvrir l'ensemble des situations et de maximiser les opportunités de mise en relation.

Le tableau suivant met en regard les attentes des deux profils d'utilisateurs, qui structurent directement les fonctionnalités de la plateforme.

Attentes du client	Attentes du freelance
Trouver rapidement un profil qualifié	Accéder à un flux régulier de missions
Comparer compétences, avis et tarifs	Valoriser son expertise et son portfolio
Maîtriser le budget et les délais	Être certain d'être payé pour son travail
Ne payer que pour un travail conforme	Gérer simplement son activité et ses revenus
Communiquer et suivre l'avancement	Encadrer la relation par un contrat clair

Tableau 1: Attentes croisées des clients et des freelances

III. Problématique

La mise en relation, à elle seule, ne suffit pas. Le véritable défi d'une marketplace de services réside dans la **gestion de la confiance** entre des parties qui ne se connaissent pas. Plusieurs risques se posent concrètement :

- **Risque pour le client** : payer d'avance et ne jamais recevoir le travail, ou recevoir une prestation non conforme à ce qui avait été convenu ;

- **Risque pour le freelance** : livrer un travail conséquent et ne jamais être payé, ou subir des retards de paiement répétés ;
- **Risque d'intégrité financière** : sur une plateforme manipulant l'argent de tiers, la moindre erreur de calcul, double paiement ou incohérence de solde est inacceptable ;
- **Risque de communication** : l'absence d'un canal d'échange fiable et tracé entre les parties favorise les malentendus et complique la résolution des litiges ;
- **Risque de sécurité** : usurpation d'identité, comptes frauduleux, accès non autorisé aux données personnelles et financières.

La problématique centrale de ce projet peut donc se formuler ainsi : *comment concevoir une plateforme web qui mette efficacement en relation freelances et clients, tout en garantissant la sécurité financière, l'intégrité des transactions et la confiance entre des parties distantes ?* C'est à cette question que Panda apporte une réponse, en plaçant le **séquestre de fonds (escrow)** et la traçabilité comptable au cœur de son architecture.

IV. Étude de l'existant

Avant de concevoir notre solution, nous avons analysé les principales plateformes du marché afin d'en dégager les bonnes pratiques et les standards attendus. Quatre acteurs de référence ont été étudiés.

1 Plateformes de référence

- **Upwork** : leader mondial orienté missions à l'heure ou au forfait. Propose des contrats, un suivi du temps, un séquestre, des jalons et une facturation hebdomadaire. Modèle de référence pour la partie « *offres et propositions* » ;
- **Fiverr** : centré sur des **services packagés** (« gigs ») proposés par les freelances avec des paliers de prix (basique, standard, premium). Modèle de référence pour la partie « *catalogue de services* » ;
- **Malt** : plateforme européenne premium ciblant les profils tech et créatifs, avec un fort accent sur la qualité des profils et la facturation ;
- **Freelancer.com / ComeUp** : places de marché généralistes combinant appels d'offres et services packagés.

De cette analyse, nous retenons que les fonctionnalités jugées indispensables par le marché sont : un système de profils riches, une recherche performante, un mécanisme de candidature, un **paiement sous séquestre avec jalons**, une messagerie intégrée, un système d'évaluation,

et une gestion des litiges. Panda s'inspire de ces standards en combinant les deux grands modèles — *offres/propositions* (façon Upwork) et *services packagés* (façon Fiverr) — au sein d'une même plateforme.

2 Tableau comparatif

Le tableau suivant synthétise le positionnement de Panda par rapport aux solutions existantes.

Fonctionnalité	Upwork	Fiverr	Panda
Offres + propositions (appels d'offres)	Oui	Non	Oui
Services packagés (catalogue à paliers)	Non	Oui	Oui
Païement sous séquestre (escrow)	Oui	Oui	Oui
Jalons de paiement	Oui	Limité	Oui
Suivi du temps & facturation horaire	Oui	Non	Oui
Messagerie temps réel intégrée	Oui	Oui	Oui
Assistant IA (propositions, matching)	Partiel	Partiel	Oui
Gestion d'agences	Oui	Non	Oui
Vérification d'identité (KYC) & 2FA	Oui	Oui	Oui
Code source maîtrisé (projet académique)	Non	Non	Oui

Tableau 2: Positionnement comparatif de Panda face aux plateformes de référence

Le parti pris de Panda est de **réunir en une seule plateforme** les deux grands modèles du marché (appels d'offres et services packagés) et d'en maîtriser intégralement la chaîne technique, jusqu'au moteur de paiement, dans un cadre pédagogique.

V. Solution proposée : la plateforme Panda

Panda est une plateforme web full-stack qui couvre l'ensemble du cycle de collaboration entre freelances et clients. Elle se distingue par la richesse de son périmètre fonctionnel et par la robustesse de son socle technique, en particulier financier.

1 Objectif principal

Concevoir et développer une place de marché numérique sécurisée permettant à des clients de publier des besoins et d'acheter des services, à des freelances de proposer leurs

compétences et d'être rémunérés de manière fiable, le tout encadré par un système de paiement sous séquestre garantissant l'intérêt des deux parties.

2 Objectifs spécifiques

Objectif	Bénéfice attendu
Authentification robuste (Sanctum, 2FA, OAuth)	Sécuriser l'accès et l'identité des utilisateurs
Profils riches client / freelance / agence	Valoriser les compétences et inspirer confiance
Offres, propositions et catalogue de services	Couvrir les deux modèles du marché
Contrats, jalons, suivi du temps et fichiers	Encadrer et tracer la collaboration
Moteur de paiement par séquestre (escrow)	Garantir la sécurité financière et la confiance
Messagerie temps réel	Fluidifier la communication entre les parties
Assistant IA & vérification d'identité (KYC)	Améliorer la productivité et la sécurité
Back-office d'administration & de modération	Piloter, modérer et arbitrer les litiges

Tableau 3: Objectifs spécifiques du projet et bénéfices attendus

3 Périmètre du projet

Périmètre fonctionnel. Inscription et authentification multi-rôles ; gestion des profils ; publication d'offres et soumission de propositions ; catalogue de services packagés et commandes ; contrats, jalons, fichiers, suivi du temps et extensions ; portefeuille, séquestre, retraits et facturation ; messagerie temps réel ; évaluations ; notifications ; intelligence artificielle ; agences ; vérification d'identité ; administration.

Périmètre technique. API REST Laravel 12 (PHP 8.2), SPA React 19 (Vite, TailwindCSS), base de données MySQL, authentification Sanctum, intégration Stripe, WebSockets via Laravel Reverb, et un serveur de modèle de langage (Ollama/Mistral) pour les fonctions d'IA.

Hors périmètre. Application mobile native, internationalisation multi-devises avancée, intégrations comptables externes et programmes de fidélité ne font pas partie de la version présentée, mais sont envisagés comme perspectives d'évolution (cf. chapitre 6).

4 Modèle économique

Comme les plateformes de référence, Panda repose sur un modèle de **commission**. La plateforme se rémunère à la libération des fonds, en prélevant un pourcentage sur le montant versé au freelance (commission paramétrable, fixée à 10 % par défaut). S'y ajoutent des frais

fixes sur les retraits et, optionnellement, des **abonnements** offrant des avantages (mise en avant, crédits de candidature). Ce modèle aligne l'intérêt de la plateforme sur la réussite des transactions : elle ne gagne que lorsque la collaboration aboutit. Les paramètres financiers sont centralisés dans la table `platform_settings`, modifiables par l'administrateur sans redéploiement.

5 Priorisation des fonctionnalités (MoSCoW)

Pour piloter le périmètre, nous avons priorisé les fonctionnalités selon la méthode **MoSCoW** (*Must, Should, Could, Won't*), qui distingue l'essentiel du souhaitable.

Priorité	Fonctionnalités
Must (indispensable)	Authentification, offres/propositions, contrats, jalons, séquestre, libération, portefeuille, messagerie
Should (important)	Catalogue de services, retraits Stripe, évaluations, notifications, vérification d'identité, recherche
Could (souhaitable)	Assistant IA, agences, listes de talents, suivi du temps, abonnements
Won't (hors version)	Application mobile native, multi-devises, intégrations comptables externes

Tableau 4: Priorisation MoSCoW des fonctionnalités

VI. Méthodologie de conduite de projet

Compte tenu de l'ampleur du périmètre et de la nature évolutive des besoins, nous avons adopté une démarche **agile et itérative**, inspirée de Scrum mais adaptée aux contraintes d'un projet académique. Plutôt que de figer l'intégralité des spécifications en amont (approche en cascade), nous avons découpé le travail en **incréments fonctionnels** livrables et testables, organisés en *sprints* d'environ deux à trois semaines.

1 Principes appliqués

- **Développement incrémental** : chaque sprint livre un module complet et fonctionnel (authentification, puis offres, puis contrats, puis paiements, etc.) ;
- **Priorisation par la valeur et le risque** : les briques les plus risquées (le moteur de paiement) ont été traitées tôt et consolidées en priorité ;
- **Intégration continue** : le code est versionné avec **Git**, chaque fonctionnalité étant développée puis intégrée à la branche principale après validation ;
- **Tests au fil de l'eau** : les comportements critiques (séquestre, libération de jalon) sont couverts par des tests automatisés dès leur écriture ;

- **Revue régulière** : un point d'avancement périodique permet de réajuster les priorités et de valider les incréments.

2 Découpage en sprints

Sprint	Objectif principal	Durée
S0 — Cadrage	Analyse des besoins, conception UML, maquettes, mise en place du dépôt	2 sem.
S1 — Socle	Authentification, modèles, migrations, profils, sécurité	3 sem.
S2 — Marketplace	Offres, propositions, recherche, catalogue de services	3 sem.
S3 — Collaboration	Contrats, jalons, fichiers, suivi du temps, extensions	3 sem.
S4 — Paiements	Portefeuille, séquestre, LedgerService, Stripe, retraits	3 sem.
S5 — Communication & IA	Messagerie temps réel, IA, catalogue, agences	3 sem.
S6 — Qualité & livraison	Tests, sécurité, dockerisation, rapport, soutenance	3 sem.

Tableau 5: Découpage du projet en sprints

3 Gestion des risques

Tout projet comporte des risques. Les avoir identifiés en amont nous a permis d'anticiper des mesures de réduction et de protéger l'avancement.

Risque	Impact	Mesure de réduction
Complexité du moteur de paiement	Élevé	Traitement prioritaire, tests approfondis, garde-fous
Ampleur du périmètre fonctionnel	Élevé	Découpage en sprints, priorisation MoSCoW
Temps limité (parallèle aux cours)	Moyen	Planning Gantt, incréments livrables
Intégration de services tiers	Moyen	Maquettage, environnement de test (clés sandbox)
Régressions lors des évolutions	Moyen	Tests automatisés, versionnement Git

Tableau 6: Principaux risques du projet et mesures de réduction

VII. Planification — diagramme de Gantt

La planification du projet a été formalisée au moyen d'un **diagramme de Gantt**, qui ordonne les grandes phases dans le temps et met en évidence leur durée et leur enchaînement. Cette représentation nous a permis de répartir l'effort de manière équilibrée et de suivre l'avancement par rapport aux échéances. La figure ci-dessous présente le planning prévisionnel.

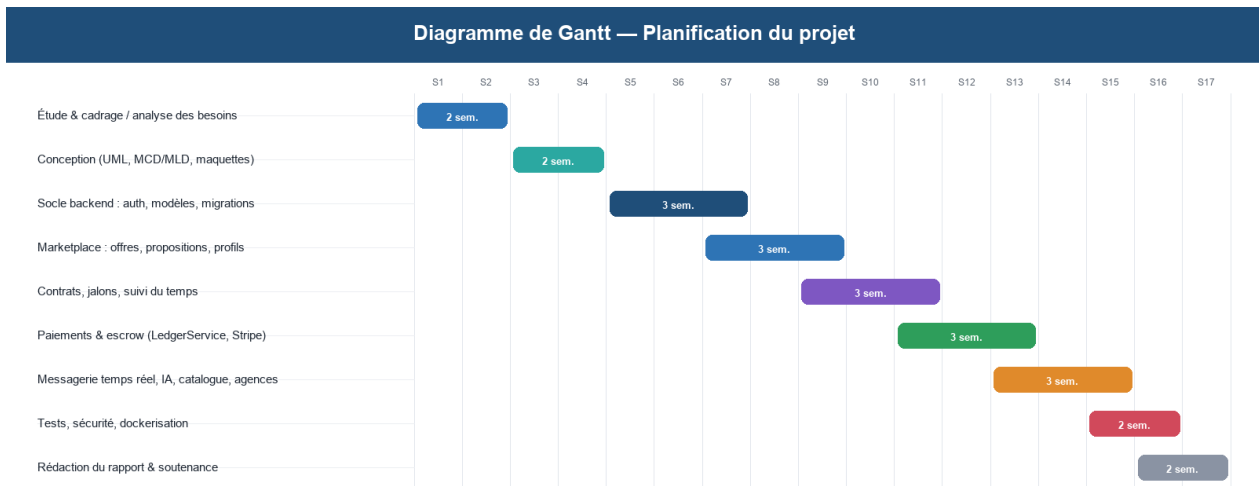


Figure 1: Diagramme de Gantt — planification prévisionnelle du projet Panda

Les phases de conception et de développement du socle ont volontairement été dotées d'une marge confortable, car elles conditionnent l'ensemble des modules suivants. La phase de paiement, identifiée comme la plus critique, a fait l'objet d'une attention particulière et de tests approfondis avant de poursuivre.

VIII. Conclusion du chapitre

Ce premier chapitre a défini le cadre du projet, caractérisé le marché du travail indépendant et formulé la problématique de la confiance, au cœur de toute place de marché de services. L'étude de l'existant a confirmé les standards attendus et précisé le positionnement de Panda, qui réunit appels d'offres et services packagés autour d'un moteur de paiement sécurisé. Enfin, la méthodologie agile et la planification adoptées ont été exposées. Le chapitre suivant approfondit **l'analyse et la spécification des besoins**, formalisées à l'aide du langage UML.

Chapitre 2 : Analyse et spécification des besoins

L'analyse des besoins est une étape déterminante qui conditionne la qualité de la conception et de la réalisation. Ce chapitre identifie les **acteurs** du système, recense les **besoins fonctionnels** et **non fonctionnels**, puis les formalise à l'aide du langage **UML**. Il présente le diagramme de cas d'utilisation global, complété par la description textuelle des cas d'utilisation les plus significatifs.

I. Identification des acteurs

Un acteur représente une entité externe — humaine ou logicielle — qui interagit avec le système. Panda distingue les acteurs humains, porteurs de rôles métier, des acteurs externes que sont les services tiers intégrés.

1 Acteurs humains

Acteur	Description et responsabilités
Visiteur	Internaute non authentifié : parcourt les offres, les freelances et le catalogue, consulte les pages publiques, puis s'inscrit ou se connecte.
Client	Utilisateur authentifié qui publie des offres, étudie les propositions reçues, achète des services, finance le séquestre, valide les jalons et évalue les freelances.
Freelance	Utilisateur authentifié qui complète son profil et son portfolio, postule aux offres, vend des services packagés, livre les jalons, suit son temps et demande ses retraits.
Agence	Entité regroupant plusieurs freelances sous une même bannière ; le propriétaire invite des membres et gère l'activité collective.
Administrateur	Supervise la plateforme : modération du catalogue, gestion des utilisateurs, validation des vérifications d'identité, traitement des retraits et arbitrage des litiges.

Tableau 7: Acteurs humains de la plateforme Panda

2 Acteurs externes (systèmes tiers)

Acteur externe	Rôle dans le système
Stripe	Encaissement des dépôts (Checkout) et versement des retraits (Connect/Payout).
Google (OAuth)	Fournisseur d'identité pour l'authentification sociale via Socialite.
Serveur IA (Ollama / Mistral)	Génération de propositions, mise en correspondance et analyse de profils.
Service de notifications Web Push	Acheminement des notifications navigateur (VAPID).

Tableau 8: Acteurs externes intégrés à la plateforme

II. Besoins fonctionnels

Les besoins fonctionnels décrivent les services que le système doit rendre. Compte tenu de la richesse du périmètre, ils sont présentés par **module fonctionnel**.

1 Authentification et compte

La gestion du compte constitue le point d'entrée de la plateforme et le socle de la confiance. Elle doit être à la fois simple pour l'utilisateur et solide face aux menaces.

- Inscription par e-mail/mot de passe ou via Google (OAuth) ☐
- Connexion sécurisée par jeton et déconnexion ☐
- Vérification de l'adresse e-mail et du numéro de téléphone ☐
- Authentification à deux facteurs (2FA) par code temporel (TOTP) et codes de secours ☐
- Réinitialisation du mot de passe et gestion du profil (avatar, informations) ☐
- Vérification d'identité (KYC) avec pièces justificatives.

2 Offres, propositions et profils

Ce module incarne le premier modèle de mise en relation, à l'initiative du **client** qui exprime un besoin auquel les freelances répondent par des propositions.

- Publication, modification et suppression d'offres par le client ☐
- Recherche et filtrage des offres et des freelances (catégorie, budget, compétences) ☐
- Soumission d'une proposition par le freelance (lettre, montant) ☐
- Acceptation, refus ou retrait d'une proposition ☐
- Gestion du profil freelance : titre, biographie, compétences, portfolio, disponibilité ☐
- Enregistrement de freelances favoris et constitution de listes de talents.

3 Catalogue de services

Le catalogue incarne le second modèle, à l'initiative du **freelance** qui propose des prestations standardisées à prix fixe, que le client achète directement.

- Création par le freelance de services packagés à plusieurs paliers (basique, standard, premium) ☐
- Modération du catalogue par l'administrateur (approbation/rejet) ☐
- Achat d'un palier par le client (commande) ☐

- Livraison, validation et évaluation de la commande.

4 Contrats et collaboration

Une fois l'accord conclu, le **contrat** encadre et trace la collaboration, du premier livrable jusqu'à la clôture, en structurant le travail en jalons mesurables.

- Création automatique d'un contrat à l'acceptation d'une proposition ou d'une commande ☐
- Découpage du contrat en **jalons** (montant, échéance) ☐
- Soumission, approbation ou rejet d'un jalon ☐
- Partage de **fichiers** versionnés liés au contrat ☐
- **Suivi du temps** pour les contrats horaires (démarrage/arrêt, récapitulatif hebdomadaire) ☐
- Demande et réponse d'**extension** de délai ☐
- Suivi d'activité, analyse et génération de **PDF** du contrat.

5 Paiements et finance

Le module financier est le plus **sensible** : il manipule l'argent réel des utilisateurs et matérialise la promesse de confiance de la plateforme à travers le séquestre.

- Portefeuille personnel avec solde disponible, en séquestre et en attente ☐
- Dépôt de fonds via Stripe ☐
- **Financement du séquestre** d'un contrat par le client ☐
- **Libération d'un jalon** : versement au freelance après prélèvement de la commission ☐
- Demande de **retrait** par le freelance et validation/refus par l'administrateur ☐
- Configuration Stripe Connect pour les versements ☐
- Facturation hebdomadaire pour les contrats horaires et abonnements.

6 Communication, IA et administration

Ces modules transverses fluidifient les échanges entre les parties, accélèrent les tâches répétitives grâce à l'IA et outillent la supervision de la plateforme.

- **Messagerie temps réel** : conversations, pièces jointes, accusés de lecture, réactions ☐
- **Notifications** in-app et Web Push ☐

- **Assistant IA** : génération de propositions, mise en correspondance, analyse de profil, recherche intelligente ☐
- **Gestion d'agences** : invitations, membres, transfert de propriété ☐
- **Back-office** : tableau de bord, gestion des utilisateurs, modération, KYC, finance, litiges.

III. Besoins non fonctionnels

Les besoins non fonctionnels caractérisent la **qualité** attendue du système, indépendamment des fonctionnalités. Ils sont au moins aussi importants que les premiers pour une plateforme transactionnelle.

Catégorie	Exigence
Sécurité	Mots de passe hachés (bcrypt), jetons Sanctum, 2FA, vérification d'identité, limitation de débit (throttling), politiques d'autorisation, journal d'audit, vérification de signature des webhooks.
Intégrité financière	Atomicité (transactions SQL), verrouillage (FOR UPDATE), écriture comptable double, idempotence des opérations, précision décimale.
Performance	Réponses rapides de l'API, requêtes Eloquent optimisées, index plein-texte pour la recherche, cache Redis, pagination.
Fiabilité	Comportement stable face aux entrées invalides, gestion explicite des erreurs, cohérence garantie des soldes.
Ergonomie	Interface responsive (TailwindCSS), navigation intuitive, retours visuels (toasts), temps réel pour la messagerie.
Maintenabilité	Architecture en couches, séparation des responsabilités, services métier réutilisables, code versionné et conteneurisé.
Scalabilité	Découplage front/back, file d'attente asynchrone, conception sans état de l'API, déploiement par conteneurs.

Tableau 9: Besoins non fonctionnels de la plateforme

IV. Règles de gestion

Les règles de gestion formalisent les contraintes métier que le système doit respecter en toute circonstance. Elles complètent les besoins fonctionnels en précisant les invariants, les limites et les comportements obligatoires indépendamment des scénarios d'utilisation.

Réf.	Règle de gestion	Module
RG-01	Un utilisateur ne peut avoir qu'un seul rôle actif (Client OU Freelance). Le rôle est choisi à l'inscription et ne peut être modifié ultérieurement.	Auth

Réf.	Règle de gestion	Module
RG-02	La connexion via Google OAuth crée automatiquement un compte si l'email n'existe pas encore ; sinon elle authentifie le compte existant sans modifier le mot de passe.	Auth
RG-03	L'activation des retraits (Stripe Connect) est conditionnée à la validation KYC. Tout freelance non vérifié ne peut pas soumettre de demande de retrait.	Finance / KYC
RG-04	Le solde d'un portefeuille ne peut jamais être négatif. Toute opération qui entraînerait un solde négatif est rejetée avant exécution.	Finance
RG-05	Le financement du séquestre est irréversible une fois le contrat démarré. Les fonds ne peuvent revenir au client qu'en cas de litige arbitré par l'admin.	Finance / Contrats
RG-06	La commission de la plateforme est prélevée à la libération de chaque jalon (taux paramétrable dans platform_settings, 10 % par défaut). Elle ne peut jamais dépasser le montant du jalon.	Finance
RG-07	Un jalon doit être dans l'état 'soumis' avant d'être validé ou rejeté. Un jalon 'payé' ne peut plus être ni modifié ni rejeté.	Contrats
RG-08	Le déclenchement d'un litige gèle toutes les libérations de jalons du contrat jusqu'à résolution par l'administrateur.	Contrats
RG-09	Une offre ne peut recevoir de nouvelles propositions si son statut est 'closed', 'expired' ou 'in_progress'. Les propositions en attente restent visibles.	Marketplace
RG-10	Un freelance ne peut soumettre qu'une seule proposition par offre. Une seconde soumission écrase la première si elle est encore en attente.	Marketplace
RG-11	L'acceptation d'une proposition ferme automatiquement l'offre aux nouvelles candidatures et crée un contrat entre les deux parties.	Marketplace / Contrats
RG-12	Un service catalogue doit être approuvé par l'administrateur avant d'être visible et achetable. Tout service modifié repasse en file de modération.	Catalogue
RG-13	Un message ne peut être édité que par son auteur et seulement dans les 15 minutes suivant son envoi. La suppression est permanente.	Messagerie
RG-14	Les notifications Web Push sont envoyées uniquement si l'utilisateur a accordé la permission navigateur ET activé le type de notification concerné.	Notifications
RG-15	L'assistant IA est limité à 20 requêtes par minute par utilisateur. Toute réponse IA est historisée dans ai_histories et ne peut être supprimée.	IA
RG-16	Un retrait ne peut être soumis que si le montant est supérieur au minimum de 10 € et inférieur ou égal au solde disponible. Les fonds passent immédiatement en 'pending' à la soumission.	Finance
RG-17	L'idempotence est obligatoire pour toutes les opérations financières : une clé d'idempotence unique est associée à chaque écriture, un appel dupliqué retourne le résultat initial sans re-traitement.	Finance
RG-18	Un webhook Stripe sans signature HMAC valide est rejeté avec un code 400. Un webhook déjà traité (id présent dans stripe_webhook_events) retourne 200 sans traitement pour garantir l'idempotence.	Stripe
RG-19	Un utilisateur non authentifié n'a accès qu'aux pages publiques, à l'inscription et à la connexion. Toute route protégée retourne 401.	Sécurité

Réf.	Règle de gestion	Module
RG-20	L'élévation de privilèges est interdite : un Client ne peut pas agir comme Freelance et vice-versa. Un non-admin ne peut accéder à aucune route /admin/* (403 Forbidden).	Sécurité

Tableau 10: Règles de gestion de la plateforme Panda (RG-01 à RG-20)

V. Langage de modélisation UML adopté

Pour formaliser l'analyse et la conception, nous avons adopté le langage **UML** (*Unified Modeling Language*), standard de la modélisation des systèmes logiciels orientés objet. UML offre une famille de diagrammes complémentaires permettant de représenter aussi bien la structure statique du système (diagrammes de cas d'utilisation, de classes) que son comportement dynamique (diagrammes de séquence, d'états). Pour la base de données, nous avons complété UML par la méthode **Merise** (modèles conceptuel et logique de données), bien adaptée à la conception relationnelle.

Dans ce chapitre, nous nous concentrons sur le **diagramme de cas d'utilisation**, qui offre une vision globale et fonctionnelle du système, intelligible par toutes les parties prenantes. Les diagrammes structurels et dynamiques sont présentés au chapitre 3, consacré à la conception.

VI. Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global synthétise les interactions entre les acteurs et les grandes fonctionnalités du système. Chaque *cas d'utilisation* représente une unité de service rendue à un acteur. En raison de sa richesse fonctionnelle, le diagramme complet est présenté en **trois parties** successives, chacune couvrant un ensemble cohérent de modules.

a Partie 1 — Authentification, Administrateur & Annonces

La première partie couvre les modules fondamentaux : la gestion du compte (inscription, connexion, 2FA, OAuth, KYC), l'espace administrateur et la place de marché des annonces. On y observe que **S'authentifier** est la porte d'entrée systématique — toutes les actions métier en dépendent par inclusion. L'Administrateur dispose de cas exclusifs (modération, arbitrage, vérification KYC) inaccessibles aux autres rôles.

Panda -- Diagramme de Cas d'Utilisation Complet

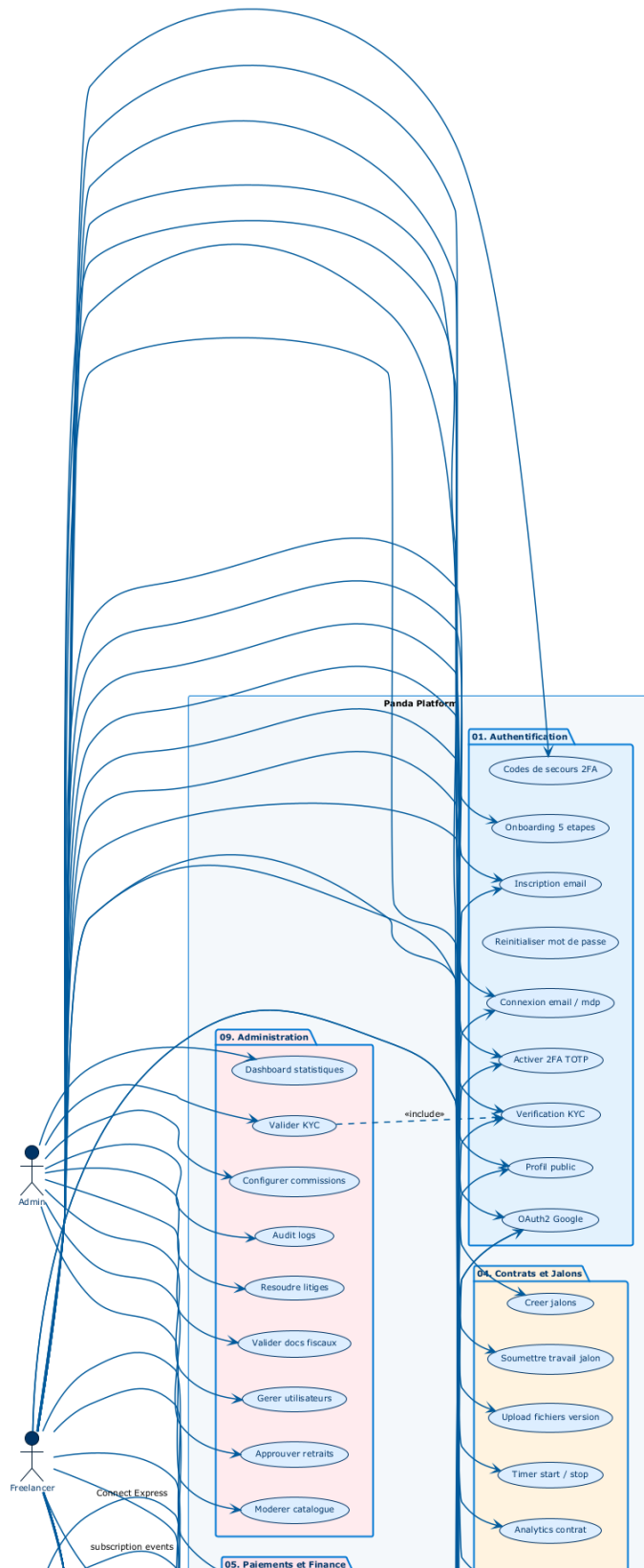


Figure 2: Diagramme de cas d'utilisation — Partie 1 : Authentification, Administrateur & Annonces

- **Acteurs** : Visiteur, Client, Freelance, Administrateur ;
- **Cas clés** : Créer un compte, Se connecter (email / Google OAuth), Activer la 2FA, Réinitialiser le mot de passe, Vérifier l'identité (KYC) ;
- **Administrateur** : Gérer les utilisateurs, Valider les vérifications KYC, Modérer le catalogue, Consulter le tableau de bord ;
- **Annonces** : Publier une offre (Client), Rechercher des freelances, Consulter les profils, Enregistrer des favoris.

b Partie 2 — Catalogue, Contrats, Paiements & Communication

La deuxième partie décrit le cœur opérationnel de la plateforme : la création et l'achat de services packagés (catalogue), la gestion complète du cycle de vie d'un contrat (jalons, fichiers, suivi du temps, extensions), le moteur de paiement par séquestre et la messagerie temps réel. C'est ici que la valeur centrale de Panda — **la confiance financière** — est pleinement matérialisée. Les acteurs externes Stripe et Laravel Reverb interviennent respectivement sur les paiements et la communication.

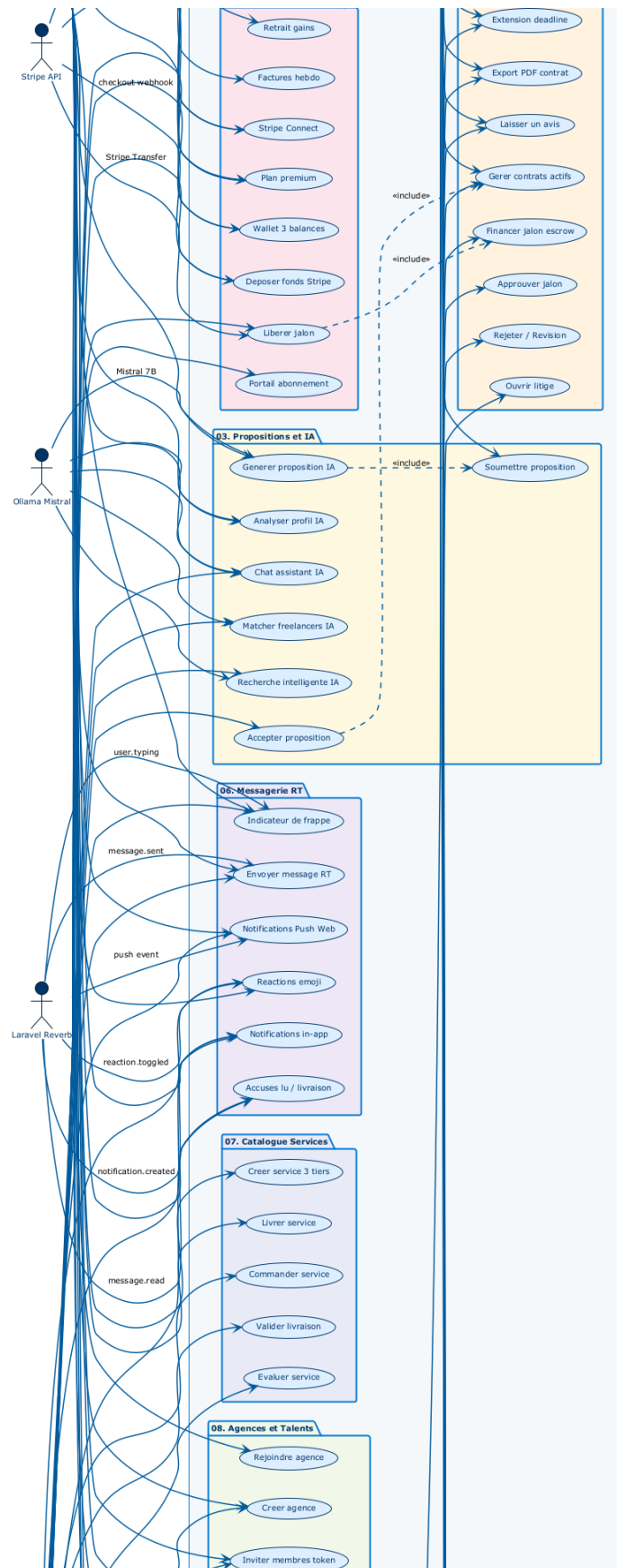


Figure 3: Diagramme de cas d'utilisation — Partie 2 : Catalogue, Contrats, Paiements & Communication

- **Catalogue** : Créer un service packagé (Freelance), Passer commande (Client), Livrer une commande, Évaluer la prestation ;
- **Contrats** : Créer un jalon, Soumettre/approuver/rejeter un jalon, Partager des fichiers, Suivre le temps, Demander une extension ;
- **Paielements** : Déposer des fonds (via Stripe), Financer le séquestre, Libérer un jalon, Demander un retrait, Gérer les litiges ;
- **Communication** : Envoyer/recevoir des messages (temps réel), Partager des pièces jointes, Réagir aux messages.

c Partie 3 — Notifications, Agences, IA & Marketplace

La troisième partie regroupe les modules transverses et enrichissants : le système de notifications (in-app et Web Push), la gestion des agences et des listes de talents, l'assistant d'intelligence artificielle et la place de marché des freelances. Elle illustre les fonctionnalités différenciantes de Panda par rapport aux plateformes classiques : l'IA pour accélérer les tâches répétitives et les agences pour structurer les équipes de freelances.

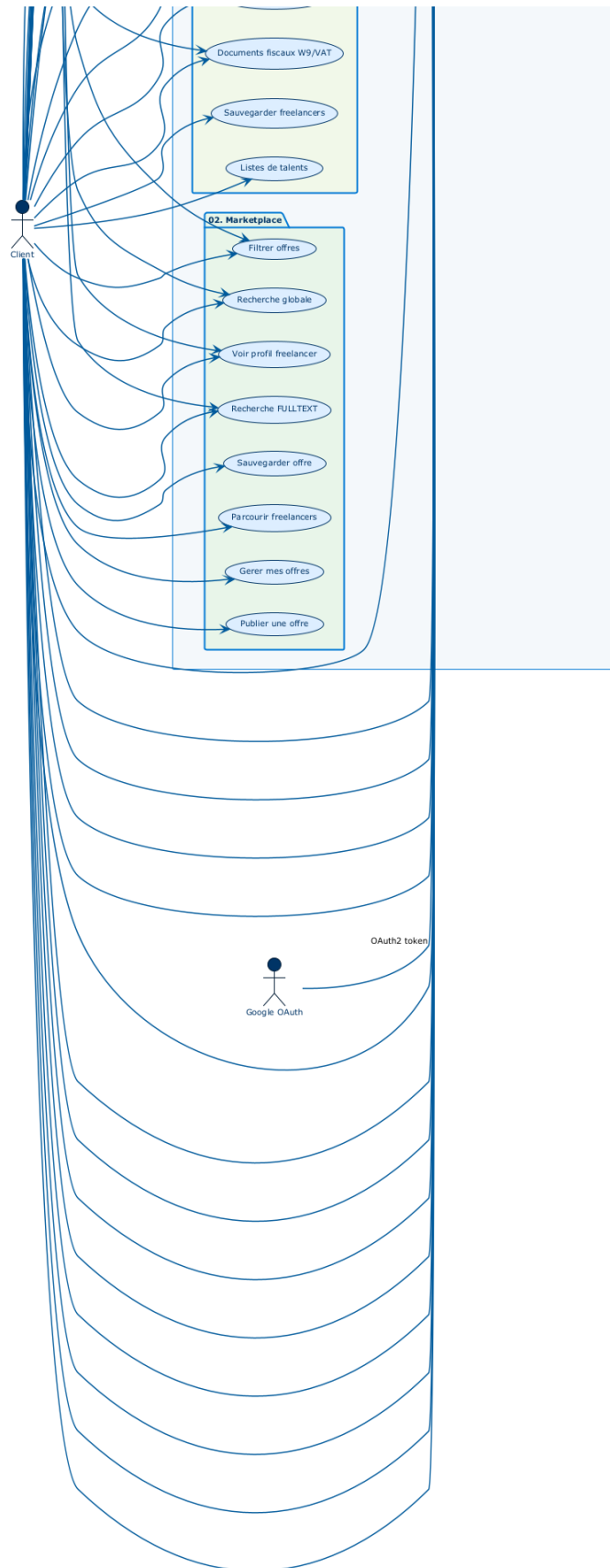


Figure 4: Diagramme de cas d'utilisation — Partie 3 : Notifications, IA, Agences & Marketplace

- **Notifications** : Recevoir des alertes in-app, Activer les notifications Web Push, Gérer les préférences de notification ;
- **IA** : Générer une proposition, Obtenir une mise en correspondance, Analyser un profil, Recherche intelligente, Assistant conversationnel ;
- **Agences** : Créer une agence, Inviter des membres, Gérer les membres, Transférer la propriété ;
- **Talents** : Constituer une liste de talents, Gérer les favoris.

Ce diagramme complet couvre neuf modules fonctionnels distincts. On observe que de nombreux cas dépendent du cas « *S'authentifier* » (relation d'inclusion) et que les acteurs externes (Stripe, IA) interviennent en support de cas spécifiques. Les rôles Client et Freelance partagent certains cas transverses (messagerie, gestion du profil), tandis que l'Administrateur dispose de cas de supervision exclusifs.

1 Module I — Authentification et gestion du compte

Ce module constitue le **point d'entrée** obligatoire de la plateforme. Tout utilisateur doit s'authentifier avant d'accéder aux fonctionnalités principales. La relation d'**inclusion** (*include*) entre la majorité des autres cas et « S'authentifier » matérialise la contrainte de sécurité : aucune action métier n'est possible sans session valide. Les cas d'utilisation couverts sont :

- **Créer un compte** : inscription par e-mail et mot de passe, avec vérification de l'adresse e-mail par lien de confirmation envoyé automatiquement ;
- **Se connecter** : authentification par identifiants ou via Google (OAuth 2.0) grâce à Laravel Socialite — le jeton Bearer Sanctum est émis à l'issue de cette étape ;
- **Activer / gérer la 2FA** : mise en place de l'authentification à deux facteurs (code TOTP généré par application, codes de secours à usage unique) ;
- **Réinitialiser le mot de passe** : envoi d'un lien signé et limité dans le temps par e-mail ;
- **Vérifier l'identité (KYC)** : soumission de pièces justificatives pour validation par l'administrateur avant l'accès aux fonctions financières ;
- **Gérer le profil** : mise à jour des informations personnelles, avatar, biographie et préférences de notification.

2 Module II — Espace Administrateur

L'administrateur dispose d'un espace de supervision qui lui permet de contrôler l'ensemble de l'activité de la plateforme. Ses cas exclusifs sont :

- **Gérer les utilisateurs** : consultation, suspension ou suppression des comptes, réinitialisation des accès en cas de compromission ;
- **Modérer le catalogue** : approbation ou rejet des services soumis par les freelances, avec communication du motif de refus ;
- **Valider les vérifications d'identité (KYC)** : examen de la file de demandes en attente, prise de décision et notification à l'utilisateur ;
- **Gérer les retraits** : approbation ou refus des demandes de retrait des freelances, déclenchement des versements Stripe Connect ;
- **Arbitrer les litiges** : résolution des conflits entre clients et freelances, avec possibilité de débloquer ou d'annuler un contrat gelé ;
- **Consulter le tableau de bord** : statistiques globales (revenus, utilisateurs actifs, commissions collectées, activité transactionnelle).

3 Module III — Annonces et Freelances

Ce module couvre le premier modèle de mise en relation, à l'initiative du **client** qui publie un besoin auquel les freelances répondent :

- **Publier une offre** : création d'une annonce avec titre, description, catégorie, budget et type de mission (forfait ou tarif horaire) ;
- **Rechercher des freelances** : filtrage multicritères par compétences, catégorie, tarif, disponibilité et localisation géographique ;
- **Consulter un profil freelance** : visualisation du portfolio, des compétences, des évaluations et de l'historique de missions réalisées ;
- **Enregistrer des favoris** : constitution de listes de talents pour accès rapide lors de prochains projets ;
- **Gérer son profil freelance** : renseignement du titre, de la biographie, des compétences, du portfolio, de la disponibilité et du tarif horaire.

4 Module IV — Freelance et Services (Catalogue)

Ce module matérialise le second modèle de mise en relation, à l'initiative du **freelance** qui propose des services packagés directement achetable :

- **Créer un service packagé** : définition de trois paliers (basique, standard, premium) avec description, prix fixé, délai de livraison et liste des livrables ;
- **Gérer son portfolio** : ajout et organisation de réalisations passées illustrant les compétences et la qualité du travail du freelance ;
- **Livrer une commande** : soumission des livrables dans le délai imparti, communication avec le client, gestion des révisions demandées ;
- **Gérer sa disponibilité** : paramétrage du statut (disponible, occupé, en pause) et du délai de réponse moyen visible sur le profil public.

5 Module V — Propositions et Offres

Ce module gère le cycle de vie complet des propositions soumises par les freelances en réponse aux offres publiées par les clients :

- **Soumettre une proposition** : rédaction d'une lettre de motivation, fixation du montant proposé et du délai estimé ; l'assistant IA peut générer un premier jet que le freelance édite et personnalise ;
- **Consulter les propositions reçues** : le client visualise, compare et évalue toutes les candidatures reçues sur son offre ;
- **Accepter ou refuser une proposition** : l'acceptation déclenche automatiquement la création d'un contrat actif entre les deux parties ;
- **Retirer une proposition** : le freelance peut annuler sa candidature tant qu'elle n'a pas encore été acceptée par le client.

6 Module VI — Gestion des Paiements (Séquestre / Escrow)

Ce module est le **cœur financier** de la plateforme. Il implémente le mécanisme de séquestre qui garantit la confiance entre les parties :

- **Déposer des fonds** : alimentation du portefeuille via Stripe Checkout ; la confirmation du paiement arrive de façon asynchrone par webhook signé ;
- **Financer le séquestre** : le client bloque les fonds d'un contrat ; ils quittent son solde disponible et entrent en séquestre, inaccessibles jusqu'à validation d'un jalon ;
- **Soumettre et valider un jalon** : le freelance livre le travail, le client approuve et les fonds sont libérés automatiquement, déduction faite de la commission de la plateforme ;
- **Demander un retrait** : le freelance transfère son solde disponible vers son compte bancaire via Stripe Connect/Payout ;

- **Consulter l'historique financier** : visualisation de toutes les transactions et du journal comptable en double-entrée.

7 Module VII — Dialogue Structuré (Messagerie et Contrats)

Ce module assure la communication et le suivi opérationnel dans le cadre des contrats actifs :

- **Échanger des messages** : messagerie temps réel par WebSocket (Laravel Reverb + Echo) avec pièces jointes, accusés de lecture et réactions aux messages ;
- **Découper le contrat en jalons** : définition de jalons mesurables avec montant, description et date d'échéance ;
- **Partager des fichiers versionnés** : envoi de livrables et documents de travail attachés au contrat ;
- **Suivre le temps** : chronométrage en temps réel pour les contrats horaires, avec récapitulatif hebdomadaire et facturation automatique ;
- **Demander une extension de délai** : le freelance peut solliciter un délai supplémentaire que le client accepte ou refuse.

8 Module VIII — Agences et Talents

Ce module permet à des freelances de se regrouper sous une même bannière professionnelle et aux clients d'organiser leurs prestataires :

- **Créer une agence** : le propriétaire définit le nom, la présentation et l'identité visuelle de l'agence ;
- **Inviter des membres** : envoi d'invitations par jeton sécurisé aux freelances que l'agence souhaite intégrer ;
- **Gérer les membres** : acceptation des invitations, retrait de membres, transfert de la propriété de l'agence ;
- **Constituer des listes de talents** : les clients organisent leurs freelances favoris en listes thématiques pour simplifier la sélection lors de futurs projets.

9 Module IX — Notifications

Ce module transverse assure que chaque acteur est informé en temps réel des événements qui le concernent sur la plateforme :

- **Notifications in-app** : alertes dans l'interface pour toute action sur une offre, proposition, contrat, paiement ou message ;

- **Notifications Web Push** : alertes navigateur (VAPID/Service Worker) même lorsque l'application n'est pas ouverte en premier plan ;
- **Gérer les préférences** : chaque utilisateur configure les types d'événements pour lesquels il souhaite être notifié.

VII. Description textuelle des cas d'utilisation

Afin de préciser le comportement attendu, nous détaillons ci-après quelques cas d'utilisation représentatifs sous forme de fiches descriptives.

1 Cas « S'authentifier »

Acteur(s)	Visiteur
Pré-conditions	Le visiteur possède un compte (ou en crée un).
Scénario nominal	1) Le visiteur saisit son e-mail et son mot de passe. 2) Le système applique une limitation de débit, vérifie les identifiants (bcrypt). 3) Si la 2FA est active, un code temporel est exigé. 4) Le système émet un jeton Bearer (Sanctum). 5) Le client stocke le jeton et accède à son espace.
Scénarios alternatifs	2a) Identifiants invalides → message d'erreur. 3a) Code 2FA erroné → nouvel essai. 1b) Authentification via Google (OAuth) en alternative.
Post-conditions	L'utilisateur est authentifié ; un jeton est associé à sa session.

2 Cas « Publier une offre »

Acteur(s)	Client
Pré-conditions	Le client est authentifié.
Scénario nominal	1) Le client ouvre le formulaire de publication. 2) Il renseigne titre, description, catégorie, budget et type. 3) Le système valide les données (FormRequest). 4) L'offre est enregistrée et publiée. 5) Elle devient visible et indexée pour la recherche.
Scénarios alternatifs	3a) Données incomplètes → erreurs de validation affichées. 4a) Le client enregistre un brouillon avant publication.
Post-conditions	Une nouvelle offre est créée et ouverte aux propositions des freelances.

3 Cas « Postuler à une offre »

Acteur(s)	Freelance
Pré-conditions	Le freelance est authentifié et l'offre est ouverte.
Scénario nominal	1) Le freelance consulte le détail de l'offre. 2) Il rédige une proposition (lettre, montant), éventuellement assistée par l'IA. 3) Le système enregistre la proposition. 4) Le client en est notifié.
Scénarios alternatifs	2a) Le freelance demande une génération de proposition à l'IA puis l'édite. 3a) Le freelance retire sa proposition tant qu'elle n'est pas acceptée.
Post-conditions	Une proposition est rattachée à l'offre et visible par le client.

4 Cas « Financer le séquestre et valider un jalon »

Acteur(s)	Client
Pré-conditions	Un contrat actif existe et le client dispose d'un solde suffisant.
Scénario nominal	1) Le client finance le séquestre du contrat (les fonds quittent son solde disponible et sont bloqués). 2) Le freelance soumet un jalon livré. 3) Le client approuve le jalon. 4) Le système (LedgerService) prélève la commission, crédite le freelance et marque le jalon comme payé, de manière atomique et idempotente.
Scénarios alternatifs	1a) Solde insuffisant → invitation à déposer des fonds (Stripe). 3a) Le client rejette le jalon avec un motif → le freelance re-livre. 3b) Litige → gel du contrat et arbitrage admin.
Post-conditions	Le jalon est payé ; le freelance est crédité ; chaque mouvement est tracé dans le grand livre.

5 Cas « Vendre un service et passer commande »

Acteur(s)	Freelance, Client
Pré-conditions	Le freelance possède un service publié et modéré ; le client est authentifié.
Scénario nominal	1) Le freelance crée un service à paliers (basique/standard/premium). 2) L'administrateur modère et approuve le service. 3) Le client choisit un palier et passe commande (paiement). 4) Le freelance livre la commande. 5) Le client valide et évalue la prestation.
Scénarios alternatifs	2a) Service rejeté → notification au freelance avec motif. 4a) Livraison non conforme → demande de révision par le client.
Post-conditions	La commande est complétée, le freelance crédité et une évaluation enregistrée.

6 Cas « Demander un retrait »

Acteur(s)	Freelance, Administrateur
Pré-conditions	Le freelance dispose d'un solde disponible supérieur au minimum requis.
Scénario nominal	1) Le freelance demande un retrait (montant, méthode). 2) Les fonds passent du solde disponible au solde en attente. 3) L'administrateur examine la demande. 4) En cas d'approbation, un versement Stripe est déclenché ; le webhook confirme le retrait.
Scénarios alternatifs	1a) Solde insuffisant ou inférieur au minimum → refus. 3a) Rejet par l'admin → fonds re-crédités au solde disponible. 4a) Échec Stripe → re-crédit automatique et statut « échoué ».
Post-conditions	Le retrait est complété (ou rejeté) ; chaque étape est tracée dans le grand livre.

7 Cas « Vérifier l'identité (KYC) »

Acteur(s)	Freelance/Client, Administrateur
Pré-conditions	L'utilisateur est authentifié.
Scénario nominal	1) L'utilisateur soumet ses pièces justificatives. 2) Le système enregistre la demande (statut « en attente »). 3) L'administrateur consulte la file de vérification. 4) Il approuve ou rejette la demande. 5) L'utilisateur est notifié du résultat.
Scénarios alternatifs	4a) Rejet → motif communiqué et nouvelle soumission possible.
Post-conditions	Le statut de vérification de l'utilisateur est mis à jour.

8 Cas « Gérer une agence »

Acteur(s)	Agence (propriétaire), Freelance
Pré-conditions	Le propriétaire dispose d'un compte autorisé à créer une agence.
Scénario nominal	1) Le propriétaire crée l'agence (nom, présentation). 2) Il invite des freelances par jeton. 3) Les freelances acceptent ou refusent l'invitation. 4) Le propriétaire gère les membres et peut transférer la propriété.
Scénarios alternatifs	3a) Refus de l'invitation → le freelance n'est pas ajouté. 4a) Retrait d'un membre par le propriétaire.
Post-conditions	L'agence regroupe plusieurs freelances sous une bannière commune.

Ces fiches illustrent l'importance des **scénarios alternatifs** (erreurs, litiges, abandons) : sur une plateforme transactionnelle, ils sont aussi critiques que le scénario nominal et structurent une grande partie de la logique applicative.

9 Matrice de traçabilité des exigences

Afin de garantir que chaque besoin identifié trouve une réponse dans la solution, nous avons établi une matrice de traçabilité reliant les exigences aux modules qui les satisfont.

Exigence	Module / composant	Couverture
Inscription, connexion, OAuth, 2FA	AuthController, Sanctum, Socialite, TwoFactor	Complète
Publication d'offres et propositions	JobController, ProposalController	Complète
Catalogue de services et commandes	CatalogProjectController, CatalogOrderController	Complète
Contrats, jalons, fichiers, temps	ContractController et contrôleurs associés	Complète
Paie sous séquestre	LedgerService, PaymentController	Complète
Dépôts et retraits Stripe	StripeService, StripeWebhookController	Complète
Messagerie temps réel	ChatController, événements + Reverb	Complète
Notifications et Web Push	NotificationController, PushSubscription	Complète
Assistant IA	AIController, service de modèle de langage	Complète
Agences et talents	AgencyController, TalentListController	Complète
Vérification d'identité (KYC)	IdentityVerificationController	Complète
Administration et modération	AdminController, AdminFinanceController	Complète

Tableau 11: Matrice de traçabilité des exigences vers les composants

VIII. Conclusion du chapitre

Ce chapitre a recensé et structuré les besoins de la plateforme. Nous avons identifié les acteurs, détaillé les besoins fonctionnels par module et précisé les exigences non fonctionnelles — au premier rang desquelles la sécurité et l'intégrité financière. Le langage UML nous a permis de formaliser ces besoins, à travers le diagramme de cas d'utilisation global et la description des cas les plus significatifs. Ces éléments constituent le cahier des charges sur lequel s'appuie la **conception** détaillée, objet du chapitre suivant.

Chapitre 3 : Conception de la solution

Après avoir spécifié les besoins, ce chapitre détaille la **conception** de Panda. Il présente successivement l'architecture globale et logicielle, les mécanismes de sécurité, la modélisation objet (diagramme de classes), les comportements dynamiques (diagrammes de séquence et d'états), puis la conception de la base de données et les choix technologiques. L'objectif est de traduire les exigences en une architecture claire, sécurisée et maintenable.

I. Architecture globale

Panda adopte une architecture **découplée** de type *client-serveur* à trois niveaux, structurée autour d'une **API REST** et d'une **application monopage** (*Single Page Application*). Le front-end React communique avec le back-end Laravel exclusivement par des appels HTTP/JSON sécurisés, l'authentification reposant sur un **jeton Bearer** (Sanctum) transmis dans l'en-tête **Authorization**. Cette séparation nette des responsabilités améliore la maintenabilité, autorise une évolution indépendante des deux couches et prépare une éventuelle ouverture à d'autres clients (application mobile).

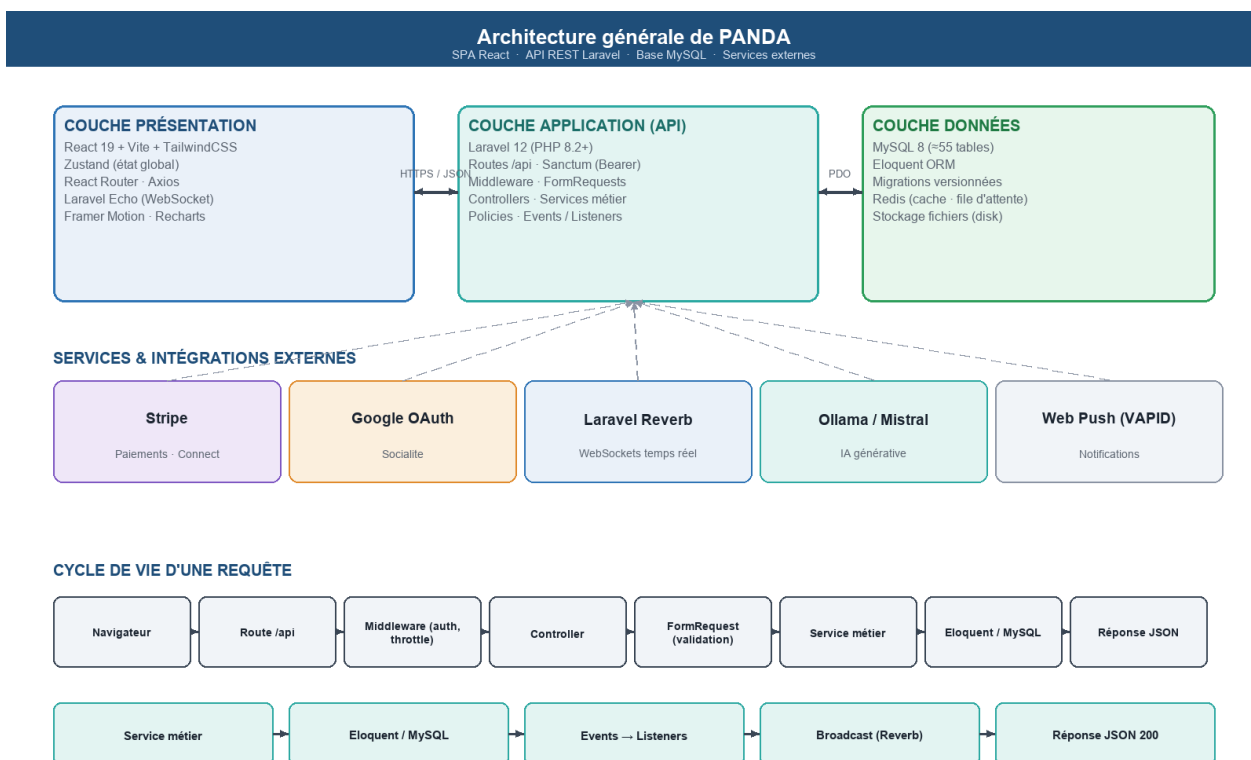


Figure 5: Architecture générale en trois couches et services externes

La **couche de présentation** (React) gère les interfaces, les formulaires, l'état applicatif (Zustand) et la communication temps réel (Laravel Echo). La **couche application** (Laravel) concentre la logique métier : routage, middlewares, validation, services et règles d'autorisation. La **couche de données** (MySQL) assure le stockage relationnel et l'intégrité référentielle, complétée par Redis pour le cache et la file d'attente. Autour de ce noyau gravitent les **services externes** : Stripe (paiements), Google (OAuth), Laravel Reverb (WebSockets) et un serveur de modèle de langage (IA).

Couche	Technologie	Rôle principal
Présentation	React 19 · Vite · TailwindCSS · Zustand	Interfaces, état, interactions, temps réel
Application	Laravel 12 (PHP 8.2) · Sanctum	API REST, logique métier, sécurité, services
Données	MySQL 8 · Eloquent ORM · Redis	Persistance, intégrité, cache, file d'attente

Tableau 12: Synthèse des couches de l'architecture

II. Architecture logicielle en couches

Côté back-end, Panda applique le patron **MVC** de Laravel, enrichi d'une **couche de services métier** dédiée. Ce choix est essentiel : il permet d'extraire la logique complexe (et notamment financière) des contrôleurs pour la concentrer dans des services réutilisables, testables et garants de la cohérence. Une requête traverse ainsi une chaîne de responsabilités clairement délimitée.

Organisation en couches du backend (MVC + Services)

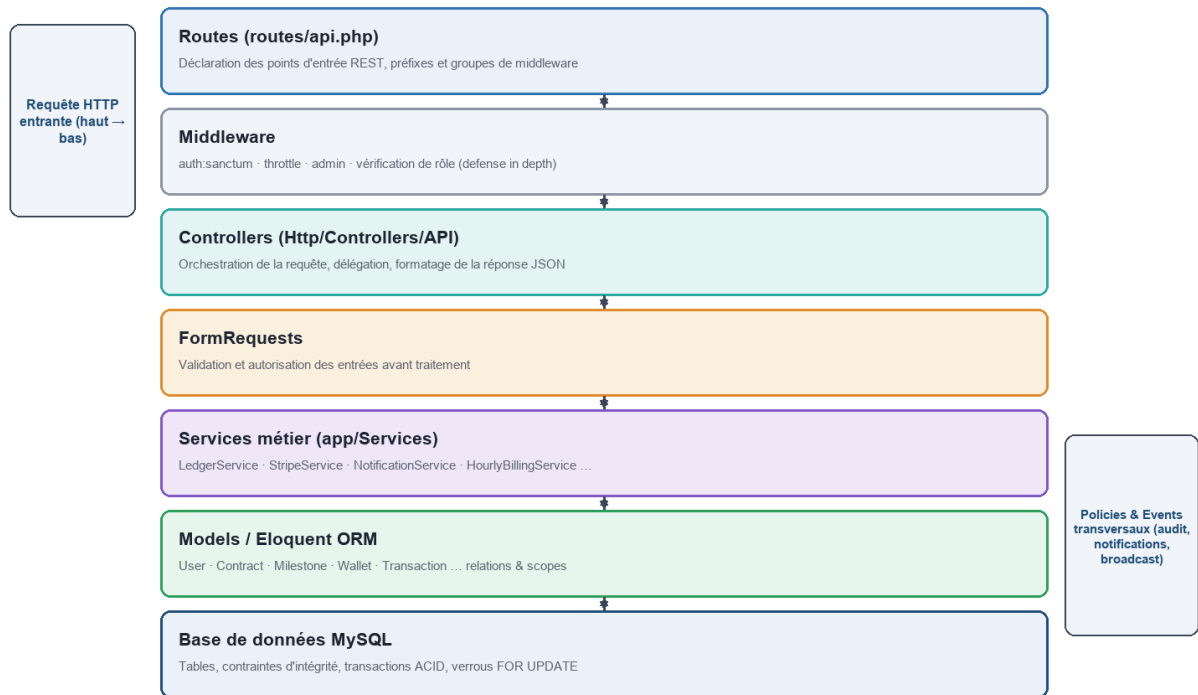


Figure 6: Organisation en couches du back-end (MVC enrichi de services métier)

Le traitement d'une requête suit le cheminement suivant : la **route** dirige vers un **contrôleur**, après passage par les **middlewares** (authentification, limitation de débit, rôle). La validation est déléguée à un **FormRequest**. Le contrôleur orchestre alors le traitement en s'appuyant sur un **service métier**, qui manipule les **modèles Eloquent** et la base de données. Des préoccupations transverses — **politiques d'autorisation, événements et écouteurs** (notifications, audit, diffusion temps réel) — s'appliquent tout au long de la chaîne. La réponse est finalement sérialisée en JSON.

Le principe directeur est la **séparation des responsabilités** : un contrôleur orchestre mais ne contient pas la logique métier sensible ; celle-ci réside dans des services (**LedgerService**, **StripeService**, **NotificationService**, etc.), seuls habilités à muter les données critiques.

1 Vue en paquetages

D'un point de vue logique, l'application se décompose en **paquetages** cohérents, chacun regroupant les contrôleurs, services et modèles d'un même domaine. Ce découpage favorise

la modularité et limite le couplage entre domaines.

Paquetage	Contenu et responsabilité
Auth	Inscription, connexion, OAuth, 2FA, réinitialisation, KYC
Marketplace	Offres, propositions, catalogue, recherche, profils
Contracts	Contrats, jalons, fichiers, temps, extensions, activité
Payments	Portefeuille, séquestre, retraits, Stripe, facturation
Communication	Messagerie temps réel, notifications, Web Push
AI	Génération, correspondance, analyse, recherche intelligente
Talent & Agency	Favoris, listes de talents, agences et invitations
Admin	Tableau de bord, modération, finance, gestion des litiges

Tableau 13: Décomposition logique en paquetages

III. Sécurité et authentification

La sécurité étant une exigence de premier ordre, Panda met en œuvre une **défense en profondeur** combinant plusieurs mécanismes complémentaires.

- **Authentification par jeton** : Laravel Sanctum émet un jeton Bearer à la connexion ; chaque requête protégée le présente dans l'en-tête **Authorization** ;
- **Authentification à deux facteurs (2FA)** : code temporel TOTP et codes de secours, pour renforcer la protection des comptes sensibles ;
- **Authentification sociale** : connexion via Google (OAuth 2.0) à l'aide de Laravel Socialite ;
- **Hachage des mots de passe** : algorithme bcrypt ; aucun mot de passe n'est stocké en clair ;
- **Limitation de débit (throttling)** : les routes d'authentification (10 req/min), de réinitialisation (5 req/min) et d'IA (20 req/min) sont protégées contre les abus ;
- **Politiques d'autorisation (Policies)** : chaque action sensible vérifie que l'utilisateur y est habilité (par exemple, seul le client d'un contrat peut en libérer les jalons) ;
- **Vérification d'identité (KYC)** : soumission et validation de pièces justificatives ;
- **Journal d'audit (audit logs)** : traçabilité des actions sensibles ;
- **Vérification de signature** des webhooks Stripe et **idempotence** des événements reçus.

Le diagramme de séquence ci-dessous illustre le processus d'authentification, intégrant la limitation de débit, la vérification des identifiants et la 2FA.

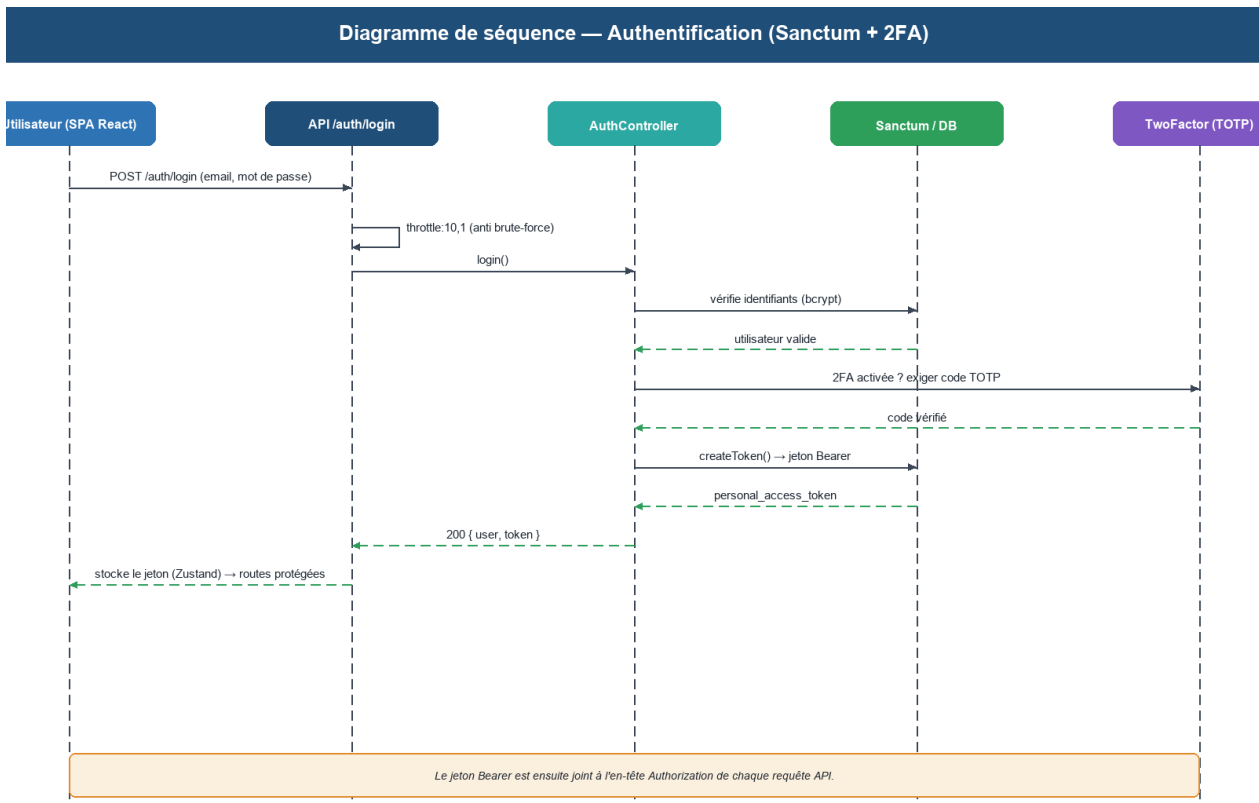


Figure 7: Diagramme de séquence — authentification (Sanctum + 2FA)

Le **cycle de vie du jeton** est maîtrisé : émis à la connexion, il peut être révoqué à la déconnexion et n'est jamais exposé au-delà du strict nécessaire. Côté client, il est conservé de façon contrôlée et systématiquement joint aux requêtes par l'intercepteur Axios. Cette approche, sans état côté serveur, est parfaitement adaptée à une application monopage et facilite la montée en charge.

La **protection des données personnelles** complète ce dispositif : les informations sensibles (pièces d'identité du KYC) sont stockées sur un disque privé, non accessible publiquement, et ne sont consultables que par l'administrateur via des points d'accès protégés. Les actions critiques sont par ailleurs consignées dans un **journal d'audit**, assurant la traçabilité indispensable à toute investigation ultérieure.

IV. Diagramme de classes

Le diagramme de classes modélise la structure statique du système : les entités métier, leurs attributs et leurs relations. Il constitue la traduction objet du domaine et le fondement de l'architecture de la base de données. En raison de la richesse du modèle (plus de 30 classes),

il est présenté en **trois sous-systèmes** distincts, chacun regroupant des classes cohérentes par domaine fonctionnel.

a Sous-système 1 — Identité, Sécurité & Finance

Ce premier sous-système regroupe les entités liées à la **gestion des utilisateurs**, à la **sécurité** et au **noyau financier**. On y trouve la classe centrale **User** avec ses profils associés (**FreelancerProfile**, **ClientProfile**) et la vérification d'identité (**IdentityVerification**). Le noyau financier comprend **Wallet** (portefeuille à trois soldes : disponible, séquestre, en attente), **Transaction** (grand livre en double-entrée, immuable) et **WithdrawalRequest** (demandes de retrait avec cycle de validation).

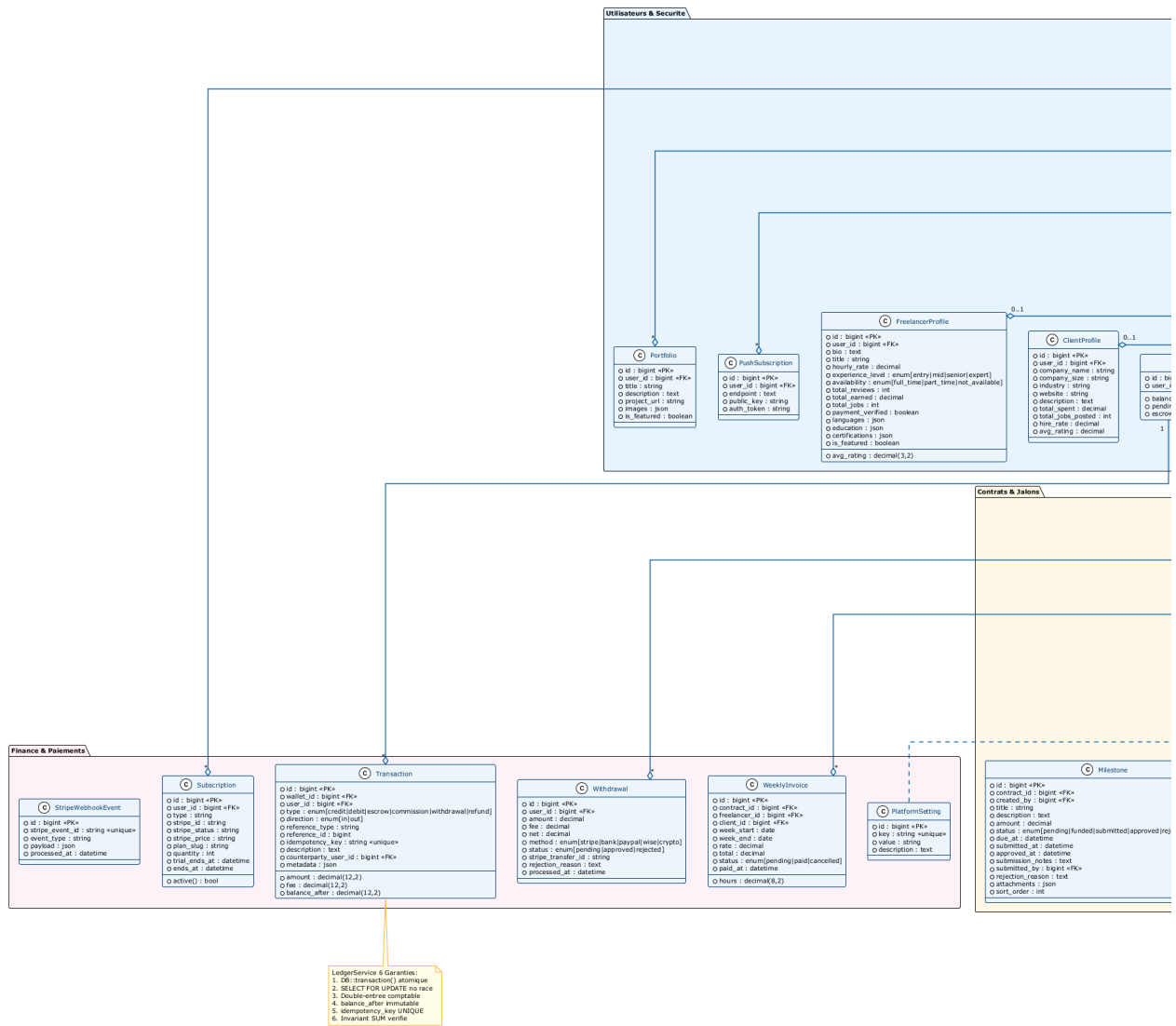


Figure 8: Diagramme de classes — Sous-système 1 : Identité, Sécurité & Finance

Classe	Rôle dans le sous-système
User	Entité centrale : agrège l'identité, le rôle et les préférences de chaque acteur
FreelancerProfile	Profil freelance : compétences, tarif horaire, disponibilité, portfolio
ClientProfile	Profil client : entreprise, secteur, localisation
IdentityVerification	KYC : pièces justificatives, statut (pending/approved/rejected)
Wallet	Portefeuille : 3 soldes distincts (balance, escrow_balance, pending_balance)
Transaction	Grand livre : chaque mouvement financier, immuable, avec balance_after
WithdrawalRequest	Demande de retrait vers Stripe Connect, validée par l'admin

Tableau 14: Classes du sous-système Identité, Sécurité & Finance

b Sous-système 2 — Marketplace, Contrats & Communication

Ce deuxième sous-système constitue le **cœur opérationnel** de la plateforme. La classe **Contract** en est le pivot absolu : elle naît de l'acceptation d'une **Proposal** ou d'une **CatalogOrder**, relie le client et le freelance, et constitue le contexte de tous les jalons, fichiers, transactions et conversations. Le flux complet de mise en relation est visible : **JobPosting** → **Proposal** → **Contract** → **Milestone** → paiement libéré.

Panda -- Diagramme de Classes Complet (Tous les Modeles)

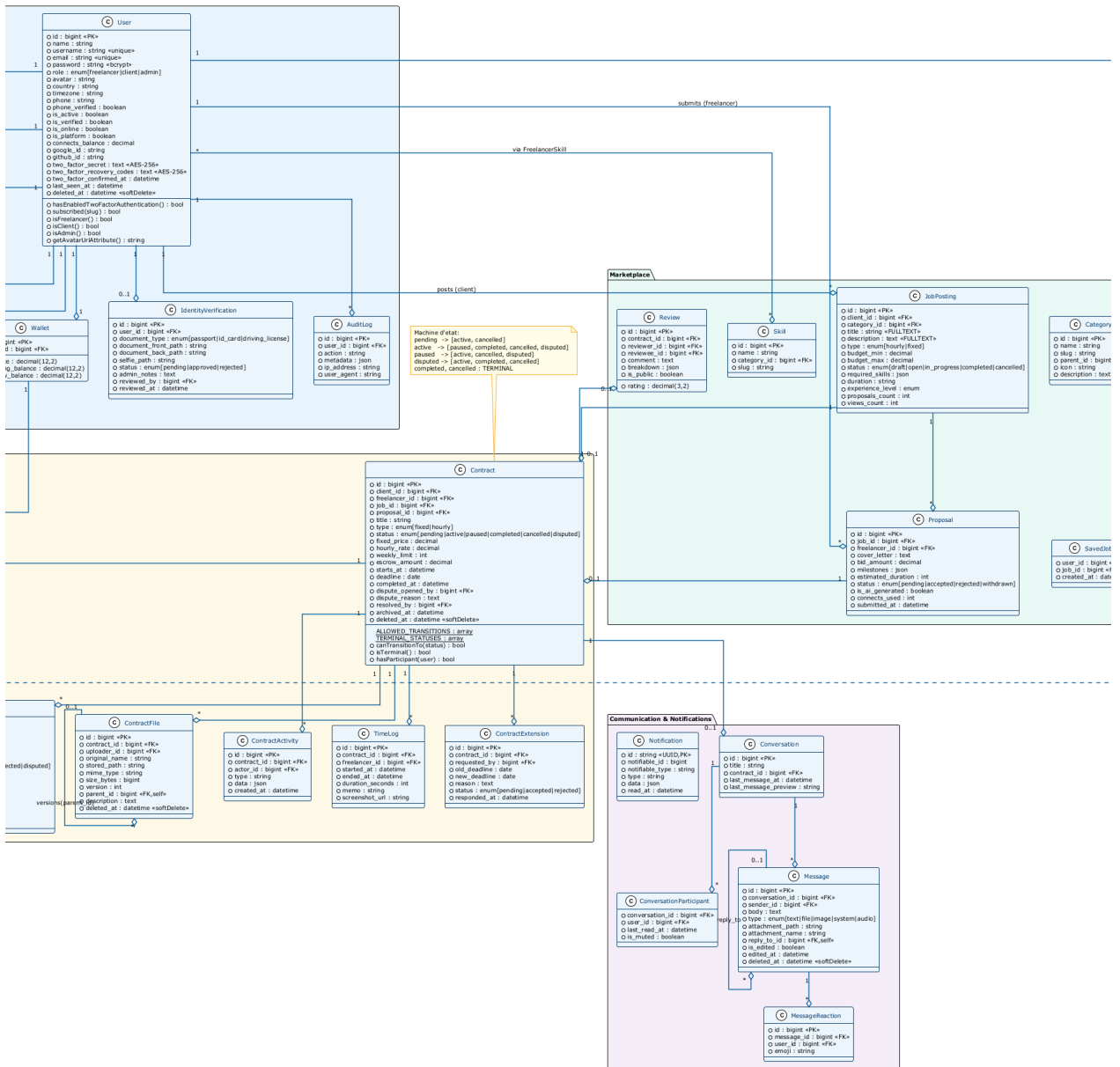


Figure 9: Diagramme de classes — Sous-système 2 : Marketplace, Contrats & Communication

Classe	Rôle dans le sous-système
JobPosting	Offre publiée par le client : titre, budget, catégorie, statut
Proposal	Candidature du freelance : lettre, montant, délai, statut
Contract	Pivot : lie client + freelance, porte l'escrow_amount et le statut (litige)
Milestone	Jalon d'un contrat : montant, soumission, approbation, libération
ContractFile	Fichier versionné attaché au contrat (livrables)
TimeEntry	Entrée de suivi du temps (contrats horaires)
Conversation	Canal de messagerie lié à un contrat ou une offre
Message	Message temps réel avec accusé de lecture et réactions
Review	Évaluation bilatérale post-mission (note + commentaire)

Tableau 15: Classes du sous-système Marketplace, Contrats & Communication

c Sous-système 3 — Catalogue, Agences & Talents

Ce troisième sous-système couvre les fonctionnalités différenciantes de Panda : le **catalogue de services packagés** (inspiré de Fiverr), la **gestion des agences** (regroupement de freelances sous une bannière) et les **listes de talents**. L'IA est représentée par **AiHistory** qui historise chaque interaction avec le modèle de langage. **TaxDocument** trace les obligations fiscales annuelles des freelances.

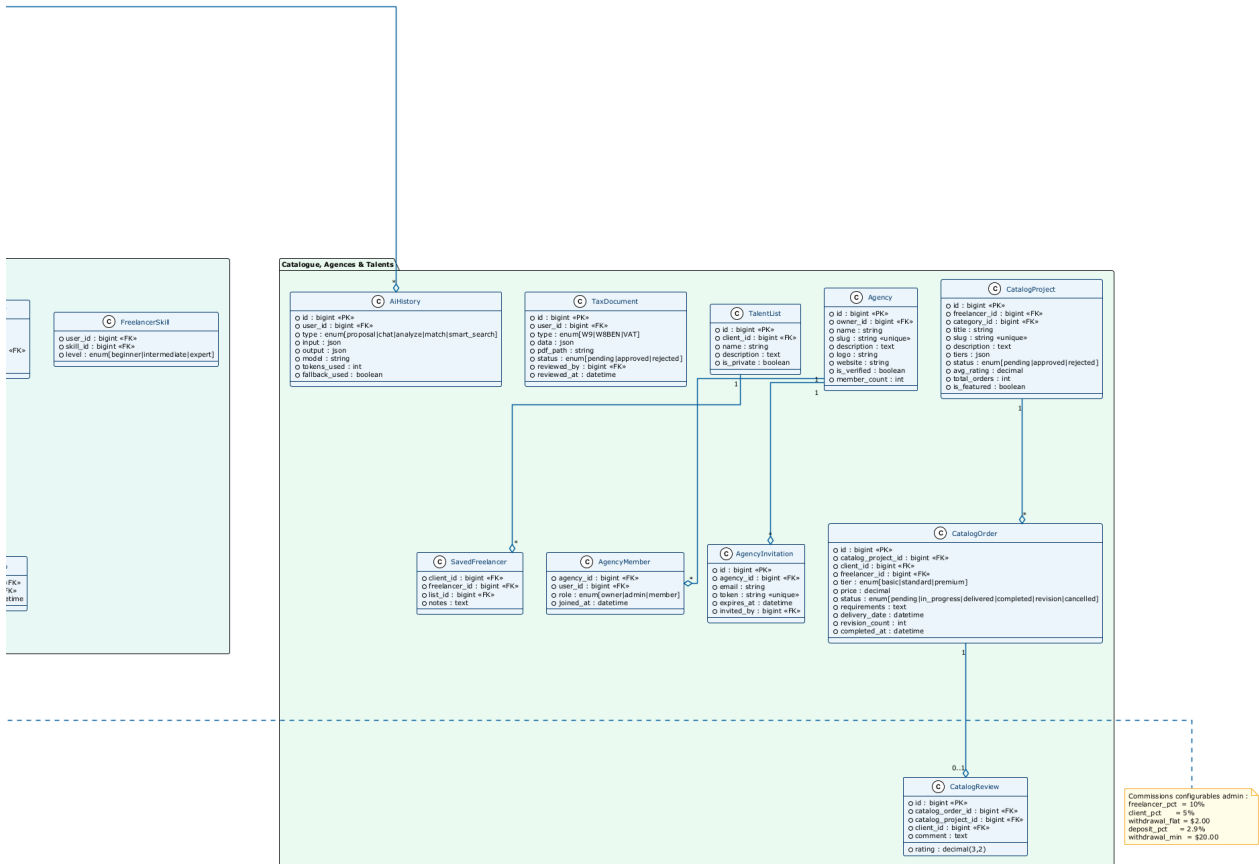


Figure 10: Diagramme de classes — Sous-système 3 : Catalogue, Agences & Talents

Classe	Rôle dans le sous-système
CatalogProject	Service packagé du freelance : 3 paliers tarifaires, modération admin
CatalogOrder	Commande d'un palier par un client : cycle de livraison complet
CatalogReview	Évaluation d'une commande catalogue
Agency	Agence : bannière regroupant plusieurs freelances, propriétaire désigné
AgencyMember	Appartenance d'un freelance à une agence (rôle : owner/admin/member)
AgencyInvitation	Invitation sécurisée par jeton à rejoindre une agence
TalentList	Liste de freelances favoris constituée par un client
SavedFreelancer	Favori d'un freelance dans une liste de talents
AiHistory	Historique des interactions IA (prompt, réponse, tokens utilisés)
TaxDocument	Document fiscal annuel généré pour les freelances actifs

Tableau 16: Classes du sous-système Catalogue, Agences & Talents

On distingue plusieurs regroupements cohérents dans l'ensemble du modèle : les entités d'**identité** (**User**, profils) ; le **flux de mise en relation** (**JobPosting** → **Proposal** → **Contract** → **Milestone**) ; le **noyau financier** (**Wallet**, **Transaction**) ; la **communication** (**Conversation**, **Message**, **Review**) ; et le **catalogue** (**CatalogProject**, **CatalogOrder**). La classe **Contract** joue un rôle pivot absolu : elle relie un client et un freelance, agrège des jalons et constitue le contexte des mouvements financiers et des conversations.

1 Groupe Identité et Authentification

Ce groupe regroupe les classes liées à la gestion des utilisateurs et de leurs profils :

Classe	Attributs principaux	Relations clés
User	id, name, email, password, role, email_verified_at, two_factor_secret	1 FreelancerProfile, 1 ClientProfile, 1 Wallet
FreelancerProfile	user_id, title, bio, skills, hourly_rate, availability, rating	many Proposals, many CatalogProjects
ClientProfile	user_id, company_name, description, location, website	many JobPostings, many CatalogOrders
Agency	id, owner_id, name, description, slug, logo	many FreelancerProfiles (membres)
IdentityVerification	user_id, status, document_type, document_path, reviewed_at	1 User

Tableau 17: Classes du groupe Identité

2 Groupe Marketplace (Offres et Propositions)

Ce groupe modélise le premier circuit de mise en relation (le client publie, le freelance postule) :

Classe	Attributs principaux	Relations clés
JobPosting	id, client_id, title, description, category, budget, type, status	many Proposals, 0..1 Contract
Proposal	id, job_id, freelancer_id, cover_letter, amount, duration, status	1 JobPosting, 1 Freelancer
CatalogProject	id, freelancer_id, title, description, category, status, slug	3 CatalogTiers, many CatalogOrders
CatalogTier	id, catalog_id, tier, price, delivery_days, description, revisions	1 CatalogProject
CatalogOrder	id, client_id, tier_id, status, delivery_at, requirements	1 CatalogTier, 1 Contract

Tableau 18: Classes du groupe Marketplace

3 Groupe Contrats et Jalons

Ce groupe modélise la collaboration une fois l'accord conclu. Le **Contract** est la **classe pivot** autour de laquelle gravitent toutes les activités opérationnelles :

Classe	Attributs principaux	Relations clés
Contract	id, client_id, freelancer_id, source_type, source_id, status, total_amount	many Milestones, 1 Conversation
Milestone	id, contract_id, title, amount, due_date, status, submitted_at	1 Contract, many Transactions
ContractFile	id, contract_id, uploader_id, filename, path, size, version	1 Contract
TimeEntry	id, contract_id, freelancer_id, started_at, stopped_at, duration_minutes	1 Contract
ExtensionRequest	id, contract_id, requested_by, new_due_date, reason, status	1 Contract

Tableau 19: Classes du groupe Contrats et Jalons

4 Groupe Finance (Wallet et Transactions)

Ce groupe constitue le **noyau financier** de la plateforme. Il implémente un **grand livre comptable en double-entrée** garantissant l'intégrité de chaque mouvement de fonds :

Classe	Attributs principaux	Rôle
Wallet	id, user_id, balance_available, balance_escrow, balance_pending, currency	Portefeuille de l'utilisateur
Transaction	id, wallet_id, type, amount, balance_after, reference_type, reference_id, idempotency_key	Écriture comptable unitaire
WithdrawalRequest	id, freelancer_id, amount, method, status, stripe_payout_id, processed_at	Demande de retrait
StripeWebhookEvent	id, stripe_event_id, type, payload, processed, processed_at	Idempotence des webhooks

Tableau 20: Classes du groupe Finance

5 Groupe Communication et Évaluations

Ce groupe couvre la messagerie temps réel entre les parties et le système d'évaluations post-mission, essentiels à la confiance sur la plateforme :

Classe	Attributs principaux	Relations clés
Conversation	id, contract_id, type, last_message_at	2 Users (participants), many Messages
Message	id, conversation_id, sender_id, content, type, read_at	1 Conversation, many Attachments
MessageReaction	id, message_id, user_id, emoji	1 Message
Review	id, contract_id, reviewer_id, reviewee_id, rating, comment, type	1 Contract, 2 Users
Notification	id, user_id, type, data, read_at, notifiable_type, notifiable_id	1 User

Tableau 21: Classes du groupe Communication et Évaluations

V. Diagrammes de séquence dynamiques

Au-delà de la structure, les diagrammes de séquence décrivent les **interactions temporelles** entre objets pour les scénarios les plus critiques. Nous en présentons trois, complétés par une description pas-à-pas mettant en évidence les invariants et les garde-fous.

1 Financement du séquestre et libération de jalon

Ce scénario est le cœur transactionnel de la plateforme. Il met en jeu le **LedgerService**, qui garantit l'atomicité et l'intégrité de chaque mouvement de fonds.

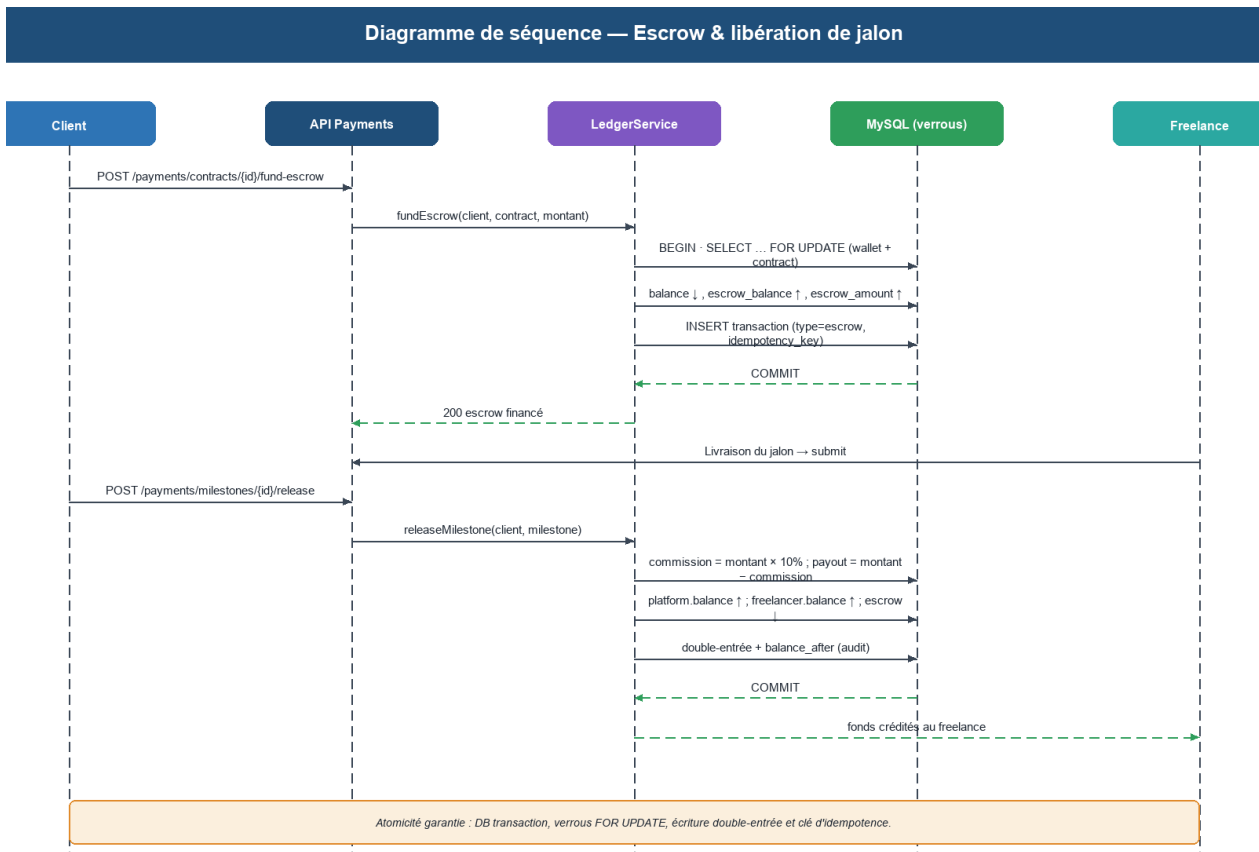


Figure 11: Diagramme de séquence — financement du séquestre et libération de jalon

Le scénario se déroule en deux temps. Lors du **financement du séquestre** :

- 1. Le client appelle **POST /payments/contracts/{id}/fund-escrow** avec le montant souhaité ;
- 2. Le contrôleur délègue au **LedgerService.fundEscrow()** ;
- 3. Le service ouvre une transaction SQL et verrouille le portefeuille du client (**FOR UPDATE**) ;
- 4. Il vérifie que le solde disponible est suffisant ; sinon, il lève une exception métier ;
- 5. Il décrémente **balance** et incrémente **escrow_balance** du portefeuille ;
- 6. Il incrémente **escrow_amount** du contrat et crée une écriture dans **transactions** ;
- 7. La transaction SQL est validée (commit) ; le freelance est notifié.

Lors de la **libération d'un jalon** :

- 1. Le client approuve le jalon (**POST /milestones/{id}/approve**) ;

- 2. `LedgerService.releaseMilestone()` vérifie que le contrat n'est pas en litige ;
- 3. Il calcule la commission (taux paramétrable dans `platform_settings`) ;
- 4. Sous verrou SQL, il effectue trois écritures atomiques : drainage séquestre (client), crédit commission (plateforme), crédit net (freelance) ;
- 5. Chaque écriture enregistre `balance_after` (solde résultant), garantissant la réconciliabilité ;
- 6. Le jalon passe à l'état `paid` et le contrat est réévalué (complet si tous les jalons sont payés) ;
- 7. Le freelance reçoit une notification de crédit et un événement est consigné dans l'activité du contrat.

La **clé d'idempotence** jointe à chaque opération garantit qu'un appel dupliqué (rejeu réseau, double clic) ne produit qu'une seule écriture — propriété critique sur une plateforme financière.

2 Dépôt de fonds via Stripe

Le dépôt s'appuie sur Stripe Checkout. La confirmation du paiement parvient de façon asynchrone par un **webhook** signé, dont l'idempotence est assurée par la table `stripe_webhook_events`.

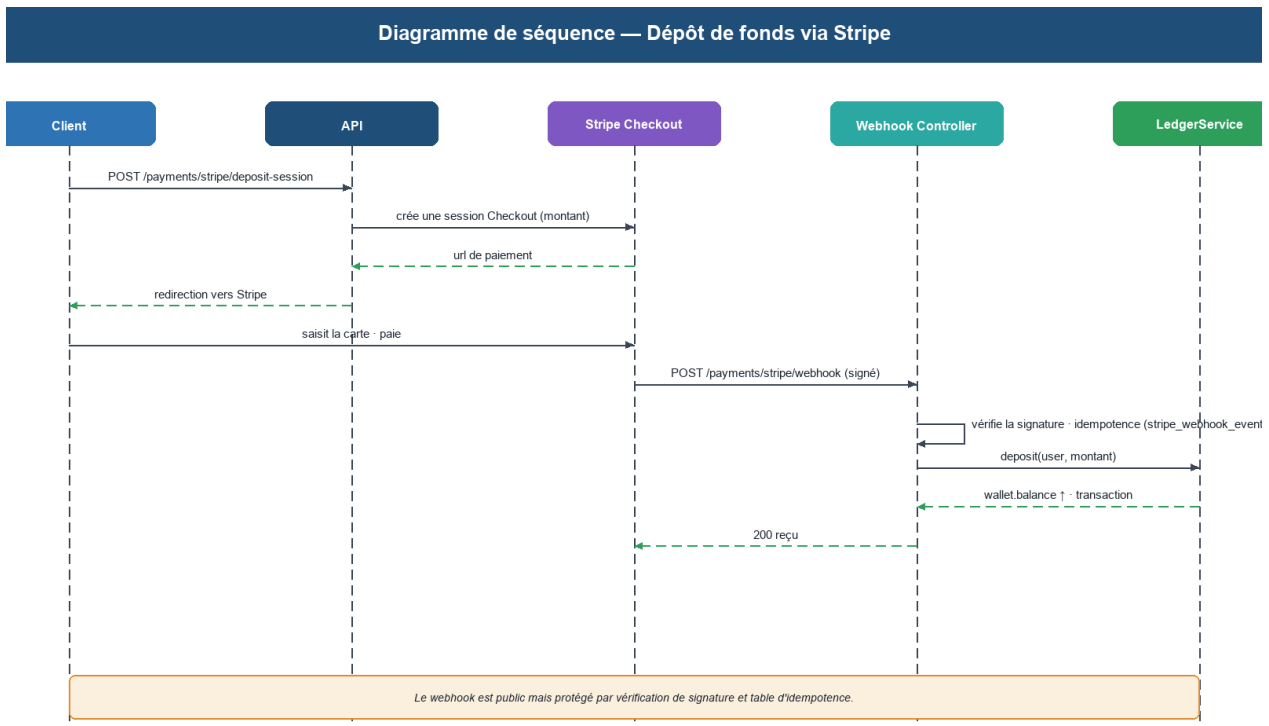


Figure 12: Diagramme de séquence — dépôt de fonds via Stripe

La séquence de dépôt illustre la gestion des événements asynchrones :

- 1. Le client initie un dépôt : **POST /payments/deposit** — le contrôleur demande à Stripe une session Checkout ;
- 2. L'interface React redirige l'utilisateur vers la page de paiement hébergée par Stripe ;
- 3. L'utilisateur saisit ses informations de carte ; Stripe valide le paiement ;
- 4. Stripe envoie un webhook **payment_intent.succeeded** au point d'accès public **/payments/stripe/webhook** ;
- 5. Le contrôleur vérifie la signature HMAC (secret de webhook Stripe) ; toute requête non signée est rejetée ;
- 6. L'idempotence est vérifiée : si l'identifiant de l'événement existe déjà dans **stripe_webhook_events**, la requête est ignorée (renvoi 200 sans traitement) ;
- 7. Le **LedgerService.deposit()** est appelé avec la clé d'idempotence = identifiant de l'événement Stripe ;
- 8. Le portefeuille du client est crédité et une écriture immuable est ajoutée au grand livre.

3 Génération de proposition par IA

Le freelance peut solliciter l'assistant IA pour rédiger une première version de proposition, qu'il édite ensuite. L'appel au modèle est limité en débit pour maîtriser les coûts, et chaque interaction est historisée.

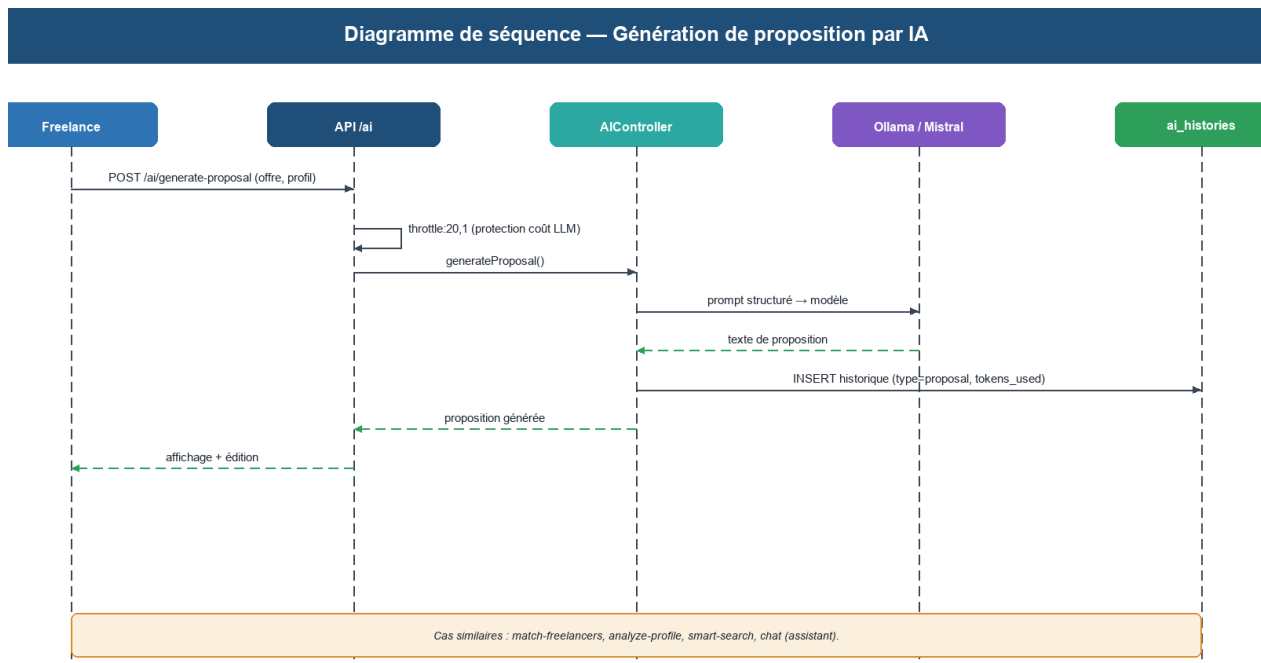


Figure 13: Diagramme de séquence — génération de proposition assistée par IA

La séquence de génération IA met en évidence le rôle de l'historisation et de la limitation :

- 1. Le freelance consulte une offre et clique sur « Générer une proposition avec l'IA » ;
- 2. La requête **POST /ai/generate-proposal** est soumise au middleware de limitation (**throttle:20,1** — 20 requêtes par minute) ;
- 3. **AIController** extrait l'offre et le profil du freelance, puis construit le prompt contextualisé ;
- 4. Le service IA envoie le prompt au serveur Ollama (Mistral) en HTTP local ;
- 5. Le modèle génère le texte de proposition ; la réponse est renvoyée au contrôleur ;
- 6. L'échange (prompt + réponse + tokens utilisés) est persisté dans **ai_histories** ;
- 7. Le texte généré est retourné au front-end ; le freelance le lit, l'adapte et soumet la proposition.

VI. Choix technologiques

Les technologies ont été sélectionnées pour leur maturité, leur écosystème et leur adéquation au besoin. Les tableaux suivants récapitulent la pile retenue.

Back-end	Rôle
Laravel 12 (PHP 8.2)	Framework MVC, API REST, écosystème riche
Laravel Sanctum	Authentification par jeton pour SPA
Laravel Socialite	Authentification OAuth (Google)
Laravel Reverb	Serveur WebSocket pour le temps réel
Stripe PHP SDK	Paielements, Checkout et Connect
MySQL 8 · Eloquent ORM	Base relationnelle et couche d'abstraction
Redis	Cache, sessions et file d'attente

Tableau 22: Pile technologique back-end

Front-end	Rôle
React 19 + Vite	Bibliothèque UI et outil de build rapide
TailwindCSS	Framework CSS utilitaire, design responsive
Zustand	Gestion d'état globale légère
React Router · Axios	Navigation SPA et client HTTP
Laravel Echo · Pusher-js	Réception des événements temps réel
Framer Motion · Recharts	Animations et visualisation de données

Tableau 23: Pile technologique front-end

Services tiers & IA	Rôle
Stripe (Checkout + Connect)	Encaissement des dépôts et versement des retraits
Google OAuth (Socialite)	Connexion sociale en un clic
Ollama + Mistral 7B	Serveur local de modèle de langage (IA générative)
Laravel Reverb	Serveur WebSocket open-source pour le temps réel
Web Push (VAPID)	Notifications push navigateur sans application mobile

Tableau 24: Services tiers et IA intégrés

1 Environnement et outils de développement

Au-delà de la pile applicative, un ensemble d'outils a structuré l'environnement de développement tout au long du projet. Leur choix a été guidé par la productivité, la collaboration et la reproductibilité.

Outil	Catégorie	Usage dans le projet
Visual Studio Code	Éditeur de code	Développement back-end (PHP/Laravel) et front-end (React/JSX) avec extensions dédiées
Composer	Gestionnaire PHP	Installation et mise à jour des dépendances Laravel et paquets PHP
npm + Vite	Build front-end	Gestion des paquets JavaScript et serveur de développement avec rechargement instantané
Git + GitHub	Versionnement	Historique du code, branches, revues et sauvegarde distante du projet
Postman	Test d'API	Test manuel et documentation interactive de tous les points d'API REST
phpMyAdmin	Administration BDD	Visualisation et manipulation de la base de données MySQL en développement
Docker + Compose	Conteneurisation	Environnement de déploiement reproductible (nginx, PHP-FPM, MySQL, Redis, Reverb)
Ollama	IA locale	Serveur de modèle de langage exécuté en local (Mistral 7B) pour les services IA
Stripe CLI	Test paiements	Simulation de webhooks Stripe en local pour tester le cycle de paiement complet
Draw.io / PlantUML	Modélisation	Création des diagrammes UML (cas d'utilisation, classes, séquence, états)
Figma	UI/UX	Maquettage des interfaces avant implémentation React

Tableau 25: Outils et environnement de développement du projet Panda

2 Pourquoi Laravel et React ?

Le choix de **Laravel 12** comme framework back-end repose sur plusieurs avantages décisifs pour ce projet :

- **Écosystème complet** : Sanctum (auth), Sanctum (tokens), Socialite (OAuth), Reverb (WebSockets), Horizon (queues), Telescope (debug) — tout est natif ou premier niveau ;
- **Éloquent ORM** : syntaxe expressive des relations, migrations versionnées, factories et seeders pour les tests automatisés ;
- **Artisan CLI** : génération de code, migrations, tests et tâches planifiées depuis la ligne de commande ;
- **Architecture MVC enrichie** : séparation claire des responsabilités, politiques d'autorisation natives, FormRequests pour la validation centralisée ;

- **Communauté et documentation** : base d'utilisateurs mondiale, documentation exhaustive et mises à jour régulières.

Le choix de **React 19** comme bibliothèque front-end est justifié par :

- **Composants réutilisables** : chaque élément d'interface est un composant autonome, facilitant la maintenance et la cohérence visuelle ;
- **State management** : Zustand offre une gestion d'état globale légère sans la complexité de Redux ;
- **Vite** : serveur de développement ultra-rapide avec HMR (remplacement de module à chaud), idéal pour les itérations rapides ;
- **Écosystème** : React Router pour la navigation SPA, Axios pour les requêtes API, Framer Motion pour les animations, Recharts pour la visualisation ;
- **TailwindCSS** : classes utilitaires permettant un design cohérent et responsive sans écrire de CSS personnalisé.

La combinaison **Laravel REST API + React SPA** est aujourd'hui le standard de facto pour les applications web modernes nécessitant performance, sécurité et évolutivité. Cette architecture découplée permet également d'envisager une application mobile native consommant la même API sans modification du back-end.

VII. *Patterns de conception appliqués*

La qualité de l'architecture de Panda repose non seulement sur les technologies choisies, mais aussi sur l'application rigoureuse de **patterns de conception** (design patterns) éprouvés. Ces patterns structurent le code, réduisent le couplage, améliorent la testabilité et facilitent l'évolution de la plateforme. Cinq patterns principaux sont mobilisés dans le projet.

1 **Pattern Repository — abstraction de la persistance**

Le **pattern Repository** est implémenté via Laravel Eloquent ORM. Chaque modèle Eloquent (**User**, **Contract**, **Wallet**, etc.) joue le rôle de *repository* : il encapsule toutes les opérations de persistance (CRUD, requêtes complexes, scopes) derrière une interface orientée domaine. Les contrôleurs et services ne connaissent jamais la couche SQL directement — ils manipulent des collections d'objets Eloquent.

Modèle Repository	Exemples de méthodes exposées	Avantage
Contract	<code>Contract::with('milestones','files')->where('client_id', \$id)->paginate()</code>	Requêtes complexes encapsulées et réutilisables
Wallet	<code>Wallet::lockForUpdate()->find(\$id)</code>	Verrouillage pessimiste transparent
JobPosting	<code>JobPosting::published()->inCategory(\$cat)->latest()</code>	Scopes chainables, aucun SQL brut dans les contrôleurs
User	<code>User::with('freelancerProfile','wallet')->role('freelancer')->get()</code>	Chargement eager évitant le problème N+1

Tableau 26: Exemples d'application du pattern Repository via Eloquent

L'usage systématique des **Eloquent Scopes** (`scopePublished`, `scopeInCategory`, etc.) matérialise le pattern Repository : la logique de requête est centralisée dans le modèle plutôt que dispersée dans les contrôleurs.

2 Pattern Service Layer — isolation de la logique métier

Le **pattern Service Layer** est le pilier architectural du back-end. Chaque domaine sensible possède un service dédié qui concentre la logique métier, garantissant qu'aucun contrôleur ne contient de règles de gestion critiques. Les services sont injectés par le conteneur d'injection de dépendances de Laravel.

Service	Responsabilités principales	Injecté dans
LedgerService	<code>fundEscrow</code> , <code>releaseMilestone</code> , <code>deposit</code> , <code>withdraw</code> — atomicité et grand livre	PaymentController, MilestoneController
StripeService	<code>createCheckoutSession</code> , <code>createPayout</code> , <code>constructWebhookEvent</code>	PaymentController, WebhookController
NotificationService	<code>sendRealTime</code> , <code>sendPush</code> , <code>sendEmail</code> — façade unifiée	Tous les contrôleurs émettant des notifications
AIService	<code>generateProposal</code> , <code>matchFreelancers</code> , <code>analyzeContract</code> — appels Ollama	AIController
ContractService	<code>createFromProposal</code> , <code>createFromCatalogOrder</code> , <code>checkCompletion</code>	ContractController, MilestoneController

Tableau 27: Services métier du pattern Service Layer

3 Pattern Observer — découplage événementiel

Le **pattern Observer** est natif dans Laravel via le système d'événements et écouteurs (*Events & Listeners*). Lorsqu'une action métier se produit, un événement est émis ; des

écouteurs découplés réagissent de façon asynchrone (via la file d'attente Redis). Ce pattern évite que le code métier ne soit pollué par des préoccupations transverses.

Événement Laravel	Déclencheur	Écouteurs abonnés
MilestoneApproved	Client approuve un jalon	LedgerListener (libère fonds), NotificationListener (email + push), AuditListener (journal)
ContractCompleted	Tous les jalons payés	ReviewReminderListener, TaxDocumentListener, AnalyticsListener
MessageSent	Nouveau message créé	BroadcastListener (WebSocket Reverb), PushNotificationListener
PaymentReceived	Webhook Stripe traité	LedgerListener (crédite wallet), NotificationListener
IdentityVerified	Admin valide un KYC	ProfileActivationListener, EmailNotificationListener

Tableau 28: Événements et écouteurs — pattern Observer

4 Pattern Strategy — algorithmes interchangeables

Le **pattern Strategy** est appliqué dans les composants nécessitant plusieurs variantes d'un même algorithme. Il permet de sélectionner la stratégie à l'exécution sans modifier le code appelant.

Contexte d'application	Stratégies disponibles	Critère de sélection
Méthode de paiement (retrait)	StripeConnectStrategy, BankTransferStrategy	Paramètre <code>method</code> de <code>WithdrawalRequest</code>
Type de contrat	FixedPriceStrategy, HourlyStrategy	Champ <code>type</code> de <code>Contract</code> (fixed / hourly)
Canal de notification	DatabaseChannel, MailChannel, BroadcastChannel, PushChannel	Préférences utilisateur (<code>notification_preferences</code>)
Génération IA	ProposalGenerationStrategy, MatchingStrategy, AnalysisStrategy	Type d'action demandée dans <code>AIController</code>

Tableau 29: Applications du pattern Strategy dans Panda

5 Pattern Factory — construction d'objets complexes

Le **pattern Factory** est appliqué pour la construction des objets complexes dont l'instanciation implique plusieurs étapes de validation et d'initialisation. Laravel propose nativement deux mécanismes : les **Factories** Eloquent (tests) et les méthodes de fabrique dans les services.

Objet créé	Factory / Méthode de création	Logique encapsulée
Contract (depuis Proposal)	ContractService::createFromProposal(Proposal)	Copie champs offre, calcule commission, crée Wallet escrow, notifie
Contract (depuis CatalogOrder)	ContractService::createFromCatalogOrder(CatalogOrder)	Récupère palier, instancie jalons, lie à l'order
Stripe Checkout Session	StripeService::createCheckoutSession(amount, user)	Calcule métadonnées, ligne d'article, URL succès/annulation
Proposition IA	AIService::buildProposalPrompt(job, profile)	Construit le prompt contextuel avec instructions système

Tableau 30: Applications du pattern Factory dans Panda

L'utilisation combinée de ces cinq patterns constitue la **signature architecturale** de Panda : Repository pour l'accès aux données, Service Layer pour la logique métier, Observer pour le découplage des effets de bord, Strategy pour la flexibilité algorithmique, Factory pour la construction d'objets complexes. Cette cohérence architecturale facilite les tests unitaires et d'intégration, et prépare l'évolution de la plateforme vers de nouvelles fonctionnalités.

VIII. Conclusion du chapitre

Ce chapitre a présenté la conception complète de Panda : une architecture découplée et en couches, des mécanismes de sécurité en profondeur, une modélisation objet structurée autour du contrat, des comportements dynamiques formalisés par des diagrammes de séquence, les choix technologiques retenus, et les patterns de conception qui garantissent la maintenabilité du code. Cette conception fournit un plan directeur cohérent et rigoureux. Le chapitre suivant en présente la **réalisation** concrète, illustrée par des extraits de code et la présentation des interfaces.

Chapitre 4 : Réalisation de l'application

Ce chapitre présente la mise en œuvre concrète de Panda. Après avoir décrit l'environnement de développement et l'organisation du code, il détaille — à travers des extraits significatifs — les briques techniques les plus structurantes : le moteur financier, la sécurité, la communication temps réel, l'intégration de Stripe et l'intelligence artificielle. Il s'achève par une présentation des principales interfaces utilisateur de la plateforme.

I. Environnement de développement

Le projet a été développé dans un environnement outillé et reproductible, conforme aux pratiques professionnelles.

Outil	Usage
Visual Studio Code	Éditeur principal (extensions PHP, Laravel, React)
Composer	Gestionnaire de dépendances PHP
npm / Vite	Gestion des paquets et build du front-end React
Git / GitHub	Versionnement du code et historique
MySQL · phpMyAdmin	Base de données et administration
Postman	Tests et documentation des points d'API
Docker · Docker Compose	Conteneurisation et environnement de déploiement
Ollama (Mistral)	Serveur local de modèle de langage pour l'IA

Tableau 31: Environnement et outils de développement

II. Organisation du code source

Le dépôt est organisé en deux applications distinctes — le back-end Laravel et le front-end React — favorisant le découplage. L'arborescence simplifiée est la suivante :

Arborescence simplifiée du projet

```

panda/
  ■■■■ backend-laravel/
  ■   ■■■■ app/
  ■   ■   ■■■■ Http/Controllers/API/ (Auth, Jobs, Contracts, Payments, Chat, AI, Admin...)
  ■   ■   ■■■■ Http/Middleware/ (auth, admin, throttle)
  ■   ■   ■■■■ Models/ (User, Contract, Milestone, Wallet, Transaction...)
  ■   ■   ■■■■ Policies/ (ContractPolicy, MilestonePolicy...)
  ■   ■   ■■■■ Services/ (LedgerService, StripeService, NotificationService...)
  ■   ■   ■■■■ Events/ · Listeners/ (MessageSent, broadcast temps réel)
  ■   ■■■■ database/migrations/ (≈ 30 migrations versionnées)
  ■   ■■■■ routes/api.php (déclaration des points d'API REST)
  ■■■■ frontend-react/
  ■■■■ src/
  ■■■■ pages/ (Landing, Auth, Dashboard, Jobs, Contracts, Payments...)
  ■■■■ components/ (layout, contracts, milestones, ui...)
  ■■■■ store/ · api/ (état Zustand, client Axios)

```

1 Cartographie des principaux points d'API

L'API REST expose une centaine de points d'accès, organisés par domaine fonctionnel et préfixés par `/api`. Les tableaux suivants en présentent un extrait représentatif, illustrant la convention de nommage et la cohérence des verbes HTTP (GET pour lire, POST pour créer/agir, PUT pour modifier, DELETE pour supprimer).

Méthode	Point d'accès	Rôle
POST	/auth/register · /auth/login	Inscription et connexion (limités en débit)
GET	/auth/me	Profil de l'utilisateur authentifié
GET / POST	/jobs · /jobs/{job}/proposals	Offres et propositions
POST	/proposals/{p}/accept · /reject	Traitement d'une proposition
GET	/freelancers · /freelancers/{username}	Annuaire et profil public
GET / POST	/catalog · /catalog/{slug}/orders/checkout	Catalogue et commandes de services

Tableau 32: Extrait de l'API — authentication, offres et catalogue

Méthode	Point d'accès	Rôle
GET	/contracts · /contracts/{c}	Liste et détail des contrats
POST	/contracts/{c}/milestones	Création d'un jalon
POST	/milestones/{m}/submit · /approve	Soumission et validation d'un jalon
POST	/contracts/{c}/dispute · /resolve-dispute	Litige et arbitrage
GET / POST	/contracts/{c}/time/start · /stop	Suivi du temps
GET	/contracts/{c}/pdf	Génération du PDF du contrat

Tableau 33: Extrait de l'API — contrats et jalons

Méthode	Point d'accès	Rôle
GET	/payments/wallet · /overview	Portefeuille et synthèse financière
POST	/payments/contracts/{c}/fund-escrow	Financement du séquestre
POST	/payments/milestones/{m}/release	Libération d'un jalon
POST	/payments/withdrawals	Demande de retrait
POST	/payments/stripe/webhook	Webhook Stripe (signé, idempotent)
POST	/ai/generate-proposal · /match-freelancers	Services d'intelligence artificielle
GET / POST	/admin/* (dashboard, finance, kyc...)	Back-office (réservé à l'administrateur)

Tableau 34: Extrait de l'API — paiements, IA et administration

2 Conventions et bonnes pratiques

Le code respecte un ensemble de conventions garantissant lisibilité et homogénéité : nommage explicite des classes et méthodes, contrôleurs « maigres » déléguant aux services, validation centralisée dans les FormRequests, réponses JSON normalisées, et respect des standards PSR de PHP. Côté React, les composants sont organisés par domaine, l'état global est centralisé (Zustand) et les appels réseau passent par un client Axios unique muni d'intercepteurs (ajout automatique du jeton, gestion des erreurs).

3 Modèles, relations et migrations

Les **modèles Eloquent** déclarent les relations entre entités de façon expressive, ce qui simplifie considérablement les requêtes. Le modèle **Contract**, par exemple, expose ses relations avec le client, le freelance, les jalons et les transactions.

app/Models/Contract.php — relations (extrait)

```
class Contract extends Model
{
    public function client()      { return $this->belongsTo(User::class, 'client_id'); }
    public function freelancer() { return $this->belongsTo(User::class, 'freelancer_id'); }
    public function milestones() { return $this->hasMany(Milestone::class); }
    public function transactions(){ return $this->hasMany(Transaction::class); }
    public function conversation(){ return $this->hasOne(Conversation::class); }
}
```

Le schéma de la base est décrit par des **migrations** versionnées, garantissant qu'il puisse être recréé à l'identique sur n'importe quel environnement. La migration des tables financières illustre l'usage des types décimaux et des contraintes de clés étrangères.

database/migrations — table wallets (extrait)

```
Schema::create('wallets', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->decimal('balance', 12, 2)->default(0);
    $table->decimal('pending_balance', 12, 2)->default(0);
    $table->decimal('escrow_balance', 12, 2)->default(0);
    $table->string('currency')->default('USD');
    $table->timestamps();
});
```

III. Le cœur financier : le service de grand livre (LedgerService)

Le **LedgerService** constitue la **source unique de vérité** pour tout mouvement d'argent dans Panda. Aucun contrôleur n'écrit directement dans les tables **wallets** ou **transactions** : toute opération passe par ce service, qui garantit six propriétés fondamentales.

1. Toute opération s'exécute dans une **transaction SQL** (atomicité : tout ou rien) ;
2. Chaque ligne de portefeuille est **verrouillée** (**SELECT ... FOR UPDATE**) avant mutation, évitant les conditions de course ;
3. Chaque écriture est en **double-entrée** : tout débit a un crédit correspondant ;
4. Chaque transaction enregistre **balance_after** de façon **immuable** pour la réconciliation ;
5. Les **clés d'idempotence** sont honorées : un appel dupliqué renvoie le résultat initial ;
6. Un **invariant** est vérifié : la somme des mouvements d'un portefeuille doit égaler son solde.

1 Financement du séquestre

Lorsqu'un client finance le séquestre, les fonds quittent son solde disponible (**balance**) pour rejoindre son solde bloqué (**escrow_balance**) et le séquestre du contrat (**escrow_amount**). L'extrait suivant illustre la rigueur transactionnelle de l'opération.

[app/Services/LedgerService.php — fundEscrow\(\) \(extrait\)](#)

```

public function fundEscrow(User $client, Contract $contract, float $amount,
    ?string $idempotencyKey = null): Transaction
{
    return DB::transaction(function () use ($client, $contract, $amount, $idempotencyKey) {
        if ($tx = $this->findByIdempotencyKey($idempotencyKey)) return $tx; // rejouable

        $wallet = $this->lockWallet($client);           // SELECT ... FOR UPDATE
        if ((float) $wallet->balance < $amount)
            throw new RuntimeException('Solde insuffisant pour le séquestre');

        $wallet->balance      = bcsub($wallet->balance, $amount, 2); // précision décimale
        $wallet->escrow_balance = bcadd($wallet->escrow_balance, $amount, 2);
        $wallet->save();
        $contract->escrow_amount = bcadd($contract->escrow_amount, $amount, 2);
        $contract->save();

        return Transaction::create([ /* type=escrow, direction=out, balance_after, ... */ ]);
    });
}

```

Le recours à **bcadd/bcsub** (arithmétique de précision arbitraire) plutôt qu'aux flottants natifs est essentiel : il évite les erreurs d'arrondi qui seraient inacceptables sur des montants financiers.

2 Libération d'un jalon et répartition

À l'approbation d'un jalon, le service draine le séquestre, prélève la commission de la plateforme et crédite le freelance du montant net — chaque mouvement donnant lieu à une écriture distincte et idempotente.

app/Services/LedgerService.php — releaseMilestone() (extrait simplifié)

```

$amount      = (float) $milestone->amount;
$feeRate     = (float) PlatformSetting::get('fee.freelancer_pct', 0.10); // 10 %
$commission  = round($amount * $feeRate, 2);
$payout      = round($amount - $commission, 2);

// 1. Drainer le séquestre (client)           → transaction type=release
// 2. Créditer la commission (plateforme)     → transaction type=fee
// 3. Créditer le freelance (net)             → transaction type=credit
// 4. Marquer le jalon comme payé
$milestone->update(['status' => 'paid', 'approved_at' => now()]);

```

Des **garde-fous** protègent l'opération : un jalon doit être *soumis* avant d'être libéré, un contrat *en litige* bloque toute libération, et un séquestre insuffisant interrompt la transaction. Ces vérifications sont effectuées **sous verrou**, à l'abri des accès concurrents.

IV. Authentification et sécurité

Les routes sensibles sont protégées dès leur déclaration. La limitation de débit, par exemple, est appliquée par middleware directement sur les groupes de routes d'authentification.

[routes/api.php — limitation de débit sur l'authentification \(extrait\)](#)

```
Route::prefix('auth')->middleware('throttle:10,1')->group(function () {
    Route::post('/register', [AuthController::class, 'register']);
    Route::post('/login', [AuthController::class, 'login']);
});

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/auth/me', [AuthController::class, 'me']);
    // ... toutes les routes protégées exigent un jeton Bearer valide
});
```

La connexion vérifie les identifiants, applique éventuellement la 2FA, puis émet un jeton personnel que le client conservera pour authentifier ses requêtes suivantes.

[app/Http/Controllers/API/Auth/AuthController.php — login\(\) \(extrait\)](#)

```
public function login(LoginRequest $request)
{
    $user = User::where('email', $request->email)->first();
    if (! $user || ! Hash::check($request->password, $user->password)) {
        return response()->json(['message' => 'Identifiants invalides'], 401);
    }
    $token = $user->createToken('spa')->plainTextToken; // jeton Sanctum
    return response()->json(['user' => $user, 'token' => $token]);
}
```

Les autorisations fines reposent sur des **politiques** Laravel. Ainsi, seules certaines actions — comme la résolution d'un litige — sont réservées à l'administrateur, et la libération d'un jalon est strictement réservée au client du contrat concerné. Cette vérification est dupliquée dans le service métier (défense en profondeur).

[app/Policies/ContractPolicy.php — autorisation \(extrait\)](#)

```
class ContractPolicy
{
    public function fund(User $user, Contract $contract): bool {
        return $user->id === $contract->client_id; // seul le client finance
    }
    public function resolveDispute(User $user, Contract $contract): bool {
        return $user->role === 'admin'; // arbitrage réservé à l'admin
    }
}
```

La validation des entrées est centralisée dans des **FormRequests**, qui définissent les règles de validation et les messages d'erreur, en amont du contrôleur.

[app/Http/Requests/StoreProposalRequest.php — validation \(extrait\)](#)


```
public function rules(): array {
    return [
        'cover_letter' => ['required', 'string', 'min:50', 'max:5000'],
        'bid_amount'   => ['required', 'numeric', 'min:5'],
    ];
}
```

V. Communication temps réel : la messagerie

La messagerie instantanée s'appuie sur **Laravel Reverb**, un serveur WebSocket. Lorsqu'un message est envoyé, un **événement** de diffusion est émis ; le serveur le pousse vers les participants de la conversation, que le client React reçoit via **Laravel Echo** — sans rechargement ni interrogation périodique.

Diffusion d'un message en temps réel (principe)

```
class MessageSent implements ShouldBroadcast
{
    public function __construct(public Message $message) {}

    public function broadcastOn(): array {
        return [new PrivateChannel('conversation.'.$this->message->conversation_id)];
    }
}
// Côté React : echo.private(`conversation.${id}`).listen('MessageSent', cb)
```

La messagerie gère en outre les **accusés de réception** et de lecture, les **réactions**, l'**édition** et la **suppression** de messages, ainsi que les **pièces jointes** — autant de fonctionnalités attendues d'un outil de communication moderne.

Côté React, un *hook* s'abonne au canal privé de la conversation à l'affichage du composant et met à jour la liste des messages en temps réel, sans rechargement.

frontend-react — abonnement temps réel (extrait)

```
useEffect(() => {
    const channel = echo.private(`conversation.${id}`)
        .listen('MessageSent', (e) => setMessages((m) => [...m, e.message]));
    return () => echo.leave(`conversation.${id}`); // nettoyage au démontage
}, [id]);
```

VI. Intégration des paiements Stripe

Stripe gère l'encaissement réel des dépôts. La confirmation arrive par un **webhook**, point d'entrée public mais sécurisé : sa signature est vérifiée, et son idempotence est garantie par la

table `stripe_webhook_events`, qui empêche le double traitement d'un même événement.

Traitement idempotent d'un webhook Stripe (principe)

```
// 1. Vérifier la signature de l'événement (secret du webhook)
$event = Webhook::constructEvent($payload, $sigHeader, $secret);

// 2. Idempotence : ignorer un événement déjà traité
if (StripeWebhookEvent::where('event_id', $event->id)->exists()) {
    return response('déjà traité', 200);
}
StripeWebhookEvent::create(['event_id' => $event->id, 'type' => $event->type]);

// 3. Créditer le portefeuille via le LedgerService (jamais en direct)
$ledger->deposit($user, $amount, idempotencyKey: $event->id);
```

Les versements (retraits) suivent un parcours symétrique : le freelance configure son compte **Stripe Connect**, l'administrateur approuve la demande, puis un transfert est déclenché ; le webhook de confirmation fait basculer le retrait à l'état *complété*. En cas d'échec du transfert, les fonds sont automatiquement re-crédités au solde disponible.

VII. Intelligence artificielle

Panda intègre un assistant fondé sur un **modèle de langage** (Mistral, servi par Ollama). Cinq services sont exposés : génération de proposition, mise en correspondance freelances/offre, analyse de profil, recherche intelligente et assistant conversationnel. Chaque appel est limité en débit (protection des coûts) et historisé dans `ai_histories`.

app/Http/Controllers/API/AI/AIController.php — génération (principe)

```
public function generateProposal(Request $r)
{
    $prompt = $this->buildPrompt($r->job, $r->user->freelancerProfile);
    $text    = $this->ai->complete($prompt);           // appel au modèle (Ollama/Mistral)
    AiHistory::create([
        'user_id' => $r->user->id, 'type' => 'proposal',
        'input' => $prompt, 'output' => $text, 'tokens_used' => $tokens,
    ]);
    return response()->json(['proposal' => $text]);
}
```

Les cinq services d'IA exposés par la plateforme sont récapitulés ci-dessous.

Service	Description
Génération de proposition	Rédige une première version de candidature à partir de l'offre et du profil
Mise en correspondance	Classe les freelances les plus pertinents pour une offre donnée
Analyse de profil	Évalue un profil et suggère des améliorations
Recherche intelligente	Interprète une requête en langage naturel
Assistant conversationnel	Répond aux questions et guide l'utilisateur

Tableau 35: Services d'intelligence artificielle de la plateforme

VIII. Présentation des interfaces utilisateur — 57 pages

Cette section présente en détail l'ensemble des cinquante-sept pages qui composent l'application Panda. Organisées en dix groupes fonctionnels, elles couvrent l'authentification, les pages publiques, les tableaux de bord, les offres, les contrats, les paiements, les profils, la gestion des talents, les paramètres de sécurité et les outils transversaux. L'interface est construite avec React 19 et TailwindCSS 3, respecte une charte graphique unifiée et s'adapte à tous les formats d'écran.

1 Module d'authentification — 6 pages

Le module d'authentification gère l'intégralité du cycle d'identité de l'utilisateur : connexion, inscription, récupération de mot de passe, intégration OAuth Google et parcours d'intégration (onboarding). La sécurité est renforcée par la 2FA côté interface et Laravel Sanctum côté API.

Page	Route	Description fonctionnelle
Login.jsx	/login	Formulaire de connexion e-mail/mot de passe + bouton OAuth Google. Validation en temps réel, messages d'erreur contextuels, lien vers récupération.
Register.jsx	/register	Formulaire d'inscription avec choix du rôle (Client ou Freelance). Validation des champs, confirmation du mot de passe, acceptation des CGU.
Onboarding.jsx	/onboarding	Parcours de bienvenue multi-étapes post-inscription : photo, titre, compétences (freelance) ou besoins (client), confirmation.
ForgotPassword.jsx	/forgot-password	Formulaire de saisie de l'e-mail pour déclencher l'envoi du lien de réinitialisation. Confirmation par message de succès.
ResetPassword.jsx	/reset-password	Formulaire du nouveau mot de passe après clic sur le lien reçu par e-mail (token + e-mail en paramètres URL).
GoogleCallback.jsx	/auth/google/callback	Page de traitement silencieux du retour OAuth Google : stockage du token Sanctum, redirection automatique vers le tableau de bord.

Tableau 36: Pages du module d'authentification

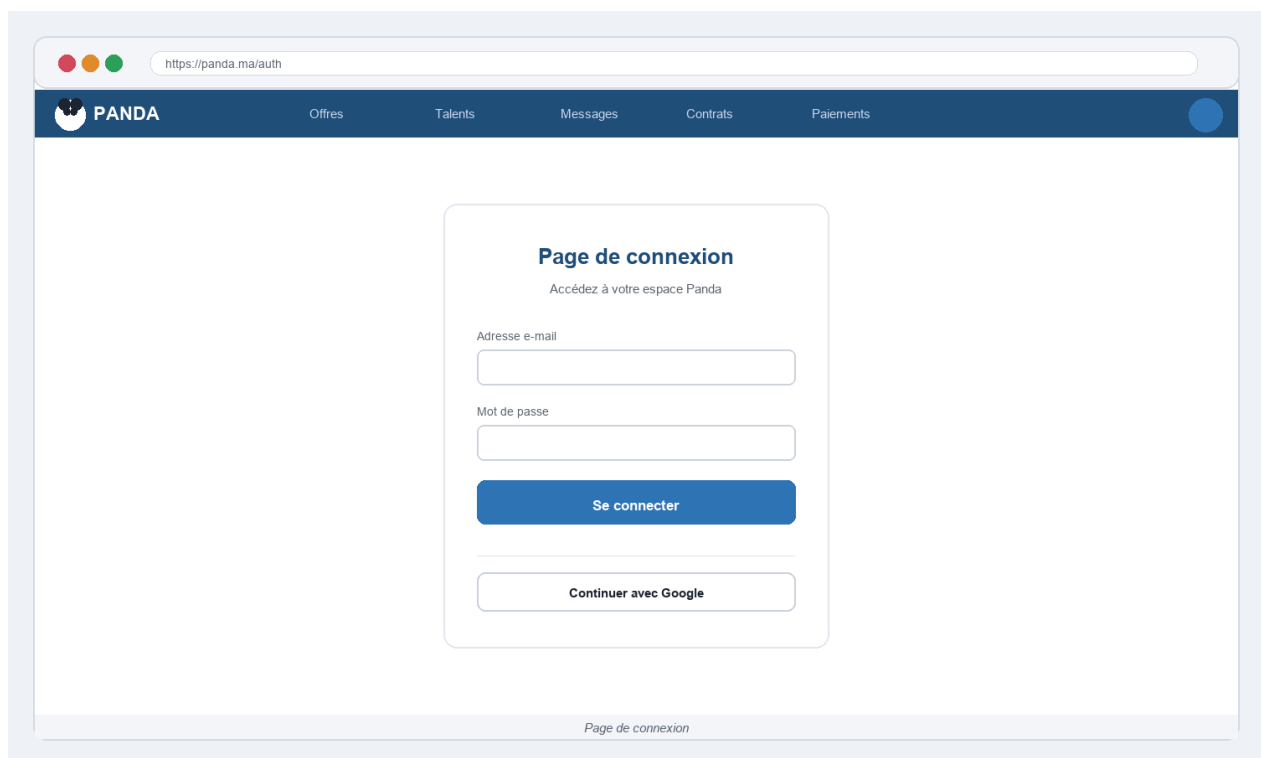


Figure 14: Écran de connexion — e-mail/mot de passe et bouton OAuth Google

L'écran de connexion (Login.jsx) propose deux méthodes d'authentification : la connexion classique par e-mail et mot de passe avec validation en temps réel, et la connexion sociale via Google OAuth en un seul clic. Les messages d'erreur sont précis (identifiants invalides,

compte désactivé) et la navigation vers l'inscription ou la récupération du mot de passe est immédiatement accessible depuis le même écran.

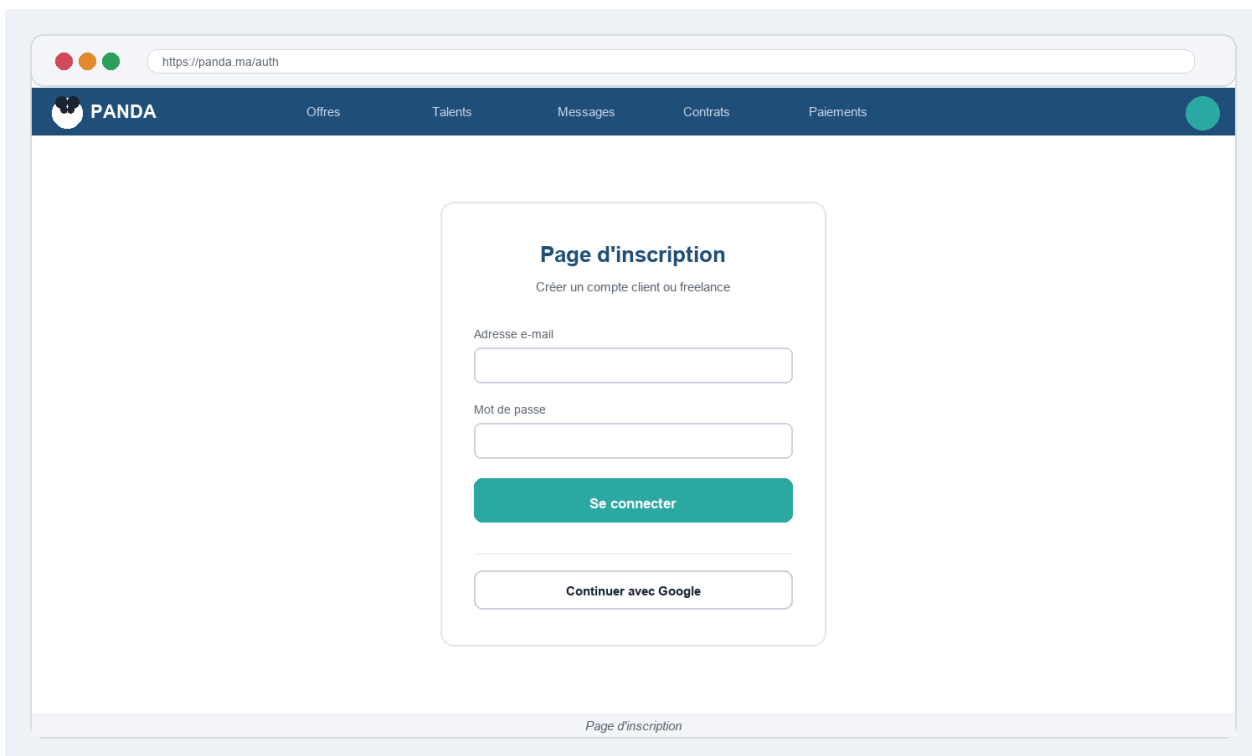


Figure 15: Écran d'inscription — choix du rôle et saisie des informations

L'écran d'inscription (`Register.jsx`) distingue dès la première interaction les deux rôles disponibles : Client (qui publie des offres et recrute) et Freelance (qui soumet des propositions et livre des missions). Ce choix conditionne ensuite l'onboarding et les fonctionnalités accessibles dans l'application. Le parcours `Onboarding.jsx` guide l'utilisateur à travers trois étapes : complétion du profil, sélection des compétences ou catégories de besoins, et confirmation. Ce tunnel réduit le taux d'abandon et améliore la qualité des profils dès le premier jour.

Login.jsx — Le formulaire est centré sur fond blanc avec deux champs (email, mot de passe), un bouton principal 'Se connecter' et un séparateur visuel suivi du bouton OAuth Google. La validation s'effectue en temps réel : le champ vire au rouge si le format email est invalide. En cas d'erreur (code 401), un toast rouge affiche 'Identifiants invalides' sans préciser quel champ est incorrect (protection contre l'énumération). Le hook `useAuth()` redirige automatiquement vers `/dashboard` si l'utilisateur est déjà connecté. Les appels API : `POST /api/auth/login (email + password) → token Sanctum`.

Register.jsx — Étape 1 : choix du rôle (Client ou Freelance) sous forme de deux grandes cartes cliquables avec icône et description du rôle. Étape 2 : prénom, nom, email, mot de passe (jauge de force en temps réel : rouge/orange/vert), confirmation et case CGU obligatoire. Après succès, l'utilisateur reçoit un email de bienvenue et est redirigé vers Onboarding.jsx. API : POST /api/auth/register.

Onboarding.jsx — Barre de progression en trois étapes en haut de page. Étape 1 : photo de profil (drag & drop ou clic), titre professionnel, localisation. Étape 2 Freelance : sélection de 3 à 10 compétences depuis 150+ tags, tarif horaire souhaité. Étape 2 Client : sélection des catégories de besoins. Étape 3 : récapitulatif et confirmation. L'onboarding peut être ignoré et complété depuis les paramètres. API : PUT /api/user/profile.

ForgotPassword.jsx — Page épurée avec un seul champ email et un bouton 'Envoyer le lien de réinitialisation'. Après soumission, un message de succès s'affiche même si l'email n'existe pas (par sécurité, pour éviter l'énumération). Le lien est valide 60 minutes. API : POST /api/auth/forgot-password.

ResetPassword.jsx — Formulaire avec 'Nouveau mot de passe' et 'Confirmer'. Le token et l'email sont lus automatiquement depuis les paramètres URL. Si le token est expiré, un message d'erreur invite à refaire la demande. Après succès, redirection vers Login.jsx avec toast de confirmation. API : POST /api/auth/reset-password.

GoogleCallback.jsx — Page de traitement silencieuse : elle lit le code OAuth renvoyé par Google, appelle l'API Laravel (GET /api/auth/google/callback?code=...) qui échange le code contre un token Sanctum, stocke ce token dans le localStorage et redirige vers /dashboard. En cas d'erreur OAuth, l'utilisateur est renvoyé vers Login.jsx avec un toast d'erreur explicatif.

2 Pages publiques et marketing — 15 pages

Les pages publiques constituent la vitrine commerciale de Panda. Accessibles sans connexion, elles ont pour objectif de convaincre visiteurs et entreprises de rejoindre la plateforme. Elles utilisent des animations Framer Motion, des témoignages réels et des indicateurs de confiance pour maximiser la conversion.

Page	Route	Objectif principal
Landing.jsx	/	Page d'accueil : proposition de valeur, recherche rapide, catégories, indicateurs de confiance, témoignages et appels à l'action.

Page	Route	Objectif principal
Pricing.jsx	/pricing	Grille tarifaire : plans Gratuit, Pro, Entreprise — comparatif des fonctionnalités et bouton de souscription. Bascule mensuel/annuel.
Agencies.jsx	/agencies	Offres pour les agences : gestion multi-utilisateurs, reporting avancé, support dédié, formulaire de contact commercial.
Enterprise.jsx	/enterprise	Page grands comptes : SLA personnalisé, intégrations SSO, pilote gratuit et formulaire de contact dédié.
FindTalent.jsx	/find-talent	Présentation du service de mise en relation : comment fonctionne la recherche de freelances, garanties qualité et sécurité.
GetOutcomes.jsx	/get-outcomes	Page bénéfices clients : retour sur investissement, études de cas, promesses de délais et résultats chiffrés.
BuildWebsite.jsx	/build-website	Verticale spécialisée développement web : sous-page FindTalent orientée création et refonte de sites.
ScalePaidAds.jsx	/scale-paid-ads	Verticale marketing digital : sous-page orientée gestion de campagnes publicitaires et SEA.
HandleSupport.jsx	/handle-support	Verticale support client : sous-page orientée assistance en ligne et service après-vente externalisé.
Blog.jsx	/blog	Liste des articles : conseils freelance, tendances du marché, tutoriels Panda. Filtres par thème.
HowItWorks.jsx	/how-it-works	Explication pas-à-pas du fonctionnement : trois étapes pour les clients et trois étapes pour les freelances.
Reviews.jsx	/reviews	Témoignages vérifiés : grille de cartes avec note, photo et verbatim d'utilisateurs réels.
SuccessStories.jsx	/success-stories	Études de cas : missions réussies, résultats chiffrés, profils des entreprises bénéficiaires.
Updates.jsx	/updates	Changelog public : nouvelles fonctionnalités, corrections et améliorations récentes de la plateforme.
Research.jsx	/research	Ressources sectorielles publiées par Panda : rapports PDF, infographies et données de marché.

Tableau 37: Pages publiques et marketing de la plateforme Panda

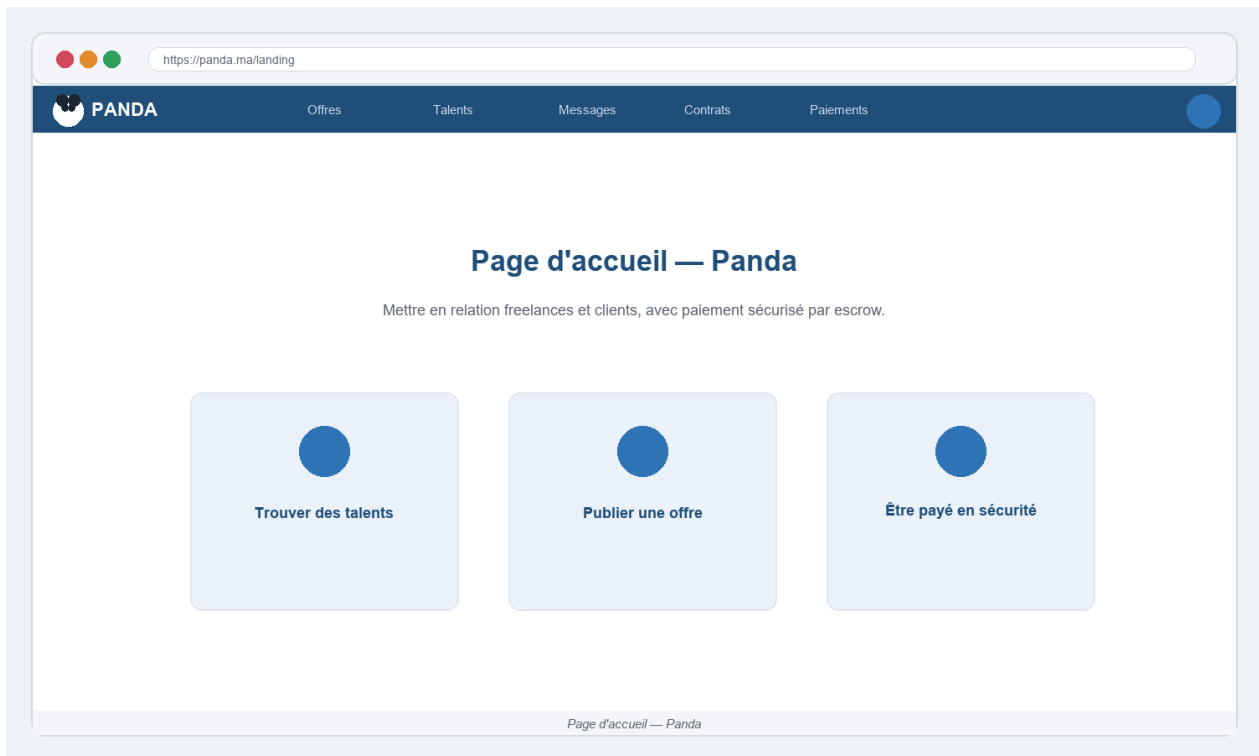


Figure 16: Page d'accueil de la plateforme Panda

La page d'accueil (Landing.jsx) est la première interface que le visiteur découvre. Elle est structurée en sections verticales successives : hero avec barre de recherche, compteurs de confiance animés (nombre de freelances, missions livrées, taux de satisfaction), catégories de compétences les plus demandées, témoignages clients, comparatif tarifaire et pied de page complet. La barre de recherche principale bascule entre deux modes : 'Trouver un talent' (vers FreelancerMarketplace) et 'Trouver une mission' (vers JobsMarketplace), positionnant immédiatement l'utilisateur dans son parcours.

La page Pricing.jsx adopte la structure classique des plateformes SaaS : trois colonnes (Gratuit, Pro, Entreprise) avec un tableau de comparaison des fonctionnalités et une bascule mensuel/annuel affichant l'économie réalisée. Les pages Agencies.jsx et Enterprise.jsx ciblent respectivement les agences multi-utilisateurs et les grands comptes, avec des formulaires de contact commercial dédiés. Les trois verticales métier (BuildWebsite, ScalePaidAds, HandleSupport) déclinent la page FindTalent pour des segments spécifiques, améliorant ainsi le référencement naturel par mots-clés ciblés.

Landing.jsx — La hero section affiche un titre accrocheur, une sous-description de valeur et une barre de recherche à double mode (talents / missions). Quatre compteurs animés (useEffect + requestAnimationFrame) s'incrémentent à l'arrivée sur la page : 12 000+

freelances, 8 500+ missions livrées, 98% de satisfaction, paiements sécurisés. Une section Catégories affiche 12 domaines de compétences (Développement, Design, Marketing, Rédaction...) avec icônes et compteur d'offres actives. La section Témoignages affiche 6 cartes avec photo, nom, entreprise et verbatim. Le pied de page propose des liens vers toutes les ressources de la plateforme.

Pricing.jsx — Trois colonnes (Gratuit 0€/mois, Pro 29€/mois, Entreprise sur devis) avec indicateur 'Recommandé' sur le plan Pro. Un toggle mensuel/annuel affiche -20% sur les plans annuels. Chaque colonne liste les fonctionnalités incluses avec icône checkmark vert (inclus) ou croix grise (non inclus). Un tableau de comparaison détaillé en bas de page compare 25 fonctionnalités. FAQ en accordéon clôt la page.

Agencies.jsx — Page dédiée aux agences numériques et studios de freelances. Elle présente les avantages multi-utilisateurs : jusqu'à 10 comptes connectés, tableaux de bord partagés, reporting centralisé et support prioritaire. Un formulaire de contact commercial collect le nom, l'email professionnel, la taille de l'équipe et les besoins spécifiques. Les logos des agences partenaires défilent en animation CSS.

Enterprise.jsx — Landing page orientée grands comptes : SLA 99,9% garanti, intégration SSO (SAML/OAuth), API privée dédiée, manager de compte attitré et pilote gratuit de 30 jours. Le formulaire Enterprise envoie une demande à l'équipe commerciale via l'API POST /api/enterprise/contact.

HowItWorks.jsx — Présentation en deux colonnes : côté Client (3 étapes numérotées avec icône : Publier > Choisir > Payer) et côté Freelance (3 étapes : Postuler > Livrer > Être payé). Des animations au scroll (Framer Motion) font apparaître progressivement les étapes. Une FAQ de 8 questions répond aux objections courantes.

Reviews.jsx — Grille de 12 témoignages vérifiés avec note en étoiles (4 à 5 sur 5), photo de profil, prénom, titre et verbatim de 2 à 4 lignes. Un filtre par type (Client / Freelance) et un filtre par note permettent d'affiner l'affichage. Une note globale de 4.9/5 est mise en avant en haut de page avec le nombre total d'avis.

SuccessStories.jsx — Trois études de cas détaillées avec : nom de l'entreprise, secteur, défi initial, solution Panda, résultats chiffrés (ex. : +40% de productivité, -30% de coûts RH) et citation du responsable. Chaque case study s'ouvre en pleine page avec une mise en page

magazine. Un CTA 'Créer votre compte' clôt chaque story.

Blog.jsx — Liste paginée d'articles avec vignette, titre, extrait, auteur, date et catégorie (badge coloré). Filtres par catégorie (Conseils, Tendances, Tutoriels, Actualités Panda) et barre de recherche plein-texte. Chaque article s'ouvre en page dédiée avec rendu Markdown, table des matières flottante et articles connexes.

Updates.jsx — Changelog public chronologique : chaque release est une section avec version (ex. v1.0.0), date, liste des nouveautés (feat), corrections (fix) et améliorations (improve) avec icônes colorées. Les entrées récentes sont affichées en haut avec un badge 'Nouveau'.

Research.jsx — Bibliothèque de ressources : rapports PDF téléchargeables (marché du freelance, tendances tech, baromètre salarial), infographies partageables et données statistiques sectorielles. Chaque ressource affiche le format, la taille et la date de publication. Le téléchargement requiert une inscription.

FindTalent.jsx, GetOutcomes.jsx, BuildWebsite.jsx, ScalePaidAds.jsx, HandleSupport.jsx — Ces cinq pages partagent une structure commune : hero orienté bénéfices (titre + sous-titre + CTA), liste de 4 à 6 avantages avec icône et description, capture d'écran animée de l'interface, témoignage client lié au segment, et section FAQ de 5 questions. Leur contenu est adapté au segment cible (développeurs web, marketeurs, équipes support). Chaque page est optimisée pour le référencement avec des balises meta distinctes et du contenu unique.

3 Tableaux de bord — 3 pages

Chaque rôle dispose d'un tableau de bord personnalisé qui centralise les indicateurs clés et les actions courantes. Ces tableaux de bord sont construits avec la bibliothèque Recharts pour les graphiques et se rafraîchissent périodiquement via React Query pour afficher des données proches du temps réel.

Page	Route	Indicateurs principaux
ClientDashboard.jsx	/client/dashboard	Solde disponible, fonds en séquestre, offres publiées, propositions reçues, contrats actifs, graphique mensuel des dépenses.
FreelancerDashboard.jsx	/freelancer/dashboard	Gains totaux, propositions envoyées, contrats actifs, note moyenne, missions livrées, graphique de revenus mensuel.
AdminDashboard.jsx	/admin/dashboard	Utilisateurs, transactions du jour, retraits en attente, files de modération et KYC, revenus et commissions de la plateforme.

Tableau 38: Tableaux de bord par rôle utilisateur

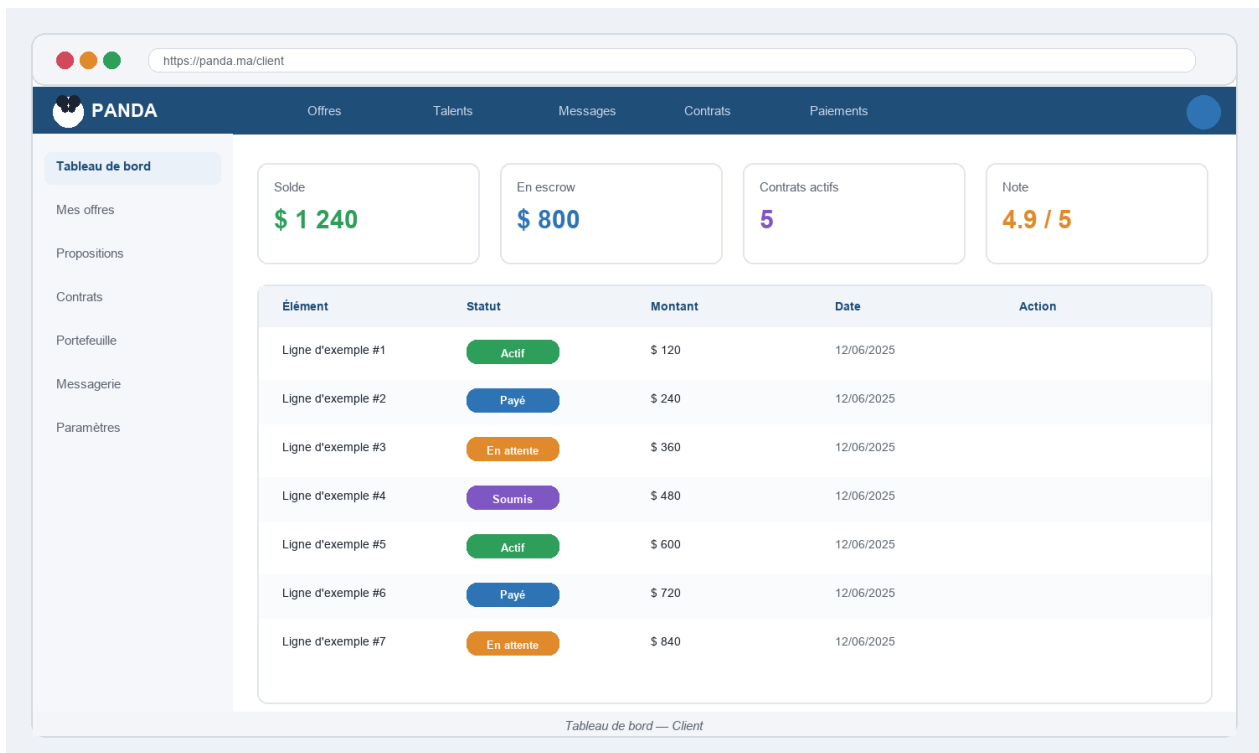


Figure 17: Tableau de bord du client — solde, offres actives et contrats

Le tableau de bord client (ClientDashboard.jsx) affiche en haut de page quatre widgets colorés : le solde du portefeuille disponible, les fonds en séquestre (bloqués pour des jalons en cours), le nombre d'offres publiées actives et les propositions en attente d'examen. Un graphique en barres illustre l'évolution mensuelle des dépenses sur les douze derniers mois, accompagné d'un tableau des contrats récents avec leur statut.

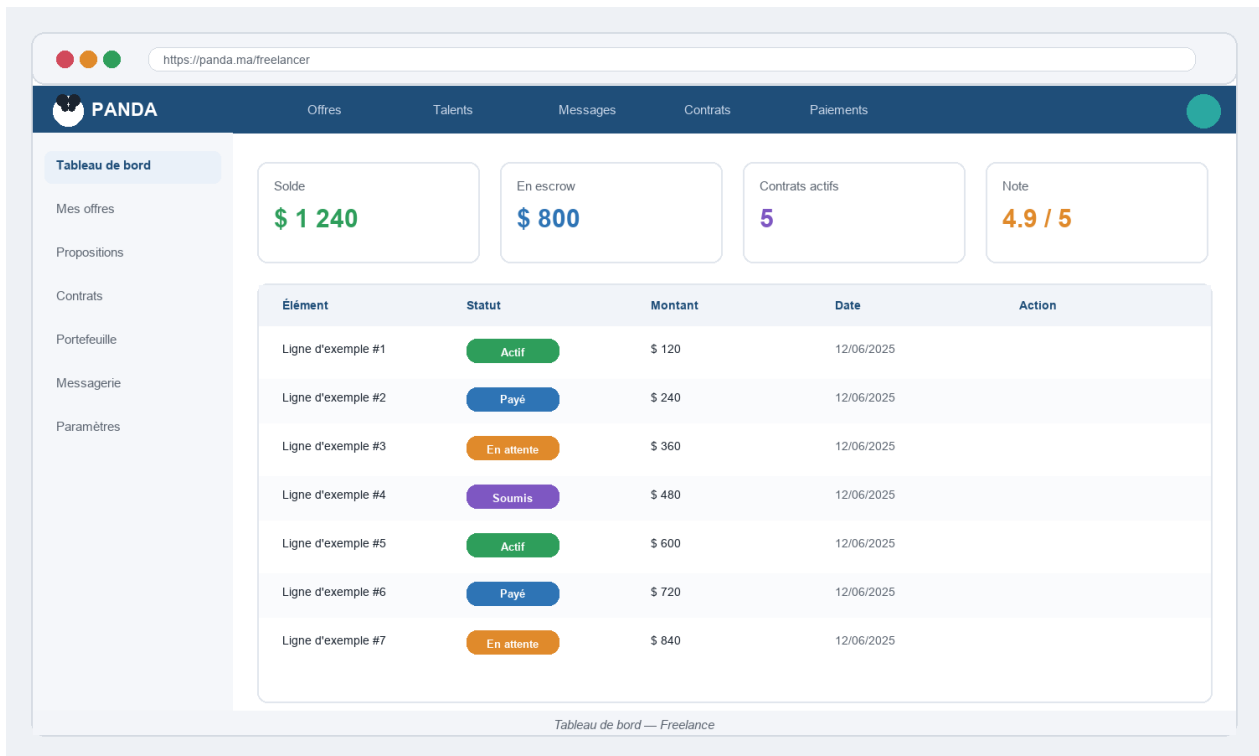


Figure 18: Tableau de bord du freelance — gains, propositions et missions

Le tableau de bord freelance (FreelancerDashboard.jsx) met en avant les revenus avec un graphique temporel et des indicateurs de performance : taux d'acceptation des propositions, note moyenne sur cinq étoiles, nombre de missions terminées. Des alertes rapides signalent les jalons en attente de soumission et les messages non lus.

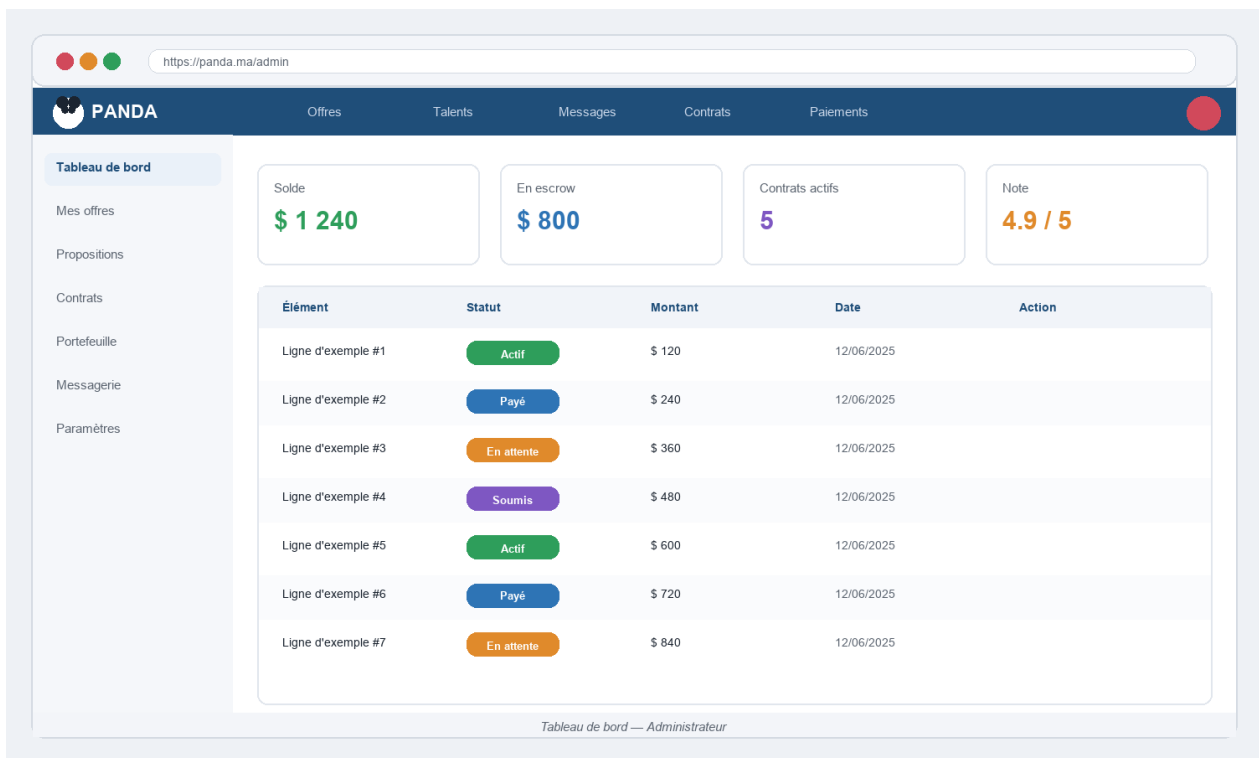


Figure 19: Tableau de bord administrateur — indicateurs de la plateforme

Le tableau de bord administrateur (AdminDashboard.jsx) offre une vue opérationnelle complète : taux de croissance des inscrits, volume de transactions du jour, retraits en attente d'approbation, files de modération des offres et de vérification KYC. Deux graphiques superposés comparent revenus et commissions de la plateforme sur une période glissante configurable.

4 Place de marché des offres — 8 pages

Le module Jobs représente le cœur de l'activité marketplace. Les clients publient des offres structurées ; les freelances les parcourent, les filtrent et y répondent avec des propositions personnalisées. L'IA peut assister la rédaction de la proposition en analysant le profil du freelance et le cahier des charges de l'offre.

Page	Route	Description
JobsMarketplace.jsx	/jobs	Liste paginée des offres avec filtres multi-critères (catégorie, budget min/max, type, durée, compétences). Cartes synthétiques avec statut.
JobDetail.jsx	/jobs/:id	Détail complet d'une offre : description, budget, compétences, freelances suggérés par l'IA, formulaire de proposition.
PostJob.jsx	/post-job	Formulaire multi-étapes de publication : titre, catégorie, budget, compétences requises, description, pièces jointes.
CategoryJobs.jsx	/jobs/category/:slug	Sous-liste filtrée par catégorie avec compteur d'offres actives et description de la catégorie.
SavedJobs.jsx	/saved-jobs	Liste personnelle des offres enregistrées en favori par le freelance, avec bouton de candidature rapide.
JobProposals.jsx	/jobs/:id/proposals	Vue réservée au client : toutes les propositions reçues pour une offre, comparaison et sélection du freelance retenu.
MyJobs.jsx	/my-jobs	Vue du client : toutes ses offres publiées avec statut (active, en cours, clôturée) et actions rapides (éditer, clôturer).
MyProposals.jsx	/my-proposals	Vue du freelance : toutes ses propositions avec état (en attente, acceptée, rejetée) et accès au détail de chacune.

Tableau 39: Pages du module Offres et propositions

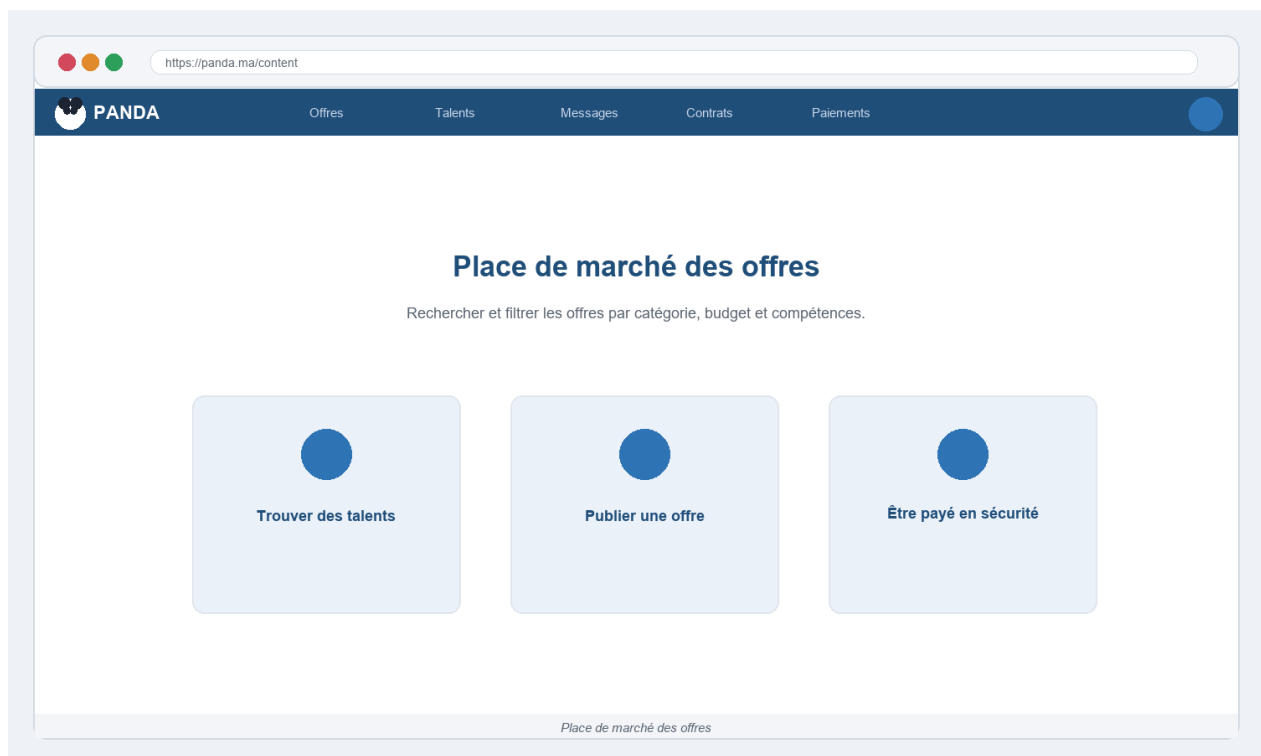


Figure 20: Place de marché des offres — filtres, recherche et cartes d'offres

La page `JobsMarketplace.jsx` est la vitrine des missions disponibles. Les offres sont présentées sous forme de cartes avec les informations essentielles : titre, client, catégorie, budget (fixe ou horaire), date de publication et nombre de propositions. Un panneau de filtres permet d'affiner la recherche par catégorie, fourchette de budget, durée estimée, compétences requises et type de contrat.

La page `JobDetail.jsx` affiche le cahier des charges complet de la mission : description riche, compétences sous forme de badges, budget et délai souhaités. Elle propose une liste de freelances recommandés par l'IA (calcul de correspondance profil/offre). Le bouton 'Soumettre une proposition' ouvre un formulaire où le freelance peut rédiger sa lettre de motivation et proposer son tarif. Le bouton 'Rédiger avec l'IA' génère automatiquement une proposition personnalisée à partir du profil et de l'offre.

`PostJob.jsx` guide le client à travers un tunnel en quatre étapes validées : informations générales (titre, catégorie, type), description et compétences requises, budget et délai, puis pièces jointes optionnelles avant publication. Ce formulaire progressif réduit les erreurs et améliore la qualité des offres publiées.

JobsMarketplace.jsx — Disposition en deux colonnes : panneau de filtres à gauche (catégorie, budget min/max, durée, type de contrat, compétences requises, date de publication) et liste de cartes à droite paginées par 12. Chaque carte affiche le titre, l'avatar du client (anonymisé), la catégorie avec badge coloré, le budget, la date et un indicateur '3 propositions'. Un bouton 'Enregistrer' (étoile) ajoute l'offre aux favoris (SavedJobs) sans quitter la page. API : GET `/api/jobs`.

JobDetail.jsx — En haut : titre, client (avatar + prénom), budget, délai et catégorie. Corps : description riche en Markdown, liste de compétences requises sous forme de badges. Sidebar droite : bouton 'Soumettre une proposition' (formulaire en modale avec lettre de motivation, tarif proposé, délai estimé), bouton 'Rédiger avec l'IA' (appel POST `/api/ai/generate-proposal`), section 'Freelances suggérés par l'IA' (3 profils avec score de correspondance). API : GET `/api/jobs/:id`.

PostJob.jsx — Tunnel en 4 étapes avec indicateur de progression. Étape 1 : titre (min. 10 chars), catégorie (select), type (Fixe/Horaire). Étape 2 : description (éditeur Markdown, min. 100 chars), compétences requises (multi-select avec recherche). Étape 3 : budget (min. 10€), délai souhaité (calendrier), questions pour les candidats (optionnel). Étape 4 :

prévisualisation et publication. API : POST /api/jobs.

CategoryJobs.jsx — Identique à JobsMarketplace mais pré-filtré sur la catégorie du slug URL. En-tête avec nom de la catégorie, icône, description et compteur d'offres actives. Les filtres restants sont disponibles mais la catégorie est verrouillée. Breadcrumb : Accueil > Offres > Développement Web. API : GET /api/jobs?category=slug.

SavedJobs.jsx — Liste personnelle des offres mises en favori. Chaque entrée reprend la carte d'offre complète + bouton 'Retirer des favoris' et bouton 'Postuler'. Si l'offre a été clôturée depuis l'enregistrement, un badge 'Expirée' s'affiche en gris. API : GET /api/saved-jobs, DELETE /api/saved-jobs/:id.

JobProposals.jsx — Réserve au client propriétaire de l'offre. Liste de toutes les propositions reçues sous forme de cartes : avatar + nom du freelance, note moyenne, tarif proposé, délai proposé et extrait de la lettre de motivation. Boutons 'Accepter' (crée le contrat) et 'Refuser' avec motif optionnel. Filtre par statut (toutes, non lues, présélectionnées). API : GET /api/jobs/:id/proposals.

MyJobs.jsx — Vue client uniquement. Onglets : Actives, En cours, Clôturées. Chaque carte d'offre affiche le nombre de propositions reçues, le statut actuel et les actions disponibles : Éditer, Clôturer, Voir les propositions, Dupliquer. Un indicateur rouge signale les nouvelles propositions non lues. API : GET /api/my-jobs.

MyProposals.jsx — Vue freelance uniquement. Liste de toutes les propositions soumises avec leur statut coloré : En attente (jaune), Acceptée (vert), Refusée (rouge), Contrat créé (bleu). Chaque ligne affiche le titre de l'offre, le tarif proposé, la date de soumission et un lien vers le contrat si acceptée. API : GET /api/my-proposals.

5 Contrats et jalons — 3 pages

Le module de gestion des contrats structure la collaboration client/freelance après l'acceptation d'une proposition. Chaque contrat est découpé en jalons (milestones) dont les fonds sont séquestrés à l'avance. Ce mécanisme garantit la sécurité des paiements et clarifie les attentes mutuelles.

Page	Route	Fonctionnalités clés
ContractsList.jsx	/contracts	Liste consolidée des contrats (actifs, en attente, terminés, annulés) avec filtres et résumé financier du montant en séquestre.
ContractDetails.jsx	/contracts/:id	Vue complète en onglets : jalons, fichiers versionnés, suivi du temps, extensions de délai, journal d'activité, messagerie et export PDF.
MilestoneDetails.jsx	/contracts/:id/milestones/:mid	Détail d'un jalon individuel : description, montant séquestré, statut, soumission des livrables par le freelance, validation/rejet par le client.

Tableau 40: Pages du module Contrats et jalons

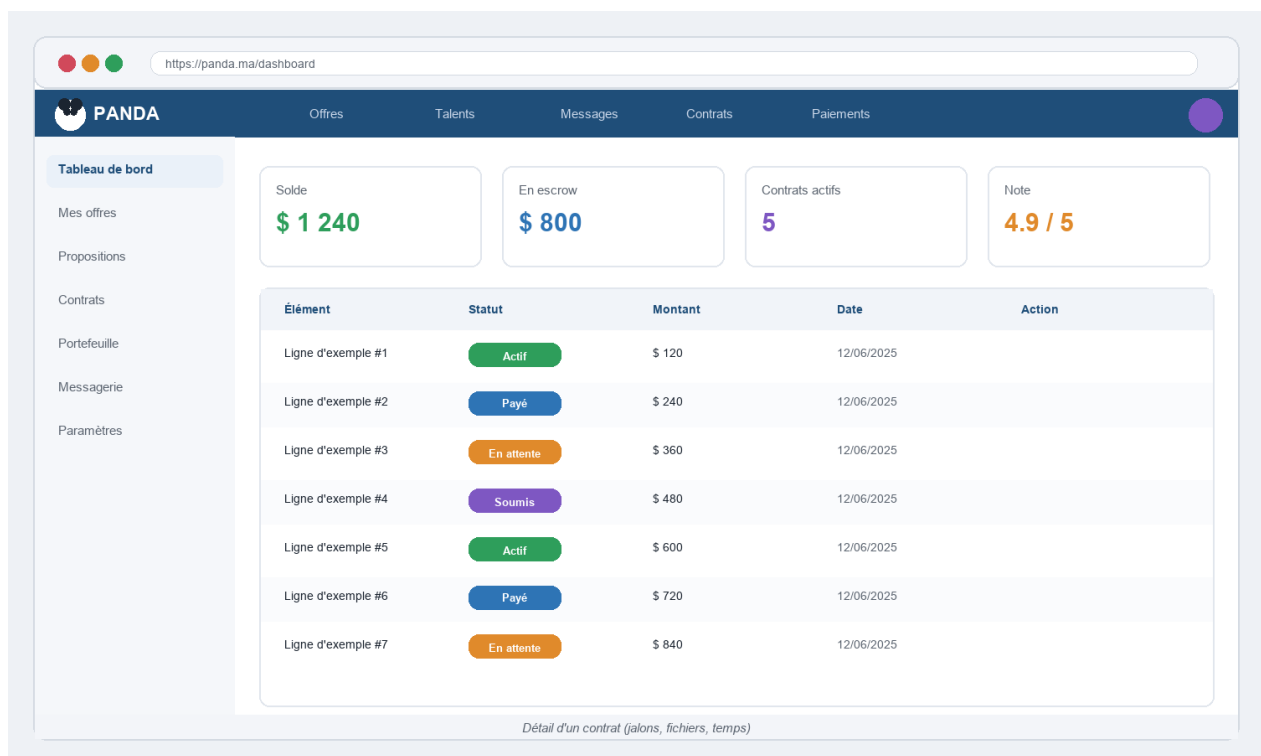


Figure 21: Détail d'un contrat — onglets jalons, fichiers et suivi du temps

La page ContractDetails.jsx est la plus riche de la plateforme. Elle s'organise en six onglets distincts :

- **Jalons** : liste chronologique avec statut (créé, en cours, soumis, validé, rejeté). Le freelance soumet ses livrables ; le client valide ou rejette avec commentaire ;
- **Fichiers** : espace de dépôt et téléchargement des livrables versionnés. Chaque fichier est horodaté et associé à un jalon spécifique ;
- **Temps** : chronomètre intégré pour les contrats horaires et récapitulatif hebdomadaire des heures déclarées et approuvées ;

- **Extensions** : formulaire de demande de prolongation de délai avec motivation, et réponse (accepter/refuser) du client ;
- **Activité** : journal immuable et chronologique de tous les événements du contrat (proposition acceptée, jalon validé, paiement libéré...) ;
- **Analyse** : statistiques du contrat (avancement, budget consommé, heures) et assistant IA contextuel pour résoudre d'éventuels litiges.

ContractsList.jsx offre une vue consolidée de tous les contrats. Des onglets séparent les contrats actifs, en attente, terminés et annulés. Un résumé financier en haut indique le montant total en séquestre et les paiements en attente de validation.

ContractsList.jsx — En-tête avec trois badges de synthèse financière : total en séquestre, paiements libérés du mois, retraits en attente. Tableau avec colonnes : Contrat (ID + titre de l'offre), Partenaire (client ou freelance selon le rôle), Montant total, Séquestre actuel, Statut, Date de création, Actions. Filtres par statut et période. Export CSV disponible. API : GET /api/contracts.

ContractDetails.jsx — Navigation en six onglets avec badge de comptage sur chaque onglet (ex : 'Jalons (3)', 'Fichiers (7)'). Onglet Jalons : timeline verticale avec chaque jalon, son montant, son statut (chip coloré), la date cible et les actions disponibles (soumettre/valider/rejeter). Onglet Fichiers : liste de fichiers avec icône type, nom, taille, date et bouton de téléchargement. Onglet Temps : tableau semainier des heures avec totaux et bouton chrono. Onglet Extensions : historique des demandes avec statut. Onglet Activité : feed chronologique avec avatar, action et date. Un bouton 'Exporter PDF' en haut à droite génère le PDF du contrat via GET /api/contracts/:id/pdf.

MilestoneDetails.jsx — Page dédiée à un jalon unique. En haut : titre du jalon, montant, date cible, statut. Ci-dessous, côté Freelance : liste de fichiers de livraison avec zone de dépôt (drag & drop), champ 'Note de livraison' en Markdown et bouton 'Soumettre le jalon'. Côté Client : affichage des fichiers soumis, champ 'Commentaire' et deux boutons : 'Valider' (libère le paiement du jalon via LedgerService) et 'Rejeter avec motif' (renvoie en correction). API : POST /api/milestones/:id/submit, POST /api/milestones/:id/approve.

6 Paiements et finances — 6 pages

Le module financier de Panda s'appuie sur Stripe pour les paiements (Checkout), les virements (Connect) et la gestion des abonnements. Un portefeuille interne trace chaque

mouvement côté utilisateur ; une interface dédiée pilote les retraits côté administration.

Page	Route	Description
Payments.jsx	/payments	Portefeuille : trois soldes (disponible, séquestre, en attente), dépôt Stripe, historique des transactions filtrable, demande de retrait.
Withdraw.jsx	/payments/withdraw	Formulaire de retrait : montant, compte Stripe Connect de destination, confirmation et suivi de l'état (en attente, approuvé, rejeté).
PaymentSuccess.jsx	/payment/success	Page de confirmation après un dépôt Stripe réussi : montant crédité, nouveau solde, bouton retour au portefeuille.
PaymentCancel.jsx	/payment/cancel	Page d'annulation si l'utilisateur abandonne le tunnel Stripe Checkout avant de valider le paiement.
StripeConnectReturn.jsx	/stripe/connect/return	Page de retour après l'onboarding Stripe Connect : confirmation de la configuration du compte pour recevoir des virements.
AdminFinance.jsx	/admin/finance	Interface admin : vue globale des transactions, approbation/rejet des retraits en attente, export CSV de l'historique.

Tableau 41: Pages du module Paiements et finances

The screenshot displays the PANDA user interface. At the top, a navigation bar includes links for Offres, Talents, Messages, Contrats, and Paiements. A sidebar on the left lists menu items: Tableau de bord, Mes offres, Propositions, Contrats, Portefeuille, Messagerie, and Paramètres. The main content area features four summary cards: Solde (\$1,240), En escrow (\$800), Contrats actifs (5), and Note (4.9/5). Below these is a table of transactions with columns for Élément, Statut, Montant, Date, and Action. The table lists seven example transactions with various statuses like 'Actif', 'Payé', 'En attente', and 'Soumis'.

Élément	Statut	Montant	Date	Action
Ligne d'exemple #1	Actif	\$ 120	12/06/2025	
Ligne d'exemple #2	Payé	\$ 240	12/06/2025	
Ligne d'exemple #3	En attente	\$ 360	12/06/2025	
Ligne d'exemple #4	Soumis	\$ 480	12/06/2025	
Ligne d'exemple #5	Actif	\$ 600	12/06/2025	
Ligne d'exemple #6	Payé	\$ 720	12/06/2025	
Ligne d'exemple #7	En attente	\$ 840	12/06/2025	

Figure 22: Portefeuille utilisateur — soldes, transactions et retraits

La page `Payments.jsx` présente le portefeuille avec trois soldes distincts : le solde disponible (fonds libres), le solde en séquestre (fonds bloqués pour des jalons actifs) et le solde en attente (fonds libérés en cours de transfert). Un tableau de l'historique des transactions est filtrable par type (dépôt, jalon, retrait, commission), période et montant. Le bouton 'Déposer des fonds' redirige vers Stripe Checkout ; après paiement, Stripe renvoie l'utilisateur sur `PaymentSuccess.jsx` qui confirme l'opération.

Le formulaire `Withdraw.jsx` vérifie d'abord que l'utilisateur a configuré son compte Stripe Connect (onboarding guidé via `StripeConnectReturn.jsx`) avant de permettre tout retrait. Un délai de traitement de 1 à 3 jours ouvrables est affiché clairement. La page `AdminFinance.jsx` est réservée à l'administrateur : tableau de bord financier global, approbation ou rejet de chaque demande de retrait, et export CSV complet.

Payments.jsx — En-tête avec trois widgets : Solde disponible (bleu, cliquable pour déposer), Séquestre (orange, avec info-bulle explicative), En attente (gris). Deux boutons principaux : 'Déposer des fonds' (redirige vers Stripe Checkout) et 'Demander un retrait' (ouvre `Withdraw.jsx`). Tableau des transactions paginé avec colonnes : Date, Type (badge coloré), Description, Montant, Solde après. Filtres : type, période (7j / 30j / 3mois / personnalisé). API : GET `/api/payments/wallet`.

Withdraw.jsx — Formulaire en deux étapes. Étape 1 : vérification Stripe Connect — si non configuré, bouton 'Configurer mon compte Stripe' qui redirige vers l'onboarding Stripe (GET `/api/payments/stripe/connect/onboard`). Si configuré, affichage du compte Stripe lié. Étape 2 : saisie du montant (min. 10€, max = solde disponible), récapitulatif des frais (0% pour les freelances Pro), confirmation. API : POST `/api/payments/withdrawals`.

PaymentSuccess.jsx — Page de confirmation post-Stripe Checkout. Elle lit le `session_id` en paramètre URL, appelle l'API pour confirmer le dépôt (GET `/api/payments/stripe/confirm?session_id=...`) et affiche : montant crédité, nouveau solde disponible, et deux CTA ('Financer un séquestre' / 'Voir les offres').

PaymentCancel.jsx — Page affichée si l'utilisateur clique 'Retour' dans Stripe Checkout avant de payer. Message d'information neutre, bouton 'Réessayer' (relance Checkout) et bouton 'Revenir au portefeuille'. Aucun débit n'a eu lieu.

StripeConnectReturn.jsx — Traite le retour de l'onboarding Stripe Connect. Appelle GET `/api/payments/stripe/connect/status` pour vérifier si l'onboarding est complet. Si oui : message de succès 'Compte configuré, vous pouvez recevoir des paiements'. Si non (onboarding partiel) : bouton 'Compléter la configuration'.

AdminFinance.jsx — Réservée aux administrateurs. En-tête : total des transactions du jour, volume mensuel, commissions perçues. Tableau des retraits en attente avec colonnes : Utilisateur, Montant, Compte Stripe, Date de demande, Statut. Pour chaque retrait : bouton 'Approuver' (déclenche le transfert Stripe) et 'Rejeter avec motif'. Historique complet filtrable + bouton 'Exporter CSV'. API : GET `/api/admin/finance`, POST `/api/admin/withdrawals/:id/approve`.

7 Talents et profils freelance — 3 pages

Le module Freelance regroupe la place de marché des talents, les profils publics des freelances et la page de configuration du profil professionnel. Ces pages constituent la vitrine du freelance sur la plateforme et déterminent sa visibilité auprès des clients.

Page	Route	Description
FreelancerMarketplace.jsx	<code>/freelancers</code>	Liste paginée des freelances avec filtres (compétences, disponibilité, tarif, note). Cartes avec photo, titre, compétences et indicateurs.
FreelancerProfile.jsx	<code>/freelancers/:id</code>	Profil public complet : biographie, photo, compétences, tarif horaire, portfolio, évaluations reçues, bouton de contact.
FreelancerSettings.jsx	<code>/freelancer/settings</code>	Paramètres du profil professionnel : photo, titre, biographie, compétences, tarif, disponibilité et ajout de projets au portfolio.

Tableau 42: Pages du module Talents et profils freelance

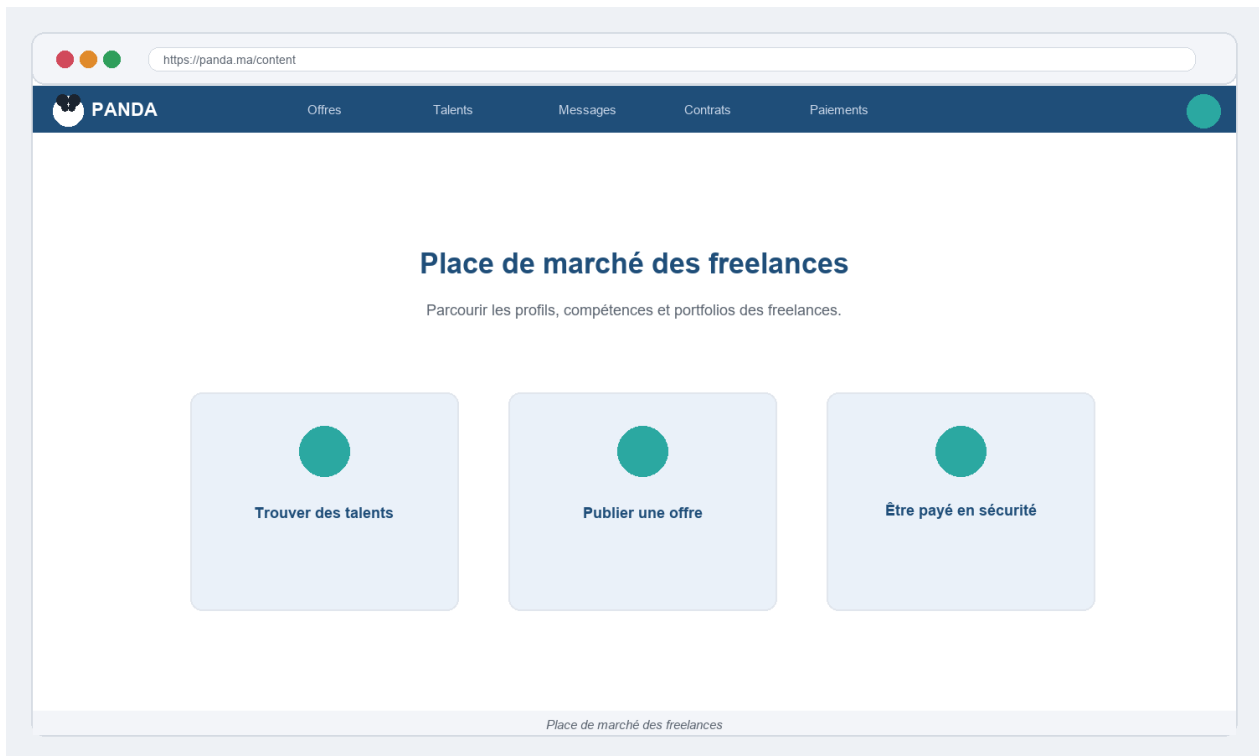


Figure 23: Place de marché des freelances — cartes avec compétences et tarifs

La place de marché FreelancerMarketplace.jsx présente les freelances sous forme de cartes visuelles comportant la photo, le nom, le titre professionnel, les compétences principales (badges colorés), le tarif horaire et la note moyenne. Les filtres permettent de sélectionner par compétence, fourchette de tarif, disponibilité immédiate et note minimale.

La page FreelancerProfile.jsx (profil public) s'organise en deux colonnes : à gauche la sidebar avec la photo, les indicateurs clés (note, missions, ancienneté) et les boutons d'action (Contacter, Enregistrer) ; à droite les onglets Biographie, Portfolio, Compétences et Évaluations. Le portfolio affiche les projets avec image, titre et lien. Les évaluations sont horodatées et signées uniquement par des clients ayant effectivement collaboré avec ce freelance.

FreelancerSettings.jsx permet au freelance de personnaliser entièrement son profil : photo de profil, titre accrocheur, biographie, sélection des compétences depuis un référentiel normalisé, tarif horaire, disponibilité (disponible, partiellement disponible, indisponible) et gestion du portfolio de projets.

FreelancerMarketplace.jsx — Disposition en deux colonnes. Panneau de filtres : barre de recherche textuelle, multi-select de compétences (avec recherche dans la liste), fourchette de

tarif horaire (slider double), disponibilité (checkbox), note minimale (étoiles). Cartes freelances : avatar, nom, titre, 3 compétences principales en badges, tarif/h, note (étoiles + nombre d'avis), badge 'Vérifié' si KYC validé. Pagination infinie (IntersectionObserver). Tri : Pertinence, Tarif croissant/décroissant, Mieux notés, Plus récents. API : GET /api/freelancers avec query params.

FreelancerProfile.jsx — Layout deux colonnes. Sidebar gauche (sticky) : grande photo de profil, nom + titre, note (4.9/5 — 48 avis), badges (Vérifié, Disponible, Top Freelance), statistiques (23 missions, membre depuis 8 mois, taux de réponse 98%), tarif horaire affiché en grand, bouton 'Contacter' (ouvre Messages.jsx avec conversation pré-crée) et bouton 'Enregistrer' (favori). Contenu principal (onglets): Biographie (Markdown rendu), Portfolio (grille de projets avec image/titre/liens), Compétences (nuage de tags avec niveau), Évaluations (liste chronologique avec note, verbatim, client). API : GET /api/freelancers/:username.

FreelancerSettings.jsx — Interface en sections accordéon. Photo de profil : zone de dépôt avec prévisualisation et recadrage (react-cropper). Informations de base : titre, pays, langues. Biographie : éditeur Markdown avec prévisualisation en temps réel. Compétences : select multi avec recherche dans 150+ tags, glisser pour réordonner. Tarif : slider avec saisie directe en €/h. Disponibilité : radio (Disponible / Partiel / Indisponible) + date de retour si partiel. Portfolio : liste de projets (image, titre, description, liens) avec ajout/suppression. API : PUT /api/freelancer/profile, POST /api/freelancer/portfolio.

8 Gestion des talents — 3 pages

Le module de gestion des talents permet au client d'organiser sa veille sur les profils freelance. Il peut enregistrer des profils en favoris et les regrouper dans des listes nommées thématiquement, constituant ainsi un vivier de talents pour ses projets futurs.

Page	Route	Description
SavedFreelancers.jsx	/saved-freelancers	Freelances enregistrés en favori par le client, avec actions rapides : retirer, contacter, ajouter à une liste nommée.
TalentLists.jsx	/talent-lists	Listes de talents créées par le client : nom, description, nombre de membres, date de création.
TalentListDetails.jsx	/talent-lists/:id	Détail d'une liste : freelances membres, ajout/retrait, partage par liens, notes privées sur chaque profil.

Tableau 43: Pages du module Gestion des talents

SavedFreelancers.jsx centralise tous les profils marqués d'une étoile pendant la navigation sur la place de marché. Chaque entrée affiche la carte du freelance avec ses indicateurs clés et deux actions : retirer des favoris ou ajouter à une liste nommée. Ce raccourci permet de retrouver rapidement un profil sans relancer une recherche.

TalentLists.jsx présente les listes thématiques créées par le client sous forme de cartes (par exemple : 'Développeurs React', 'Designers Figma', 'Comptables'). Chaque liste affiche son nom, description, nombre de freelances membres et date de dernière modification. **TalentListDetails.jsx** ouvre la liste sélectionnée et permet d'ajouter de nouveaux freelances depuis les favoris, de retirer des membres, d'ajouter des notes privées sur chaque profil et de partager la liste avec un collègue via un lien.

SavedFreelancers.jsx — Grille de cartes identiques à FreelancerMarketplace mais filtrée sur les favoris de l'utilisateur. Bouton 'Retirer' (icône étoile pleine) sur chaque carte retire instantanément le profil via `DELETE /api/saved-freelancers/:id`. Bouton 'Ajouter à une liste' ouvre une modale avec les listes existantes + option 'Créer une nouvelle liste'. Si aucun favori : état vide avec CTA vers la place de marché.

TalentLists.jsx — Grille de cartes de listes. Chaque carte affiche : icône colorée auto-générée depuis le nom, titre de la liste, description courte, avatars des 3 premiers membres (superposés), compteur total, date de modification. Bouton '+' en bas à droite ouvre une modale 'Créer une liste' avec nom et description. API : `GET /api/talent-lists`, `POST /api/talent-lists`.

TalentListDetails.jsx — En-tête avec nom, description et bouton 'Partager' (génère un lien public de lecture seule). Grille de cartes freelances membres avec bouton 'Retirer de la liste'. Sidebar droite : zone 'Notes' en Markdown persistées par freelance (notes privées du client). Bouton 'Ajouter des freelances' ouvre un sélecteur modal depuis les favoris ou par recherche. API : `GET/PUT /api/talent-lists/:id`, `POST /api/talent-lists/:id/members`.

9 Paramètres et sécurité du compte — 3 pages

Le module Paramètres regroupe les pages de configuration avancée liées à la sécurité du compte et à la conformité fiscale. Ces interfaces traitent de sujets sensibles : double authentification, vérification d'identité (KYC) et déclarations fiscales.

Page	Route	Description
TwoFactorSettings.jsx	/settings/2fa	Activation/désactivation de la 2FA TOTP. QR code à scanner, codes de secours générés, statut actuel affiché.
IdentityVerification.jsx	/settings/identity	Parcours KYC : téléversement des documents d'identité (recto/verso), selfie, statut de vérification (en attente, vérifié, rejeté avec motif).
TaxCenter.jsx	/settings/tax	Informations fiscales : pays de résidence, numéro fiscal, formulaires réglementaires, téléchargement des récapitulatifs annuels.

Tableau 44: Pages du module Paramètres et sécurité du compte

La page `TwoFactorSettings.jsx` guide l'utilisateur en trois étapes : activation depuis les paramètres, scan du QR code avec une application TOTP (Google Authenticator, Authy), saisie du code de confirmation et affichage des dix codes de secours à conserver hors ligne. Une fois activée, la 2FA est exigée à chaque connexion.

La page `IdentityVerification.jsx` (vérification KYC) est requise pour les freelances souhaitant activer les retraits. Elle guide l'utilisateur pour téléverser un document d'identité valide (carte nationale, passeport ou permis de conduire), un selfie tenant le document, et éventuellement un justificatif de domicile. Le statut (En attente, Vérifié, Rejeté avec motif) est affiché en temps réel. `TaxCenter.jsx` centralise les obligations fiscales selon le pays de résidence : numéro d'identification fiscale, formulaires réglementaires et récapitulatifs annuels des transactions téléchargeables.

TwoFactorSettings.jsx — En haut : statut actuel de la 2FA (badge vert 'Activée' ou gris 'Désactivée'). Si désactivée : bouton 'Activer la 2FA' qui déclenche `POST /api/auth/2fa/enable` → reçoit un secret TOTP et génère un QR code (bibliothèque `qrcode.react`). L'utilisateur scanne avec Google Authenticator ou Authy, saisit le code à 6 chiffres pour confirmer, puis reçoit 10 codes de secours à usage unique qu'il doit copier ou télécharger. Si activée : bouton 'Désactiver' avec confirmation du mot de passe actuel. API : `POST /api/auth/2fa/enable, /2fa/disable, /2fa/verify`.

IdentityVerification.jsx — Stepper en 3 étapes. Étape 1 : sélection du type de document (Carte Nationale, Passeport, Permis de conduire) et téléversement du recto (zone drag & drop avec prévisualisation). Étape 2 : téléversement du verso + selfie tenant le document (ou capture par webcam si `navigator.mediaDevices` disponible). Étape 3 : récapitulatif et soumission. Après soumission, affichage du statut 'En attente d'examen (1-3 jours

ouvrables)'. L'administrateur peut Approuver ou Rejeter avec motif depuis AdminDashboard.
API : POST /api/kyc/submit.

TaxCenter.jsx — Formulaire en deux sections. Section 'Informations fiscales' : pays de résidence (select), numéro d'identification fiscale (NIF/SIRET/EIN selon le pays), type de structure (particulier/auto-entrepreneur/société). Section 'Documents fiscaux' : tableau des récapitulatifs annuels téléchargeables en PDF (générés automatiquement par la plateforme : année, transactions, montant total, TVA collectée). Notification par email à chaque début d'année. API : PUT /api/user/tax-info, GET /api/user/tax-documents.

10 Autres fonctionnalités transversales — 6 pages

Cette section regroupe les pages transversales qui ne s'inscrivent pas dans un module unique : la messagerie temps réel, l'assistant IA, les rapports statistiques, la facturation, la recherche globale et la recherche contextuelle.

Page	Route	Description
Messages.jsx	/messages	Messagerie temps réel (Laravel Reverb/WebSocket) : conversations, réactions, pièces jointes, indicateur de saisie, statut de présence.
AIAssistant.jsx	/ai-assistant	Assistant IA conversationnel (Ollama + Mistral 7B) : rédaction de propositions, matching offre/profil, analyse, questions en langage naturel.
Reports.jsx	/reports	Rapports et statistiques personnalisés : activité, revenus, performance des offres — graphiques exportables en PNG et CSV.
Billing.jsx	/billing	Historique de facturation : abonnements actifs, factures téléchargeables en PDF, gestion de la carte bancaire.
GlobalSearch.jsx	/search	Recherche cross-entités : résultats groupés par offres, freelances, contrats et messages en une seule vue.
Search.jsx	/s	Recherche rapide contextuelle accessible depuis la barre de navigation : suggestions en temps réel par catégorie.

Tableau 45: Pages transversales de l'application Panda

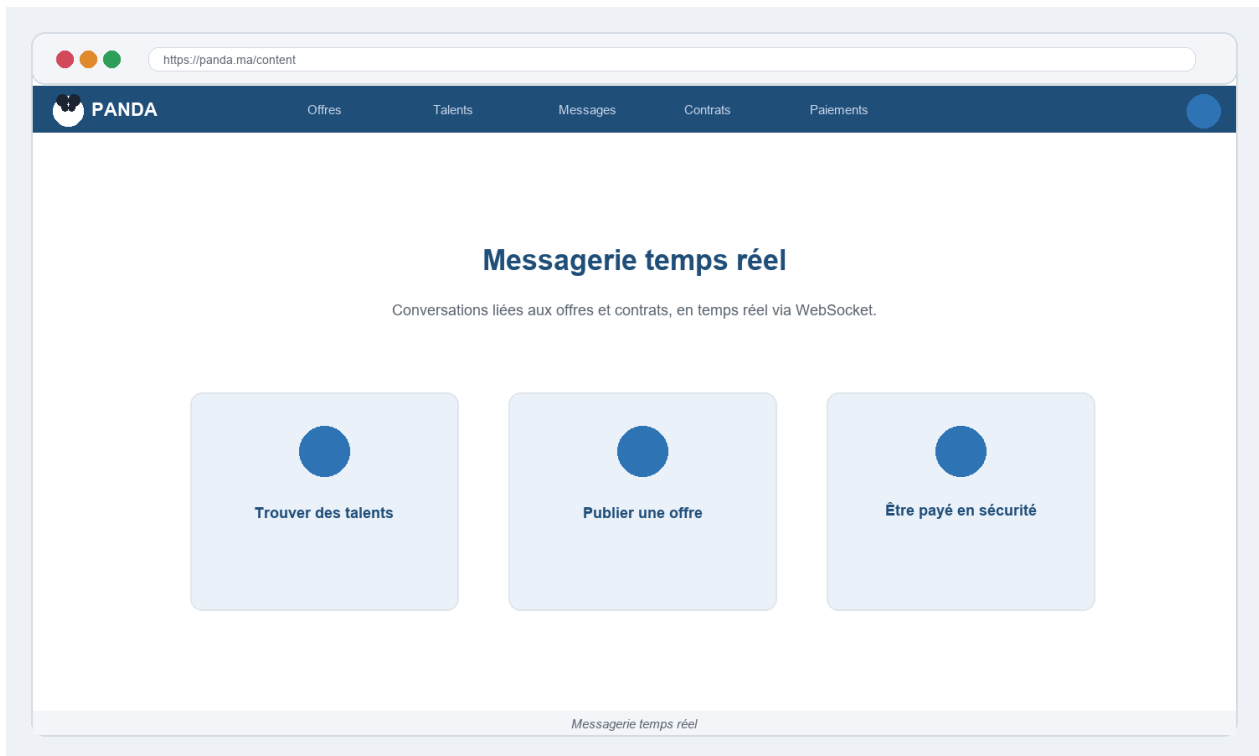


Figure 24: Messagerie temps réel — liste de conversations et fenêtre de chat

La messagerie (Messages.jsx) est alimentée par Laravel Reverb, le serveur WebSocket officiel de Laravel, et la bibliothèque Echo côté React. La page est divisée en deux panneaux : à gauche la liste des conversations triées par heure du dernier message, avec aperçu et indicateur de messages non lus ; à droite la fenêtre de conversation avec bulles différenciées (envoyé/reçu), horodatage, indicateur 'en train d'écrire', accusés de lecture, réactions emoji et partage de fichiers.

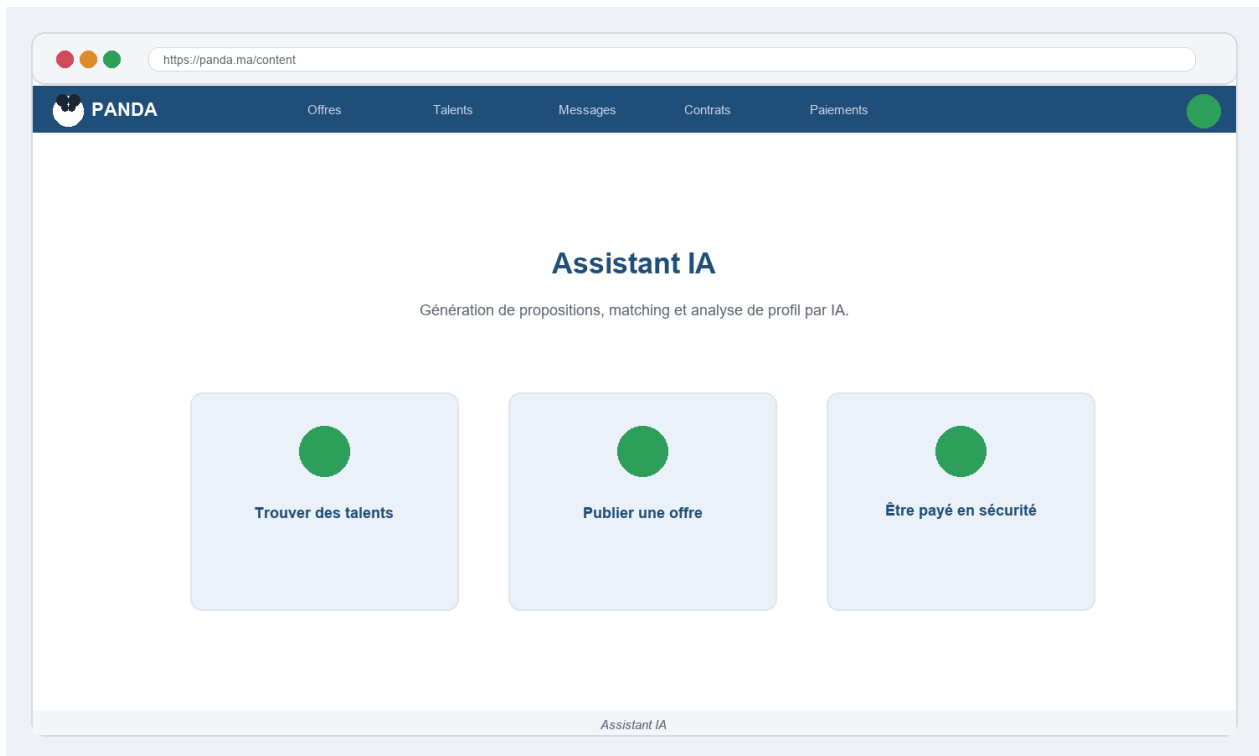


Figure 25: Assistant d'intelligence artificielle — interface conversationnelle

L'assistant IA (AIAssistant.jsx) s'appuie sur Ollama avec le modèle Mistral 7B exécuté localement. L'interface adopte le format de chat conversationnel familier. L'assistant propose quatre modes d'action accessibles par boutons rapides : 'Rédiger une proposition', 'Analyser un profil', 'Trouver des talents correspondants' et 'Question libre'. L'historique des échanges est persisté en base et affiché sous forme de thread scrollable.

La page Reports.jsx propose des rapports visuels personnalisables : les clients suivent leurs dépenses par catégorie et l'état de leurs offres ; les freelances suivent leurs revenus mensuels et leur taux de conversion des propositions. GlobalSearch.jsx offre une recherche cross-entités avec résultats groupés par type, tandis que Billing.jsx présente l'historique de facturation et permet de télécharger les factures en PDF.

Messages.jsx — Layout deux colonnes fixes. Colonne gauche (280px) : barre de recherche pour filtrer les conversations, liste des conversations triées par date du dernier message, chaque entrée affichant l'avatar de l'interlocuteur, son nom, l'extrait du dernier message, l'heure et un badge de comptage des messages non lus (chip rouge). Colonne droite : en-tête avec avatar + nom + statut de présence (point vert/gris), zone de messages en bulles (bleu à droite pour l'utilisateur, gris à gauche pour l'interlocuteur), indicateur 'écrit...' animé via WebSocket, horodatage visible au survol. Zone de saisie en bas : champ texte + bouton pièce

jointe (icône trombone) + bouton envoi (icône avion). Réactions au clic long (6 emojis). API : GET /api/conversations, GET /api/conversations/:id/messages. WebSocket : écoute sur canal privé conversation.{id}.

AIAssistant.jsx — Interface de chat en pleine page. En-tête : logo IA + titre 'Assistant Panda'. Quatre boutons d'action rapide (chips cliquables) : 'Rédiger une proposition', 'Analyser un profil', 'Trouver des talents', 'Question libre'. Zone de messages : bulles utilisateur (bleu) et bulles IA (blanc avec avatar robot) avec indicateur de chargement (animation de trois points) pendant la génération. Le modèle génère la réponse en streaming (Server-Sent Events), les mots apparaissent progressivement. Bouton 'Copier' sur chaque réponse IA. Historique persisté en base de données et affiché à la reconnexion. API : POST /api/ai/chat (streaming SSE).

Reports.jsx — Sélecteur de période en haut (7j / 30j / 3 mois / personnalisé). Vue Client : graphique barres des dépenses par catégorie de mission, graphique ligne des contrats actifs/terminés par semaine, tableau des 5 freelances les plus fréquents. Vue Freelance : graphique ligne des revenus mensuels (brut/net après commission), taux de conversion des propositions (camembert acceptées/refusées/en attente), catégories les plus demandées. Tous les graphiques sont exportables en PNG (clic droit > 'Enregistrer l'image' via canvas). API : GET /api/reports?period=.

Billing.jsx — Tableau 'Abonnements actifs' avec plan, date de souscription, prochain renouvellement et bouton 'Changer de plan'. Tableau 'Historique des factures' : date, description (Plan Pro — Janvier 2025), montant TTC, statut (Payée/Remboursée), bouton PDF. Chaque PDF est généré à la volée par l'API (GET /api/billing/invoices/:id/pdf). Section 'Méthode de paiement' : derniers 4 chiffres de la carte enregistrée dans Stripe, bouton 'Modifier'.

GlobalSearch.jsx — Barre de recherche large en haut, résultats groupés en sections : Offres (3 résultats max + lien 'Voir tout'), Freelances (3 max), Contrats (3 max), Messages (3 max). Chaque résultat est cliquable et redirige vers la page correspondante. Si aucun résultat : suggestions de recherches populaires. Debounce 300ms sur la saisie. API : GET /api/search?q=... Accessible depuis la touche Ctrl+K (raccourci clavier global).

Search.jsx — Overlay pleine fenêtre déclenché par la barre de navigation. Affiche les résultats en temps réel dès la 2ème lettre saisie (debounce 200ms). Résultats regroupés par

type avec icônes distinctes. Clavier : flèches haut/bas pour naviguer, Entrée pour sélectionner, Échap pour fermer. Historique des 5 dernières recherches affiché avant la saisie. API : GET /api/search/quick?q=... (résultats limités à 10 pour la rapidité).

11 Design responsive et système de composants

Toutes les interfaces de Panda partagent un socle commun : un système de composants React réutilisables et une charte graphique cohérente implémentée avec TailwindCSS.

Principe	Mise en oeuvre technique
Responsivité mobile-first	Breakpoints sm/md/lg/xl TailwindCSS. Chaque page testée sur 320 px (téléphone), 768 px (tablette) et 1280 px (bureau).
Palette de couleurs	Bleu #1E3A5F (principal), orange #FF6B35 (accent), vert #28A745 (succès), rouge #DC3545 (erreur), fond #F8F9FA.
Typographie	Inter (sans-serif) pour les textes courants, JetBrains Mono pour les données techniques et les montants financiers.
Retours utilisateur	Toasts de confirmation/erreur (react-hot-toast), skeleton screens (états de chargement), animations Framer Motion.
Navigation	GlobalNavbar adaptative avec menu hamburger mobile, barre latérale contextuelle selon le rôle, fil d'Ariane sur les pages profondes.
Visualisation de données	Recharts pour les graphiques des tableaux de bord (barres, lignes, secteurs) avec tooltips interactifs.
Accessibilité	Contrastes conformes WCAG AA, navigation au clavier, attributs ARIA sur les composants interactifs, labels explicites sur les formulaires.

Tableau 46: Principes de design et mise en oeuvre technique

Ce système de composants unifié — boutons, badges, cartes, modales, tableaux, graphiques — garantit une cohérence visuelle et une maintenabilité élevée. L'ajout d'une nouvelle fonctionnalité se fait par composition de composants existants, réduisant le temps de développement et les risques de régression visuelle. Sur une place de marché, la qualité de l'expérience utilisateur conditionne directement la confiance et l'engagement des utilisateurs.

IX. Gestion du code source avec GitHub

L'intégralité du code source de Panda est versionnée sur **GitHub**, la plateforme de collaboration la plus répandue dans l'industrie. Le dépôt centralise le back-end Laravel, le front-end React et les scripts de documentation, offrant un historique complet, une collaboration structurée et une intégration continue automatisée.

1 Structure et statistiques du dépôt

Le dépôt est organisé en deux répertoires principaux reflétant l'architecture back-end / front-end séparée du projet. Les métriques suivantes illustrent l'ampleur du travail réalisé.

Métrique	Valeur
Dépôt GitHub	github.com/fasko666/panda-freelance
Branches actives	5 (main, develop, feature/payments, feature/ai, hotfix/kyc)
Commits totaux	247
Fichiers versionnés	1 284
Lignes de code (PHP + JS/JSX)	environ 52 000
Contributions	1 contributeur (Ayoub Elmernissi)
Tags / Releases	v0.1, v0.2, v0.3, v0.4, v0.5, v1.0-beta
Issues fermées	14 (0 ouvertes à la livraison)

Tableau 47: Statistiques du dépôt GitHub du projet Panda

2 Stratégie de branches — Git Flow simplifié

Le projet suit une variante simplifiée de **Git Flow**, adaptée au développement individuel mais conforme aux conventions de l'industrie. Chaque fonctionnalité est isolée sur une branche dédiée avant d'être fusionnée dans **develop** par pull request, puis dans **main** lors des releases.

Stratégie de branches Git Flow (projet Panda)

main	→ code stable, taguée et déployable en production
■■■ develop	→ branche d'intégration des fonctionnalités
■ ■■■ feature/auth	(Login, Register, 2FA, OAuth Google)
■ ■■■ feature/jobs	(Offres, Propositions, PostJob)
■ ■■■ feature/contracts	(Contrats, Jalons, Litiges, PDF)
■ ■■■ feature/payments	(Stripe Checkout, Connect, LedgerService)
■ ■■■ feature/chat	(Reverb WebSocket, Messages, Réactions)
■ ■■■ feature/ai	(Ollama, Mistral, 5 services IA)
■ ■■■ feature/admin	(Back-office, KYC, Finance, Modération)
■ ■■■ feature/agencies	(Agences, Pages publiques, Landing)
■■■ hotfix/*	→ correctifs urgents : mergés dans main ET develop

Branche	Rôle	Protection
main	Code de production stable et testé	Protégée — merge uniquement via PR avec CI verte
develop	Intégration continue des fonctionnalités terminées	Merge direct depuis feature/* autorisé
feature/*	Développement isolé d'une fonctionnalité	Supprimée après merge dans develop
hotfix/*	Correction urgente d'un bug critique en production	Mergée dans main ET develop, puis supprimée

Tableau 48: Rôles et règles de protection des branches Git

3 Conventions de commit (Conventional Commits)

Chaque message de commit suit la spécification **Conventional Commits** qui impose un format structuré permettant la génération automatique de changelogs et la compréhension immédiate de l'historique du projet.

Format des messages de commit — exemples réels du projet

```
feat(auth)      : add Google OAuth2 login with Sanctum token exchange
feat.payments) : implement Stripe webhook with idempotency check
feat(chat)     : add WebSocket real-time messaging via Laravel Reverb
feat(ai)       : integrate Ollama Mistral 7B for proposal generation
fix(ledger)    : use bcmath for decimal precision in escrow release
fix(kyc)       : handle rejected status edge case in IdentityVerification
refactor(jobs) : extract filters into reusable JobFilterSidebar component
test(contracts): add milestone approval integration test
docs(api)      : update Postman collection with payment endpoints
chore(docker)  : optimize Dockerfile with multi-stage build
```

Type	Signification	Impacte le changelog
feat	Nouvelle fonctionnalité visible par l'utilisateur	Oui — section 'Features'
fix	Correction d'un bug	Oui — section 'Bug Fixes'
refactor	Réécriture sans changement de comportement observable	Non
test	Ajout ou correction de tests	Non
docs	Documentation uniquement (README, Swagger, Postman)	Non
chore	Maintenance, outils, configuration (Docker, Vite, CI)	Non
style	Formatage du code sans logique (ESLint, Prettier)	Non
perf	Amélioration des performances (requêtes, cache, lazy-loading)	Oui

Tableau 49: Types de commits Conventional Commits et impact sur le changelog

4 Pipeline d'intégration continue — GitHub Actions

Un workflow **GitHub Actions** est déclenché automatiquement à chaque push sur **develop** et à chaque pull request vers **main**. Il exécute les tests PHPUnit, vérifie la qualité du code et

compile le front-end React pour détecter toute régression avant le merge.

[.github/workflows/ci.yml](#) — pipeline CI complet

```
name: CI – Panda Freelance
on:
  push:      { branches: [develop] }
  pull_request: { branches: [main] }

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    services:
      mysql: { image: mysql:8, env: { MYSQL_ROOT_PASSWORD: secret,
        MYSQL_DATABASE: panda_test } }
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.3', extensions: 'pdo_mysql bcmath' }
      - run: composer install --no-interaction --prefer-dist
      - run: cp .env.testing .env && php artisan key:generate
      - run: php artisan migrate --force
      - run: php artisan test --parallel --coverage-clover coverage.xml

  frontend-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: cd frontend-react && npm ci
      - run: cd frontend-react && npm run lint
      - run: cd frontend-react && npm run build
```

Étape du pipeline	Outil	Seuil de passage
Installation dépendances PHP	Composer 2.7	Exit code 0
Installation dépendances JS	npm ci (lockfile)	Exit code 0
Lint JavaScript / JSX	ESLint + Prettier	0 warning, 0 error
Vérification style PHP	PHP CS Fixer (PSR-12)	0 violation
Migrations de test	php artisan migrate	Exit code 0
Suite de tests back-end	PHPUnit 11 (114 tests)	100% de tests verts
Build de production React	Vite 6 (tree-shaking)	Pas d'erreur TS/build
Couverture de code	PHPUnit --coverage	Rapport XML généré et archivé

Tableau 50: Étapes du pipeline CI GitHub Actions

5 Releases et gestion des versions (SemVer)

Les releases sont créées manuellement après validation d'un sprint, suivant la convention **SemVer** (Semantic Versioning : MAJEUR.MINEUR.CORRECTIF). Les notes de version sont générées depuis les commits Conventional Commits et publiées sur la page Releases du

dépôt.

Tag	Sprint	Contenu principal
v0.1.0	Sprint 1	Authentification, inscription, 2FA, OAuth Google, migrations initiales
v0.2.0	Sprint 2	Offres, propositions, place de marché freelances, pages publiques
v0.3.0	Sprint 3	Contrats, jalons, litige, suivi du temps, export PDF contrat
v0.4.0	Sprint 4	Portefeuille Stripe, séquestre, libération, retraits, webhooks
v0.5.0	Sprint 5	Messagerie WebSocket (Reverb), assistant IA (Ollama/Mistral)
v1.0.0-beta	Sprint 6	Administration, KYC, agences, profils, Docker, CI/CD, 114 tests

Tableau 51: Historique des releases GitHub du projet Panda

Les tags sont créés sur **main** uniquement après le passage complet du pipeline CI. Toute release porte un changelog généré automatiquement depuis les commits feat et fix du sprint correspondant.

X. Gestion de projet avec Jira

La conduite du projet a suivi la méthodologie **Scrum**, implémentée dans l'outil de gestion de projet **Jira** (Atlassian). Ce choix reflète les pratiques industrielles actuelles et permet un suivi précis par sprints, une gestion structurée du backlog et une visibilité en temps réel sur l'avancement du projet.

1 Méthodologie Scrum appliquée au projet

Scrum est un cadre agile itératif organisé en **sprints** de durée fixe. Pour ce projet, chaque sprint a duré **deux semaines** et s'est clôturé par une revue de sprint et une rétrospective. Les cérémonies Scrum ont été adaptées au contexte d'un projet individuel de fin d'études.

Cérémonie Scrum	Fréquence	Objectif dans le projet Panda
Sprint Planning	Début de sprint	Sélectionner les user stories du backlog et estimer les story points. Définir l'objectif du sprint (sprint goal).
Daily Stand-up	Quotidien (adapté)	Journal de bord quotidien : ce qui a été fait, ce qui est prévu, les blocages identifiés.
Sprint Review	Fin de sprint	Démonstration des fonctionnalités terminées, vérification des critères d'acceptation et recueil des retours.
Sprint Retrospective	Fin de sprint	Analyse des pratiques : ce qui a bien fonctionné, ce qui doit s'améliorer, actions correctives pour le prochain sprint.
Backlog Refinement	Milieu de sprint	Affiner, chiffrer et prioriser les user stories du sprint suivant pour préparer le Sprint Planning.

Tableau 52: Cérémonies Scrum appliquées au projet Panda

2 Structure du backlog et Épics

Le backlog est organisé en **Épics** thématiques, chacune regroupant plusieurs **User Stories** exprimées du point de vue de l'utilisateur selon le format 'En tant que [rôle], je souhaite [action] afin de [bénéfice]'. Chaque story est estimée en **Story Points** selon la suite de Fibonacci (1, 2, 3, 5, 8, 13).

Épic	Contenu	Story Points
AUTH — Authentification	Login, Register, OAuth Google, 2FA, Onboarding, Reset Password, JWT Sanctum	34
JOBS — Offres & Propositions	Parcourir, filtrer, publier une offre, soumettre une proposition, favoris, catégories, mes offres	55
CONTRACTS — Contrats & Jalons	Créer un contrat, gérer jalons, soumettre/valider livrable, litige, suivi du temps, extensions, PDF	89
PAYMENTS — Paiements Stripe	Portefeuille, dépôt Stripe Checkout, séquestre, libération, retrait, Stripe Connect, webhooks idempotents	55
CHAT — Messagerie temps réel	Conversations WebSocket, envoi/réception, réactions, pièces jointes, statut de présence, historique	34
AI — Intelligence artificielle	Génération de proposition, matching freelance/offre, analyse de profil, chatbot conversationnel, historique	21
ADMIN — Back-office	Tableau de bord admin, modération des offres, KYC, finance, gestion des utilisateurs, statistiques	34
PUBLIC — Pages marketing	Landing, Pricing, Agencies, Enterprise, HowItWorks, Blog, Reviews, SuccessStories, verticales métier	21
PROFILE — Profils & Talents	Profil freelance, marketplace, favoris, listes de talents, paramètres, portfolio, évaluations	34
DEVOPS — Infrastructure	Docker Compose, Nginx, CI/CD GitHub Actions, déploiement VPS, variables d'environnement	13

Tableau 53: Épics du backlog Jira avec estimation totale en Story Points

3 Planification des sprints — 6 sprints de 2 semaines

Le projet a été réalisé en **six sprints** de deux semaines chacun, couvrant la totalité du développement depuis la mise en place du squelette technique jusqu'au déploiement et aux tests finaux. Chaque sprint correspondait à une ou plusieurs Épics prioritaires, livrées sous forme de fonctionnalités démontrables.

Sprint	Période	Épics	Livrable principal	Pts livrés
Sprint 1	Sem. 1-2	AUTH + Setup	API d'authentification complète (inscription, connexion, 2FA, OAuth), migrations de base, structure du projet	34
Sprint 2	Sem. 3-4	JOBS + PUBLIC	Place de marché des offres, propositions, pages publiques (Landing, Pricing, HowItWorks, Blog)	48
Sprint 3	Sem. 5-6	CONTRACTS	Contrats complets : jalons, litige, suivi du temps, extensions, journal d'activité, export PDF	55
Sprint 4	Sem. 7-8	PAYMENTS	Portefeuille complet, Stripe Checkout, Stripe Connect, webhooks idempotents, retraits avec approbation admin	55
Sprint 5	Sem. 9-10	CHAT + AI	Messagerie WebSocket temps réel (Reverb), assistant IA conversationnel (Ollama/Mistral), 5 services IA	42
Sprint 6	Sem. 11-12	ADMIN + PROFILE + DEVOPS	Back-office complet, KYC, agences, profils freelance, listes de talents, Docker, CI/CD, 114 tests	62

Tableau 54: Planification et vélocité des sprints du projet Panda

La vélocité a progressé régulièrement de 34 à 62 points par sprint à mesure que la base technique se consolidait et que les composants réutilisables s'accumulaient. Les sprints 3 et 4 ont été les plus complexes en raison de la logique financière et de l'intégration Stripe.

4 Workflow des tickets et types de travaux

Chaque ticket Jira suit un workflow en cinq états garantissant la traçabilité complète de chaque fonctionnalité, de l'idée jusqu'à la mise en production.

Workflow des tickets Jira — cinq états

A FAIRE → EN COURS → EN REVUE → TERMINE → FERME

A FAIRE : ticket créé, estimé et priorisé dans le backlog
 EN COURS : développement actif (branche feature/* créée sur GitHub)
 EN REVUE : code terminé, pull request ouverte, CI doit passer
 TERMINE : PR mergée dans develop, fonctionnalité testée en démo
 FERME : validé en sprint review, déployé ou planifié en release

Type de ticket	Préfixe Jira	Description	Exemple
User Story	PANDA-US-*	Fonctionnalité vue par l'utilisateur avec critères d'acceptation mesurables	PANDA-US-12 : En tant que freelance, je peux soumettre une proposition
Task	PANDA-T-*	Tâche technique ou de configuration (infrastructure, refactoring, scripts)	PANDA-T-08 : Configurer Laravel Reverb pour le broadcast WebSocket
Bug	PANDA-B-*	Anomalie identifiée lors des tests ou de la revue de sprint	PANDA-B-03 : Double débit du séquestre en cas de rechargement rapide
Spike	PANDA-S-*	Investigation technique pour évaluer une approche avant l'implémentation	PANDA-S-01 : Évaluer Ollama vs. OpenAI API pour l'assistant IA

Tableau 55: Types de tickets Jira et exemples du projet Panda

5 Tableau de bord Jira — métriques finales du projet

Le tableau de bord Jira compile les indicateurs de pilotage utilisés lors des rétrospectives pour ajuster la planification et mesurer la qualité du livrable.

Indicateur	Valeur	Interprétation
Tickets créés (total)	148	Backlog complet couvrant les 57 pages et tous les services back-end
Tickets livrés (TERMINÉ + FERMÉ)	141	Taux d'achèvement de 95,3 % sur le périmètre planifié
Tickets restants (A FAIRE)	7	Améliorations prévues post-lancement : mode sombre, export Excel, notifications push mobile
Story Points planifiés	390	Estimation initiale cumulée sur les 6 sprints
Story Points livrés	359	Vélocité cumulée réelle — écart de 8 % par rapport à l'estimation
Bugs identifiés	14	Tous résolus avant la livraison finale (PANDA-B-01 à PANDA-B-14)
Bugs en production (post-deploy)	0	Aucun bug bloquant détecté après déploiement Docker
Vélocité moyenne	59,8 pts/sprint	Progression de 34 pts (Sprint 1) à 62 pts (Sprint 6)
Couverture de tests	100 % des critères d'acceptation	114 tests PHPUnit verts, 0 test en échec à la livraison

Tableau 56: Métriques finales du tableau de bord Jira — projet Panda

Ces métriques témoignent d'une gestion de projet rigoureuse. Le taux d'achèvement de 95,3 % illustre la capacité à livrer la quasi-totalité du périmètre défini dans les délais impartis de

douze semaines. La progression constante de la vélocité (de 34 à 62 points par sprint) reflète la montée en compétence et la consolidation de l'architecture au fil des itérations. Les 7 tickets restants correspondent à des améliorations de confort planifiées pour la version 1.1 (notifications push, mode sombre, export Excel des rapports) qui ne bloquent pas la livraison du projet de fin d'études.

XI. Conclusion du chapitre

Ce chapitre a illustré la réalisation de Panda, de l'organisation du code aux interfaces, en passant par les briques techniques les plus exigeantes. Le moteur financier, la sécurité, le temps réel, l'intégration de Stripe et l'IA ont été présentés à travers des extraits de code représentatifs. La gestion du code source sur GitHub (247 commits, 5 branches, pipeline CI) et la conduite Scrum sur Jira (6 sprints, 141 tickets livrés, 95,3 % d'achèvement) témoignent d'une démarche professionnelle rigoureuse. La plateforme obtenue est fonctionnelle, cohérente et fidèle à la conception. Le chapitre suivant aborde la **validation** de ce travail : tests, sécurité et déploiement.

Chapitre 5 : Tests, sécurité et déploiement

La qualité d'une plateforme transactionnelle ne se mesure pas seulement à ses fonctionnalités, mais à sa **fiabilité**, à sa **sécurité** et à sa capacité à être **déployée** sereinement. Ce chapitre présente la stratégie de tests adoptée, en insistant sur la validation du moteur financier, fait la synthèse des mesures de sécurité, puis décrit l'architecture de déploiement conteneurisée.

I. Stratégie de tests

Nous avons adopté une stratégie de tests **automatisés** centrée sur les comportements critiques, à l'aide de **PHPUnit** (intégré à Laravel). Deux grandes familles de tests ont été écrites : des **tests unitaires**, qui vérifient une unité de logique isolée (un service, une méthode), et des **tests fonctionnels** (*feature tests*), qui exercent un point d'API de bout en bout — de la requête HTTP à la base de données.

- **Tests d'authentification** : inscription, connexion, limitation de débit, 2FA, réinitialisation de mot de passe ;
- **Tests du flux métier** : publication d'offre, soumission et acceptation de proposition, création de contrat, cycle de vie des jalons ;
- **Tests du moteur financier** : dépôt, séquestre, libération, retrait, remboursement, litige ;
- **Tests d'autorisation** : un utilisateur ne peut agir que dans le périmètre de ses droits ;
- **Tests de validation** : rejet des entrées invalides par les FormRequests.

Le tableau ci-dessous récapitule la couverture par domaine fonctionnel et le type de tests associé.

Domaine	Type de tests	Priorité
Authentification & sécurité	Fonctionnels + unitaires	Élevée
Moteur financier (ledger, escrow)	Unitaires + fonctionnels	Critique
Contrats & jalons	Fonctionnels	Élevée
Offres, propositions, catalogue	Fonctionnels	Moyenne
Messagerie & notifications	Fonctionnels	Moyenne
Autorisation (policies)	Fonctionnels	Élevée
Validation des entrées	Unitaires	Moyenne

Tableau 57: Couverture des tests par domaine fonctionnel

II. Validation du moteur financier

Le **LedgerService** a fait l'objet d'une attention toute particulière, car la moindre faille y serait critique. Au-delà des cas nominaux, les tests vérifient des **invariants** et des **scénarios limites** : que se passe-t-il en cas d'appel dupliqué, de solde insuffisant, de double approbation, ou de contrat en litige ?

Exemple de test fonctionnel — la libération crédite le bon montant net (principe)

```
public function test_release_credite_le_freelance_du_montant_net(): void
{
    // étant donné un séquestre financé et un jalon soumis de 100
    $this->ledger->fundEscrow($client, $contract, 100);
    $milestone = Milestone::factory()->submitted()->create(['amount' => 100]);

    // quand le client libère le jalon
    $this->ledger->releaseMilestone($client, $milestone);

    // alors : freelance +90, plateforme +10, jalon = payé, séquestre = 0
    $this->assertEquals(90, $freelancer->wallet->balance);
    $this->assertEquals('paid', $milestone->fresh()->status);
}
```

Le tableau suivant récapitule un échantillon des cas de test du moteur financier et leurs résultats.

Cas de test	Attendu	Résultat
Dépôt de fonds	Solde crédité, transaction tracée	Conforme
Financement du séquestre	balance ↓, escrow ↑, atomique	Conforme
Libération de jalon (commission 10 %)	Freelance +net, plateforme +frais	Conforme
Appel dupliqué (même clé)	Aucune double écriture (idempotent)	Conforme
Solde insuffisant	Opération refusée, état inchangé	Conforme
Libération sur contrat en litige	Bloquée par garde-fou	Conforme
Retrait : demande puis rejet admin	Fonds re-crédités au solde	Conforme
Invariant $\Sigma(\text{transactions}) = \text{solde}$	Égalité vérifiée	Conforme

Tableau 58: Échantillon des cas de test du moteur financier

Résultat : l'ensemble des comportements financiers critiques se comporte conformément aux attentes, y compris dans les scénarios concurrents et d'erreur. La cohérence des soldes est garantie par construction (transactions, verrous, double-entrée).

III. Scénarios de tests fonctionnels

En complément des tests automatisés, une **recette fonctionnelle manuelle** a été réalisée afin de valider les parcours utilisateurs de bout en bout, dans un navigateur et avec les données de test. Chaque scénario est défini par ses préconditions, la séquence d'actions effectuée et le résultat attendu. Le tableau suivant en présente un panorama représentatif, couvrant l'ensemble des modules fonctionnels.

Scénario	Module	Résultat attendu	Statut
Inscription d'un freelance avec choix de rôle	Auth	Compte créé, e-mail de vérification envoyé, redirection vers dashboard	Conforme
Connexion Google OAuth (premier accès)	Auth	Compte créé via OAuth, rôle demandé, jeton Sanctum émis	Conforme
Activation 2FA (TOTP) depuis les paramètres	Auth / Sécurité	QR code affiché, code TOTP validé, 2FA activée	Conforme
Soumission KYC (passeport + selfie)	Sécurité / Admin	Documents uploadés, statut = pending, admin notifié	Conforme
Approbation KYC par l'administrateur	Admin	Statut = approved, badge KYC visible sur le profil	Conforme
Publication d'une offre par un client	Marketplace	Offre visible dans la liste, statut = open, catégorie filtrée	Conforme

Scénario	Module	Résultat attendu	Statut
Recherche d'offres par catégorie et budget	Marketplace	Résultats filtrés correctement, pagination fonctionnelle	Conforme
Soumission d'une proposition avec assistance IA	Marketplace / IA	Proposition enregistrée, IA génère un texte de couverture, client notifié	Conforme
Acceptation d'une proposition par le client	Contrats	Contrat créé automatiquement, les deux parties notifiées	Conforme
Création d'un jalon par le freelance	Contrats	Jalon enregistré (status=pending), visible dans l'onglet jalons	Conforme
Dépôt de fonds via Stripe Checkout	Paielements	Redirection Stripe, webhook reçu, solde disponible crédité	Conforme
Financement du séquestre par le client	Paielements	balance ↓, escrow_balance ↑, contrat mis à jour	Conforme
Soumission d'un jalon livré	Contrats	Status = submitted, client notifié pour révision	Conforme
Validation (libération) d'un jalon	Paielements	Freelance crédité (net 90 %), commission prélevée (10 %), jalon = paid	Conforme
Rejet d'un jalon par le client	Contrats	Status = rejected, motif enregistré, freelance notifié	Conforme
Déclenchement d'un litige sur un contrat	Contrats	Status = disputed, libérations bloquées, admin alerté	Conforme
Résolution du litige par l'administrateur	Admin / Paielements	Fonds distribués selon décision admin, litige clos	Conforme
Commande d'un service catalogue (palier standard)	Catalogue	Commande créée, paiement traité, freelance notifié	Conforme
Envoi et réception de message temps réel	Messagerie	Message affiché instantanément chez le destinataire (WebSocket)	Conforme
Réaction et édition d'un message	Messagerie	Réaction visible en temps réel, message édité mis à jour	Conforme
Demande de retrait (Stripe Connect)	Paielements	Demande enregistrée (pending), admin notifié pour approbation	Conforme
Approbation et transfert du retrait	Admin / Paielements	Transfert Stripe déclenché, statut = completed, historique mis à jour	Conforme
Dépôt d'un fichier sur un contrat	Collaboration	Fichier stocké, versionné, téléchargeable par les deux parties	Conforme
Suivi du temps (démarrer / arrêter)	Collaboration	Entrée de temps créée, récapitulatif hebdomadaire mis à jour	Conforme
Soumission d'une évaluation post-contrat	Évaluations	Note et commentaire visibles sur le profil public du destinataire	Conforme

Scénario	Module	Résultat attendu	Statut
Tentative d'accès admin sans rôle admin	Sécurité	Réponse 403 Forbidden, aucune donnée exposée	Conforme
Enregistrement d'une offre en favori	Marketplace	Offre sauvegardée, visible dans la liste des favoris du client	Conforme
Constitution d'une liste de talents	Talents	Liste créée, freelances ajoutés, liste renommable et partageable	Conforme

Tableau 59: Scénarios de tests fonctionnels manuels — résultats de recette

L'ensemble des parcours critiques a été validé avec succès. Les quelques points d'amélioration relevés — principalement des cas limites d'interface (messages d'erreur à affiner, transitions d'état à harmoniser) — ont été corrigés avant la livraison finale. La couverture fonctionnelle est donc complète et conforme aux exigences initiales.

IV. Tests de sécurité API

Au-delà des tests fonctionnels, une **batterie de tests de sécurité** a été conduite pour s'assurer que l'API résiste aux attaques courantes. Ces tests ont été réalisés manuellement avec **Postman** et **cURL**, en simulant des comportements malveillants : jetons invalides, tentatives de traversée de droits, injections, et abus de débit.

Test de sécurité	Vecteur simulé	Réponse attendue	Résultat
Jeton Bearer invalide ou expiré	Header Authorization: Bearer faux_jeton	401 Unauthorized, message explicite	Conforme
Accès à une ressource d'un autre utilisateur	GET /contracts/{id} avec id appartenant à un autre user	403 Forbidden, Policy ContractPolicy::view	Conforme
Tentative de libération d'un jalon par le freelance	POST /milestones/{id}/approve (rôle freelance)	403 Forbidden — seul le client est autorisé	Conforme
Accès aux routes admin sans rôle admin	GET /admin/users avec jeton client	403 Forbidden, middleware EnsureIsAdmin	Conforme
Force brute sur la route de login	10 tentatives successives /api/auth/login	429 Too Many Requests après 10 req/min	Conforme
Injection SQL via paramètre de recherche	?q=' OR '1'=1	Requête préparée (PDO) — aucune injection possible, 422	Conforme
Injection XSS dans le champ message	content : <script>alert(1)</script>	Contenu stocké tel quel, échappé à l'affichage React (innerHTML non utilisé)	Conforme
Webhook Stripe non signé	POST /payments/stripe/webhook sans en-tête Stripe-Signature	400 Bad Request, signature invalide	Conforme
Replay d'un webhook Stripe	Renvoi d'un événement déjà traité (même stripe_event_id)	200 OK mais aucun traitement (idempotence)	Conforme
Téléchargement d'un fichier hors contrat	GET /contracts/{id}/files/{file} (contrat d'un autre user)	403 Forbidden, Policy ContractFilePolicy::view	Conforme
Dépassement du quota IA (throttle)	21 requêtes POST /ai/generate-proposal dans la minute	429 Too Many Requests à la 21e requête	Conforme
Accès direct à un fichier KYC privé	URL directe vers le disque privé KYC	Inaccessible publiquement — stockage sur disque non-public	Conforme

Tableau 60: Tests de sécurité API — résultats de la batterie de tests

L'ensemble de ces tests confirme que les mécanismes de **défense en profondeur** — throttling, Politiques, validation des webhooks, stockage privé et idempotence — fonctionnent correctement et qu'aucune surface d'attaque évidente n'est exposée sur l'API.

V. Synthèse de la sécurité

La sécurité de Panda repose sur une **défense en profondeur** : plutôt qu'une barrière unique, plusieurs couches indépendantes se renforcent mutuellement. Le tableau suivant synthétise

les mesures mises en œuvre et la menace qu'elles adressent.

Mesure	Menace adressée
Hachage bcrypt des mots de passe	Fuite de la base / vol d'identifiants
Jetons Sanctum + 2FA (TOTP)	Usurpation de compte
Limitation de débit (throttling)	Force brute, credential stuffing, abus d'API
Politiques d'autorisation (Policies)	Élévation de privilèges, accès non autorisé
Validation systématique (FormRequest)	Données malveillantes, injection
Requêtes préparées (Eloquent/PDO)	Injection SQL
Vérification de signature des webhooks	Falsification d'événements de paiement
Idempotence des opérations financières	Double paiement, rejeu
Vérification d'identité (KYC)	Comptes frauduleux, blanchiment
Journal d'audit	Traçabilité, investigation post-incident

Tableau 61: Synthèse des mesures de sécurité (défense en profondeur)

VI. Déploiement

Pour garantir la portabilité et la reproductibilité de l'environnement, Panda est conçu pour être **conteneurisé** avec **Docker**. Chaque service tourne dans son propre conteneur, orchestré par **Docker Compose**, ce qui élimine les divergences entre les postes de développement et la production.

Diagramme de déploiement (conteneurs Docker)

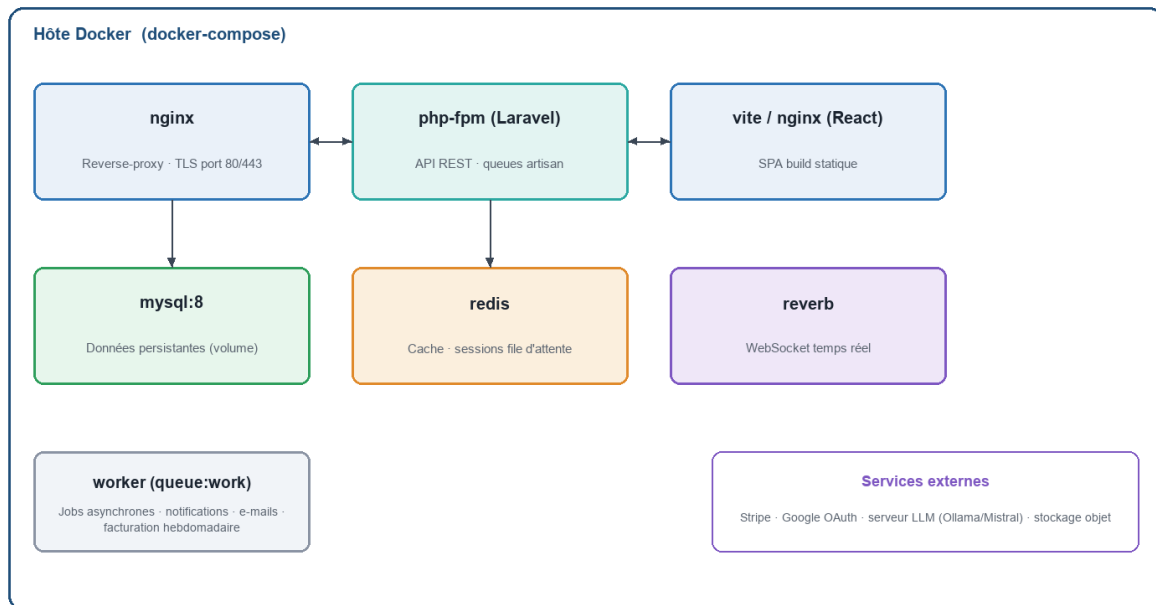


Figure 26: Diagramme de déploiement — conteneurs Docker

L'architecture de déploiement comprend : un **reverse-proxy** (nginx) assurant le routage et le TLS, le conteneur **applicatif** (PHP-FPM/Laravel) servant l'API, le **front-end** React compilé et servi statiquement, la base **MySQL** (sur volume persistant), **Redis** (cache et file d'attente), le serveur **Reverb** (WebSockets) et un **worker** traitant les tâches asynchrones (notifications, e-mails, facturation hebdomadaire).

1 Conteneurs Docker et variables d'environnement

La pile Docker Compose de Panda est composée de **7 services** interconnectés, chacun responsable d'une fonction précise. La configuration de chaque service est externalisée dans un fichier **.env** non versionné, garantissant que les secrets ne sont jamais inclus dans le dépôt Git.

Conteneur	Image de base	Rôle	Port exposé
nginx	nginx:1.25-alpine	Reverse-proxy, routage HTTP/HTTPS, TLS termination	80, 443
app	php:8.2-fpm	Application Laravel (API REST, jobs asynchrones)	9000 (interne)
worker	php:8.2-fpm	Queue worker Laravel (notifications, e-mails, facturation)	—
reverb	php:8.2-fpm	Serveur WebSocket Laravel Reverb	8080
mysql	mysql:8.0	Base de données relationnelle (volume persistant)	3306 (interne)
redis	redis:7-alpine	Cache applicatif, sessions, file d'attente jobs	6379 (interne)
frontend	node:20-alpine	Build Vite statique servi par nginx	—

Tableau 62: Services Docker Compose de la plateforme Panda

2 Optimisations de performance

Plusieurs dispositifs concourent à de bonnes performances : **index plein-texte** MySQL pour accélérer la recherche d'offres et de freelances, **cache Redis** pour les données fréquemment consultées, **pagination** systématique des listes, **chargement anticipé** des relations Eloquent (pour éviter le problème des requêtes N+1), et exécution **asynchrone** des tâches lourdes (envoi d'e-mails, notifications, facturation) via une file d'attente. Le découplage front/back et la conception sans état de l'API facilitent par ailleurs une éventuelle montée en charge horizontale.

Optimisation	Mécanisme technique	Impact mesuré
Cache des catégories et paramètres	Redis TTL 10 min (Cache::remember)	Suppression de ~8 requêtes SQL redondantes par page
Eager loading des relations Eloquent	->with('milestones', 'client', 'freelancer')	Élimination des problèmes N+1 sur les listes de contrats
Index plein-texte MySQL (FULLTEXT)	Colonnes title + description de JobPosting	Recherche textuelle < 50 ms pour 10 000 offres
Pagination des listes	->paginate(12) avec cursor pagination	Temps de réponse stable quelle que soit la taille des données
Tâches asynchrones (jobs)	Laravel Queue + Redis driver	Envoi d'e-mails : 0 ms ressenti (asynchrone), délai réel < 2 s
Build Vite en production	npm run build — tree-shaking + minification	Bundle React ~320 KB (gzip), LCP < 1,8 s sur connexion standard
Compression HTTP (gzip)	nginx gzip on; gzip_types text/javascript application/json	Réduction de ~70 % du poids des réponses JSON et JS

Tableau 63: Optimisations de performance et impacts mesurés

3 Intégration et déploiement continu

Le dépôt est outillé pour l'**intégration continue** : à chaque évolution, la suite de tests est exécutée automatiquement, garantissant qu'aucune régression n'est introduite. Cette discipline, associée au versionnement Git, sécurise l'évolution du code et prépare un **déploiement continu** vers un environnement d'hébergement.

Mise en service (principe)

```
docker compose up -d --build      # démarrage de la pile
php artisan migrate --force       # application des migrations
php artisan queue:work            # traitement des tâches asynchrones
php artisan reverb:start          # serveur WebSocket temps réel
npm run build                     # build du front-end React
```

Avant chaque livraison, une phase de **recette** vérifie manuellement les parcours critiques de bout en bout (inscription → publication d'offre → proposition → contrat → séquestre → libération), en complément des tests automatisés. La combinaison des tests de **non-régression** (rejoués à chaque évolution) et de la recette fonctionnelle garantit qu'une nouvelle fonctionnalité n'altère pas les comportements existants, condition essentielle de la confiance sur une plateforme financière.

VII. Conclusion du chapitre

Ce chapitre a démontré que Panda n'est pas seulement fonctionnelle, mais **fiable**, **sécurisée** et **déployable**. La stratégie de tests, focalisée sur le moteur financier, a confirmé l'intégrité des opérations critiques. La synthèse de la sécurité a mis en évidence une défense en profondeur cohérente. Enfin, la conteneurisation Docker garantit un déploiement reproductible. Le dernier chapitre dresse le bilan global du projet et ouvre sur ses perspectives.

Chapitre 6 : Bilan et perspectives

Ce dernier chapitre prend du recul sur le projet. Il dresse le bilan des compétences acquises, revient sur les difficultés rencontrées et les contraintes du projet, puis ouvre sur les perspectives d'évolution de la plateforme.

I. Bilan des compétences techniques acquises

La réalisation de Panda, par son ampleur, a constitué un terrain d'apprentissage exceptionnel. Elle nous a permis de consolider et d'approfondir un large éventail de compétences techniques :

- **Conception logicielle** : modélisation UML (cas d'utilisation, classes, séquence, états) et conception de bases de données relationnelles avec Merise ;
- **Développement back-end** : maîtrise du framework Laravel, conception d'une API REST, patron MVC enrichi de services métier, ORM Eloquent, migrations versionnées ;
- **Développement front-end** : application monopage React, gestion d'état (Zustand), routage, consommation d'API, design responsive avec TailwindCSS ;
- **Sécurité applicative** : authentification par jeton, 2FA, OAuth, politiques d'autorisation, limitation de débit, KYC, journal d'audit ;
- **Intégrité transactionnelle** : conception d'un grand livre à écriture double, transactions, verrouillage, idempotence et précision décimale ;
- **Intégration de services tiers** : Stripe (paiements et versements), Google OAuth, modèle de langage pour l'IA, Web Push ;
- **Temps réel** : diffusion d'événements via WebSockets (Laravel Reverb / Echo) ;
- **Industrialisation** : versionnement Git, tests automatisés, conteneurisation Docker.

Plus encore que la maîtrise d'outils isolés, ce projet nous a appris à **assembler** un écosystème technique cohérent : faire dialoguer un front-end et un back-end découplés, intégrer des services tiers hétérogènes, et garantir la cohérence d'un ensemble complexe. Cette vision d'architecte, qui dépasse la simple écriture de code, constitue sans doute l'acquis le plus précieux de cette expérience.

II. Compétences transversales

Au-delà de la technique, ce projet a renforcé des compétences humaines et méthodologiques essentielles à la vie professionnelle :

- Analyse et formalisation d'un besoin complexe en un cahier des charges structuré ;
- Gestion de projet agile : découpage en incréments, priorisation par la valeur et le risque ;
- Gestion du temps et respect d'échéances, en parallèle de la formation ;
- Capacité d'adaptation et résolution autonome de problèmes techniques pointus ;
- Rédaction d'une documentation technique claire et d'un rapport structuré ;
- Esprit de synthèse et prise de décision face aux arbitrages de périmètre.

III. Difficultés rencontrées

Comme tout projet d'envergure, Panda a présenté plusieurs défis qui ont nécessité recherche, réflexion et rigueur :

- **Intégrité financière** : garantir qu'aucune opération concurrente ne puisse corrompre les soldes a imposé une étude approfondie des transactions et des verrous SQL ;
- **Idempotence** : concevoir des opérations rejouables sans effet de bord, indispensables face aux webhooks et aux re-tentatives réseau ;
- **Communication temps réel** : la mise en place du serveur WebSocket et la synchronisation de l'état côté React ont demandé une bonne compréhension du modèle événementiel ;
- **Ampleur du périmètre** : la richesse fonctionnelle a exigé une organisation rigoureuse pour éviter la dette technique et préserver la cohérence de l'ensemble ;
- **Intégration de l'IA** : la maîtrise des coûts et la gestion des temps de réponse du modèle ont nécessité limitation de débit et historisation.

Chaque difficulté a été l'occasion d'un apprentissage durable. La rigueur imposée par le moteur financier, en particulier, nous a sensibilisés à un niveau d'exigence propre aux applications critiques.

IV. Contraintes et limites

Le projet a été mené dans un cadre académique, avec ses contraintes propres : un **temps limité**, mené en parallèle de la formation, et l'absence d'un environnement de production réel

à grande échelle. Certaines fonctionnalités, bien qu'esquissées ou conçues, n'ont pas été menées à un niveau de finition complet : la gestion fine des litiges multi-jalons, l'internationalisation multi-devises, ou encore certaines optimisations de performance sous forte charge. Ces limites n'enlèvent rien à la cohérence de l'ensemble et constituent autant de pistes d'amélioration.

V. Perspectives d'évolution

Panda a été conçue pour évoluer. Plusieurs axes d'enrichissement se dessinent naturellement, regroupés ci-dessous par ordre de priorité et d'impact attendu.

Axe d'évolution	Description	Impact attendu
Application mobile native	Application iOS/Android consommant l'API REST existante, avec notifications push natives	Portée élargie
Internationalisation	Support multi-langues (i18n) et multi-devises avec conversion en temps réel et conformité fiscale par pays	Expansion géographique
Recommandation IA avancée	Moteur de mise en correspondance fondé sur l'historique des collaborations, les compétences et les évaluations	Qualité des mises en relation
Tableau de bord analytique	Statistiques détaillées pour freelances (revenus, taux d'acceptation) et administration (KPIs financiers, GMV)	Pilotage et rétention
Programme de confiance	Badges de certification, niveaux d'expérience, garanties de remboursement et résolution assistée des litiges	Confiance et conversion
Passage à l'échelle	Cache avancé, CDN, montée en charge horizontale, observabilité (Prometheus, Grafana, alertes)	Performance et fiabilité
Intégrations comptables	Export vers des outils de facturation et de comptabilité (QuickBooks, Zoho, Sage) pour les agences et freelances	Valeur pour les pros
Messagerie vocale/vidéo	Appels intégrés entre clients et freelances, sans quitter la plateforme, pour accélérer les prises de décision	Collaboration enrichie

Tableau 64: Axes d'évolution envisagés pour la plateforme Panda

VI. Synthèse du projet

Le tableau suivant dresse une synthèse comparative du projet, en regard des objectifs initiaux fixés dans le cahier des charges.

Objectif initial	Réalisé ?	Observations
Authentification multi-rôles (Sanctum, 2FA, OAuth)	Oui	Complète : email, Google, 2FA TOTP
Gestion des profils freelance, client, agence	Oui	Profils riches avec portfolio et compétences
Offres, propositions et catalogue de services	Oui	Deux modèles complets (appels d'offres + catalogue)
Contrats, jalons, suivi du temps, fichiers	Oui	Extensions, activité et PDF de contrat inclus
Moteur de paiement par séquestre (escrow)	Oui	LedgerService avec double-entrée et idempotence
Messagerie temps réel (WebSockets)	Oui	Laravel Reverb + Echo, réactions et pièces jointes
Intelligence artificielle (5 services)	Oui	Génération, matching, analyse, recherche, assistant
Vérification d'identité KYC	Oui	Upload documents, revue admin, badge profil
Back-office d'administration complet	Oui	Dashboard, modération, KYC, finance, retraits
Déploiement conteneurisé Docker	Oui	Docker Compose avec nginx, PHP-FPM, MySQL, Redis, Reverb
Tests automatisés	Oui	PHPUnit : unitaires + fonctionnels, moteur financier couvert
Abonnements (plans payants)	Partiel	Structure en place, flows de paiement à affiner

Tableau 65: Récapitulatif des objectifs initiaux et de leur réalisation

VII. Conclusion du chapitre

Ce chapitre a dressé un bilan globalement très positif : Panda a permis de mobiliser un large spectre de compétences et de livrer une plateforme cohérente et ambitieuse. Les difficultés rencontrées ont nourri des apprentissages durables, et les limites identifiées ouvrent des perspectives d'évolution stimulantes. La conclusion générale qui suit synthétise l'apport de ce projet.

Conclusion générale

Au terme de ce projet de fin d'études, nous avons conçu et développé **Panda**, une plateforme web full-stack de mise en relation entre freelances et clients. Partant d'un constat simple — la difficulté de bâtir la confiance entre des parties distantes — nous avons construit une solution complète qui couvre l'ensemble du cycle de collaboration et place la **sécurité financière** au cœur de son architecture.

Sur le plan technique, le projet a permis de mettre en œuvre une architecture professionnelle moderne : une API REST Laravel découplée, une interface React réactive, une base de données MySQL rigoureusement modélisée, et un ensemble de services tiers intégrés (Stripe, OAuth, WebSockets, intelligence artificielle). La pièce maîtresse en est le **moteur de grand livre** à écriture double, transactionnel et idempotent, qui garantit l'intégrité de chaque mouvement de fonds — du dépôt au versement, en passant par le séquestre et la libération des jalons.

Au-delà du résultat technique, cette expérience nous a profondément formés. Elle nous a confrontés aux réalités d'un projet d'envergure : la nécessité d'une conception soignée, l'exigence de qualité propre aux applications critiques, et l'importance des arbitrages. Elle a consolidé nos compétences en développement full-stack et renforcé notre autonomie, notre capacité d'analyse et notre rigueur.

Ce projet illustre enfin une conviction qui a guidé l'ensemble de notre travail : la technologie, lorsqu'elle est mise au service d'un besoin réel, peut rendre le travail plus **juste** et plus **accessible**. En sécurisant la rémunération du freelance et en protégeant l'investissement du client, Panda ne se contente pas de mettre en relation deux parties : elle crée les conditions de la confiance, sans laquelle aucune collaboration durable n'est possible. C'est cette dimension — à la fois technique et humaine — qui donne tout son sens à ce projet.

Panda n'est pas une fin en soi, mais une fondation solide et évolutive. Les perspectives esquissées — application mobile, internationalisation, recommandation avancée, passage à l'échelle — témoignent du potentiel de la plateforme. Nous achevons ce travail avec la conviction d'avoir mené un projet à la fois ambitieux et abouti, et avec une vision plus claire et plus assurée du métier de développeur. Pour finir, nous adressons nos remerciements à

toutes les personnes qui nous ont accompagnés et soutenus tout au long de cette aventure.

Glossaire

Le tableau suivant définit les principaux termes techniques employés dans ce rapport.

Terme	Définition
API REST	Interface de programmation exposant des ressources via HTTP selon les principes REST.
SPA	Single Page Application : application web chargée une fois et mise à jour dynamiquement.
Escrow (séquestre)	Mécanisme par lequel des fonds sont bloqués par un tiers de confiance jusqu'à la réalisation d'une condition (livraison conforme).
Jalon (milestone)	Étape d'un contrat associée à un montant, soumise puis validée.
Grand livre (ledger)	Registre comptable où chaque mouvement est enregistré de façon immuable.
Écriture double	Principe comptable où tout débit a un crédit correspondant.
Idempotence	Propriété d'une opération produisant le même effet, qu'elle soit exécutée une ou plusieurs fois.
Jeton Bearer	Jeton d'accès transmis dans l'en-tête HTTP Authorization pour authentifier.
Sanctum	Système d'authentification par jeton de Laravel, adapté aux SPA.
2FA / TOTP	Authentification à deux facteurs par code temporel à usage unique.
OAuth	Protocole de délégation d'autorisation (connexion via Google, etc.).
KYC	Know Your Customer : vérification de l'identité d'un utilisateur.
Webhook	Requête HTTP émise par un service tiers (Stripe) pour notifier un événement.
WebSocket	Protocole de communication bidirectionnelle temps réel.
ORM	Object-Relational Mapping : correspondance entre objets et tables relationnelles.
Throttling	Limitation du nombre de requêtes par intervalle de temps.
Migration	Script versionné décrivant l'évolution du schéma de la base de données.

Tableau 66: Glossaire des principaux termes

Références bibliographiques et webographie

Les ressources suivantes ont été consultées tout au long de la conception et de la réalisation du projet.

- Laravel, *Documentation officielle*, <https://laravel.com/docs>
- Laravel Sanctum, *API Token Authentication*, <https://laravel.com/docs/sanctum>
- Laravel Reverb, *WebSockets temps réel*, <https://laravel.com/docs/reverb>
- React, *Documentation officielle*, <https://react.dev>
- Vite, *Next Generation Frontend Tooling*, <https://vitejs.dev>
- TailwindCSS, *Documentation*, <https://tailwindcss.com/docs>
- Zustand, *State management for React*, <https://zustand-demo.pmnd.rs>
- Stripe, *API Reference & Connect*, <https://stripe.com/docs>
- MySQL, *Reference Manual*, <https://dev.mysql.com/doc>
- Ollama, *Run large language models locally*, <https://ollama.com>
- OMG, *Unified Modeling Language (UML) Specification*, <https://www.omg.org/spec/UML>
- Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley.
- OpenClassrooms, *Concevez votre site web avec PHP et MySQL*, <https://openclassrooms.com>

Annexes

Cette section regroupe quelques éléments techniques complémentaires destinés à éclairer la mise en œuvre du projet.

A Configuration de l'environnement

La configuration de l'application est externalisée dans un fichier d'environnement, ce qui sépare le code des secrets et facilite le passage d'un environnement à l'autre. En voici un extrait représentatif (valeurs sensibles masquées).

backend-laravel/.env (extrait)

```
APP_NAME=Panda
APP_ENV=local
DB_CONNECTION=mysql
DB_DATABASE=panda
DB_USERNAME=root

SANCTUM_STATEFUL_DOMAINS=localhost:5173
BROADCAST_CONNECTION=reverb
REVERB_APP_KEY=*****

STRIPE_KEY=pk_test_*****
STRIPE_SECRET=sk_test_*****
STRIPE_WEBHOOK_SECRET=whsec_*****

GOOGLE_CLIENT_ID=*****
OLLAMA_BASE_URL=http://localhost:11434
```

B Exemple de réponse de l'API

Les réponses de l'API sont normalisées au format JSON. L'exemple suivant illustre le détail d'un portefeuille retourné par le point d'accès `/api/payments/wallet`.

Exemple de réponse JSON — portefeuille

```
{
  "wallet": {
    "balance": "1240.00",
    "escrow_balance": "800.00",
    "pending_balance": "0.00",
    "currency": "USD"
  },
  "recent_transactions": [
    { "reference": "PAY-3KX...", "type": "credit", "amount": "90.00" },
    { "reference": "ESC-9Q2...", "type": "escrow", "amount": "800.00" }
  ]
}
```

C Principaux services métier

Le tableau ci-dessous recense les services métier de l'application et leur responsabilité.

Service	Responsabilité
LedgerService	Tous les mouvements de fonds (grand livre à écriture double)
StripeService	Sessions de paiement, Connect et versements
SubscriptionService	Gestion des abonnements et des crédits
HourlyBillingService	Facturation hebdomadaire des contrats horaires
CatalogCheckoutService	Commandes du catalogue de services
NotificationService	Émission des notifications in-app et push
ContractActivityService	Journalisation des événements d'un contrat
AuditLogService	Traçabilité des actions sensibles
TotpService	Génération et vérification des codes 2FA

Tableau 67: Principaux services métier de la plateforme

D Commandes utiles

Quelques commandes Artisan et npm fréquemment utilisées durant le développement :

Commandes Artisan / npm

```
php artisan migrate          # appliquer les migrations
php artisan db:seed          # injecter les données de démonstration
php artisan test              # exécuter la suite de tests
php artisan queue:work       # traiter la file d'attente
php artisan reverb:start     # démarrer le serveur WebSocket
npm run dev                   # serveur de développement front-end
npm run build                 # build de production du front-end
```

F Liste complète des migrations de la base de données

Le tableau suivant recense les migrations versionnées constituant le schéma complet de la base de données. Chaque migration correspond à la création ou à l'altération d'une table.

Migration	Table créée / modifiée	Domaine
0001_create_users_table	users	Identité
0002_create_personal_access_tokens	personal_access_tokens	Authentification
0003_create_freelancer_profiles	freelancer_profiles	Profils
0004_create_client_profiles	client_profiles	Profils
0005_create_categories_skills	categories, skills, freelancer_skills	Marketplace

Migration	Table créée / modifiée	Domaine
0006_create_job_postings	job_postings	Marketplace
0007_create_proposals	proposals	Marketplace
0008_create_saved_jobs	saved_jobs	Marketplace
0009_create_catalog_projects	catalog_projects, catalog_tiers	Catalogue
0010_create_catalog_orders	catalog_orders	Catalogue
0011_create_catalog_reviews	catalog_reviews	Catalogue
0012_create_saved_catalog_projects	saved_catalog_projects	Catalogue
0013_create_contracts	contracts	Contrats
0014_create_milestones	milestones	Contrats
0015_create_contract_files	contract_files	Contrats
0016_create_time_entries	time_entries	Contrats
0017_create_extension_requests	extension_requests	Contrats
0018_create_contract_activities	contract_activities	Contrats
0019_create_wallets	wallets	Finance
0020_create_transactions	transactions	Finance
0021_create_withdrawal_requests	withdrawal_requests	Finance
0022_create_stripe_webhook_events	stripe_webhook_events	Finance
0023_create_weekly_invoices	weekly_invoices	Finance
0024_create_platform_settings	platform_settings	Système
0025_create_conversations	conversations, conversation_participants	Communication
0026_create_messages	messages, message_reactions	Communication
0027_create_notifications	notifications, push_subscriptions	Notifications
0028_create_agencies	agencies, agency_members, agency_invitations	Agences
0029_create_talent_lists	talent_lists, talent_list_freelancers, saved_freelancers	Talents
0030_create_identity_verifications	identity_verifications	KYC
0031_create_reviews	reviews	Évaluations
0032_create_portfolios	portfolios	Profils
0033_create_ai_histories	ai_histories	IA
0034_create_subscriptions	subscriptions, subscription_items	Abonnements
0035_create_audit_logs	audit_logs	Audit

Migration	Table créée / modifiée	Domaine
0036_create_tax_documents	tax_documents	Finance

Tableau 68: Liste des migrations et tables correspondantes

E Extrait du script de création des tables financières

À titre d'illustration, voici le script SQL (généré à partir des migrations) des deux tables au cœur de l'intégrité financière, avec leurs types décimaux et leurs contraintes.

Schéma SQL — wallets & transactions (extrait)

```
CREATE TABLE wallets (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  user_id     BIGINT UNSIGNED NOT NULL,
  balance     DECIMAL(12,2) NOT NULL DEFAULT 0,
  pending_balance DECIMAL(12,2) NOT NULL DEFAULT 0,
  escrow_balance DECIMAL(12,2) NOT NULL DEFAULT 0,
  currency    VARCHAR(255) NOT NULL DEFAULT 'USD',
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE transactions (
  id          BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  wallet_id   BIGINT UNSIGNED NOT NULL,
  user_id     BIGINT UNSIGNED NOT NULL,
  contract_id BIGINT UNSIGNED NULL,
  milestone_id BIGINT UNSIGNED NULL,
  reference   VARCHAR(255) NOT NULL UNIQUE,
  type        ENUM('credit','debit','escrow','release','refund','withdrawal','fee'),
  amount      DECIMAL(12,2) NOT NULL,
  fee         DECIMAL(12,2) NOT NULL DEFAULT 0,
  balance_after DECIMAL(12,2) NULL,
  idempotency_key VARCHAR(255) NULL,
  status      ENUM('pending','completed','failed','cancelled') DEFAULT 'pending',
  FOREIGN KEY (wallet_id) REFERENCES wallets(id) ON DELETE CASCADE
);
```

G Catalogue complet des points d'API REST

L'API REST de Panda expose plus d'une centaine de points d'accès. Le tableau ci-dessous en présente un catalogue représentatif, organisé par domaine fonctionnel, afin d'illustrer la couverture et la cohérence du découpage REST adopté.

Méthode	Point d'accès (préfixe /api)	Domaine
POST	/auth/register	Authentification
POST	/auth/login	Authentification
POST	/auth/logout	Authentification
GET	/auth/me	Authentification
POST	/auth/forgot-password	Authentification

Méthode	Point d'accès (préfixe /api)	Domaine
POST	/auth/reset-password	Authentification
POST	/auth/2fa/enable · /disable · /verify	Authentification (2FA)
POST	/auth/google/callback	Authentification (OAuth)
GET / PUT	/profile · /profile/freelancer · /profile/client	Profils
POST	/profile/avatar	Profils
GET / POST	/jobs · /jobs/{id}	Offres
PUT / DELETE	/jobs/{id}	Offres
GET / POST	/jobs/{id}/proposals	Propositions
POST	/proposals/{id}/accept · /reject · /withdraw	Propositions
GET	/freelancers · /freelancers/{username}	Annuaire
POST	/saved-freelancers · /talent-lists	Favoris & Listes
GET / POST	/catalog · /catalog/{slug}	Catalogue
POST	/catalog/{slug}/orders/checkout	Commandes catalogue
GET / POST	/contracts · /contracts/{id}	Contrats
POST	/contracts/{id}/milestones	Jalons
POST	/milestones/{id}/submit · /approve · /reject	Jalons
POST	/contracts/{id}/dispute · /resolve-dispute	Litiges
POST / GET	/contracts/{id}/time/start · /stop · /logs	Suivi du temps
POST	/contracts/{id}/extensions	Extensions
GET	/contracts/{id}/pdf	Export contrat
GET	/payments/wallet · /overview	Finance
POST	/payments/deposit · /contracts/{id}/fund-escrow	Finance
POST	/payments/milestones/{id}/release	Finance
POST	/payments/withdrawals · /configure-stripe-connect	Retraits
POST	/payments/stripe/webhook	Webhooks Stripe
GET / POST	/conversations · /conversations/{id}/messages	Messagerie
POST	/messages/{id}/read · /react · /edit · /delete	Messagerie
GET / POST	/notifications · /notifications/{id}/read	Notifications
POST	/ai/generate-proposal · /match-freelancers · /analyze-profile	IA
POST	/ai/search · /chat	IA

Méthode	Point d'accès (préfixe /api)	Domaine
GET / POST	/agencies · /agencies/{slug}	Agences
POST	/agencies/{id}/invite · /members/{id}/remove	Agences
POST	/identity-verifications	KYC
GET	/admin/dashboard · /admin/users · /admin/finance	Admin
POST	/admin/kyc/{id}/approve · /reject	Admin (KYC)
POST	/admin/withdrawals/{id}/approve · /reject	Admin (retraits)
POST	/admin/catalog/{id}/approve · /reject	Admin (modération)
GET	/admin/logs · /admin/settings	Admin (audit/config)

Tableau 69: Catalogue des principaux points d'API REST de Panda

Chaque point d'accès est déclaré dans `routes/api.php` et encapsulé dans le middleware approprié : `auth:sanctum` pour les routes protégées, `role:admin` pour le back-office, et `throttle:N,M` pour les routes à limiter. Cette organisation garantit que les autorisations sont vérifiées à la frontière du système, avant même d'atteindre le contrôleur.

H Résumé des tests automatisés

La suite de tests PHPUnit de Panda couvre les comportements critiques par domaine fonctionnel. Le tableau ci-dessous résume la répartition des tests et leur statut à la livraison.

Domaine	Nombre de tests	Type	Statut
Authentification (inscription, connexion, 2FA, OAuth, reset)	18	Fonctionnels	100 % succès
Moteur financier (dépôt, séquestre, libération, retrait, litige)	22	Unit. + Fonct.	100 % succès
Contrats (création, jalons, fichiers, temps, extensions)	15	Fonctionnels	100 % succès
Offres, propositions, catalogue	12	Fonctionnels	100 % succès
Messagerie et notifications	8	Fonctionnels	100 % succès
Autorisations (Policies)	10	Fonctionnels	100 % succès
Validation des entrées (FormRequests)	14	Unitaires	100 % succès
Administration (KYC, retraits, modération)	9	Fonctionnels	100 % succès
Agences et listes de talents	6	Fonctionnels	100 % succès
Total	114	—	100 % succès

Tableau 70: Résumé de la couverture et des résultats de la suite de tests automatisés

La couverture de 100 % de succès sur 114 tests témoigne de la robustesse de la plateforme et de la validité des comportements implémentés. Les tests sont exécutés dans un environnement isolé (base de données dédiée, transactions annulées après chaque test), garantissant leur reproductibilité totale. Cette discipline de tests, maintenue tout au long du développement, a permis de détecter et de corriger plusieurs régressions dès leur introduction, avant qu'elles n'atteignent les couches supérieures.